Fully Homomorphic Encryption with Chosen-Ciphertext Security from LWE

Rupeng Yang*, Zuoxia Yu, and Willy Susilo

Institute of Cybersecurity and Cryptology, School of Computing and Information Technology, University of Wollongong, Wollongong NSW, Australia
rupengy@uow.edu.au, zyu@uow.edu.au, wsusilo@uow.edu.au

Abstract. We construct (1-hop) fully homomorphic encryption (FHE) schemes with chosen-ciphertext (CCA) security from the learning with errors (LWE) assumption in the standard model. Security of our construction only relies on the circular-secure LWE, which matches the assumptions needed for FHE with the basic chosen-plaintext security. Besides, the scheme achieves a security notion that is strictly stronger than the CCA1 security. Prior FHE schemes with even just CCA1 security require either the random oracle model or non-falsifiable assumptions.

The construction follows the well-known Naor-Yung double encryption paradigm. However, unlike previous works [Boneh et al., ITCS 2012; Canetti et al., PKC 2017; Manulis and Nguyen, Eurocrypt 2024], which employ general zero-knowledge succinct non-interactive arguments of knowledge (ZK-SNARKs), we design a special succinct argument to prove the validity of FHE ciphertexts. The succinct argument is constructed from batch arguments for NP and a new primitive called *predicate extractable commitment*, which may be of independent interest.

1 Introduction

Fully homomorphic encryption (FHE) schemes [RAD78, Gen09b, Gen09a] enable one to perform homomorphic evaluations over encrypted data without decrypting them. Generally, it can be viewed as a special public-key encryption (PKE) scheme with an additional evaluation algorithm: given a circuit $\mathtt{C}$ and ciphertexts that encrypt messages $m_1, \ldots, m_l$, the algorithm outputs an evaluated ciphertext that can be decrypted to $\mathtt{C}(m_1, \ldots, m_l)$. To rule out trivial constructions that simply include the circuit in the evaluated ciphertext and push the computation to the decryption algorithm, the ciphertext size and the decrypting time is required to be *compact*, which can be upper-bounded by a predefined polynomial in the security parameter.

The existence of a FHE scheme was a long standing open problem and this was changed in 2009 when Gentry proposed the first candidate construction of FHE from lattices [Gen09b, Gen09a]. Then in a long line of works (e.g., [BV11b, BV11a, BGV12, GSW13, BV14, ASP14]), this problem was (nearly) settled:

* Corresponding author.

- Assuming the standard learning with errors (LWE) assumption [Reg05], we can construct FHE schemes that can evaluate circuits of an a-priori bounded depth. The schemes are usually denoted as leveled FHE [Gen09a, BGV12].
- If we additionally assume the circular security of the above LWE-based (leveled) FHE schemes, where the assumptions are usually denoted as the circular-secure LWE assumption, we can construct pure FHE schemes that support circuits of unbounded depth.

Subsequent works mainly focus on constructing practical FHE schemes that are applicable in real-world applications. Celebrated examples include BFV [Bra12, FV12], FHEW [DM15], CKKS [CKKS17], and TFHE [CGGI20].

Most FHE schemes developed in the literature are only proved to have the basic chosen-plaintext (CPA) security, which prevents the adversary from learning any information about the message encrypted in a challenge ciphertext given the public key.[1] This is insufficient due to the active attacks available in practice; we refer the readers to [FHR22] for a recent survey on active attacks for (CPA-secure) FHE schemes. In contrast, the de facto security notion for traditional PKE schemes is the chosen-ciphertext (CCA) security [NY90, RS91, DDN91], which allows the adversary to access a decryption oracle that decrypts the submitted ciphertext. The CCA security is necessary to provide protections against active attacks as discussed in [Sho98].

FHE against Active Attacks. The standard CCA security for PKE schemes is the so called CCA2 security. It allows the adversary to query the decryption oracle both before and after seeing a challenge ciphertext ct^*, with a restriction that ct^* cannot be submitted to the decryption oracle in the post-challenge phase; the restriction can prevent the trivial attack that asks the decryption oracle to decrypt the challenge ciphertext directly. However, CCA2 security is not achievable for FHE schemes, since the adversary can evaluate an invertible function f on ct^*, submit the evaluation result to the decryption oracle, and compute f^{-1} on the decryption result to recover the message encrypted in ct^*. Therefore, it was believed that CCA1 security, where the adversary can only query the decryption oracle in the pre-challenge phase, is the best possible security we can hope for FHE schemes. Below, we discuss prior works on FHE schemes that are secure with decryption queries. We roughly divide them into three categories according to the relation between their security and CCA1 security.

Within CCA1 security. Some works construct FHE schemes that are secure with a restricted decryption oracle. In [LM21], Li and Micciancio present the notion of CPA^D security, where the adversary can query a decryption oracle that only decrypt a ciphertext if (1) it is encrypted by the challenger or (2) it is evaluated from ciphertexts encrypted by the challenger. The new security property

[1] More formally, the adversary can choose two messages m_0^*, m_1^* after seeing the public key and gets a challenge ciphertext ct^* that encrypts either m_0^* or m_1^*, then it is required to distinguish which message is encrypted. This security is denoted as indistinguishability against chosen-plaintext attacks, or IND-CPA security for short. In this work, we only consider indistinguishability-based security definition for FHE schemes and will sometimes omit the term "IND" for simplicity.

is significant for FHE schemes that have an approximate evaluation algorithm [LM21] or noticeable decryption errors [CSBB24, CCP+24]. But it can be implied by the CPA security (and thus the CCA1 security) if the FHE scheme has perfect (evaluation) correctness.[2]

In [AGHV22], Akavia et al. present the notion of FuncCPA security, where the adversary can query the decryption oracle arbitrarily, but the decryption results will be encrypted before being returned. They also construct FHE (resp. leveled FHE) schemes with FuncCPA security from circular-secure LWE (resp. standard LWE) assumptions. However, the FuncCPA security was later shown to be implied by (and weaker than) the CCA1 security [DHW23, STTY24].

CCA1 security. The first (leveled) FHE scheme with CCA1 security is constructed by Loftus et al. in [LMSV11]. The schemes is adapted from the CPA-secure scheme given in [SV10], and a new lattice-based knowledge assumption is introduced to prove its plaintext awareness (PA-1) security. The CCA1 security of the scheme then comes from its CPA security and PA-1 security following [BP04]. However, the CPA security of schemes in [SV10, LMSV11] are threatened by attacks proposed in [CDPR16, BEF+17]; although the attacks run in sub-exponential time, they can practically recover the secret key of the schemes for the parameters suggested in [SV10].

Then in [BSW12, CRRV17], CCA1 secure FHE schemes are constructed via the double encryption paradigm [NY90, Sah99]. The constructions need to use a general zero-knowledge succinct non-interactive argument of knowledge (ZK-SNARK) [Kil92, Mic94], which can be instantiated in the random oracle model [Mic94] or in the standard model from some non-falsifiable knowledge assumptions [Gro10]. The random oracle model or the knowledge assumptions are essential to construct general ZK-SNARKs due to the strong negative results from [GW11]. Therefore, it would be difficult to construct FHE scheme from standard assumptions in the standard model via this approach.

Besides, Canetti et al. [CRRV17] and Yasuda et al. [YKT18] construct CCA1 secure FHE schemes from multi-identity identity-based FHE schemes [CM15] via the CHK transform [CHK04, BCHK07], and instantiate it from LWE. However, the scheme is not fully compact: the size of an evaluated ciphertext generated by evaluating a circuit over l ciphertexts will be polynomial in l. A fully compact scheme is also given in [CRRV17], but the scheme was later shown insecure in [WWX18]. Although [WWX18] provides a patched construction, there seems to be some issues in their security proof[3]. Even without these (potential) security issues, the constructions rely on the heavy hammer of sub-exponentially secure indistinguishability obfuscations.

[2] Nonetheless, the CPA^D security and the CCA1 security are incomparable in general. Please see [BCF+25] for a more in-depth discussion between their relations.

[3] In more detail, indistinguishability between their Game 2 and Game 3 relies on the pseudorandomness of $\mathcal{F}_K(id^*)$, but $\mathcal{F}_K(id^*)$ also appears in some obfuscated circuits included in the public key, without being claimed to be hidden in previous games. Thus, the indistinguishability between these two games may not hold.

Beyond CCA1 security. Recently, Manulis and Nguyen [MN24] present the notion of *verified CCA* (vCCA) security for FHE schemes. It allows the adversary to make queries to the decryption oracle even after seeing the challenge ciphertext, but the decryption oracle will reject a queried ciphertext if it is evaluated from the challenge ciphertext. To enable the check, the definition also requires the existence of an extraction algorithm, which takes as input an evaluated ciphertext and extracts the ciphertexts and the circuit used to generate it. The vCCA security is strictly stronger than the CCA1 security. It can also imply some other intermediate security definition between CCA1 security and CCA2 security, such as the CCA1.5 security [DDA13] and the homomorphic CCA security [PR08], which were defined for partially homomorphic encryption schemes. FHE schemes with vCCA security are constructed from CPA-secure FHE schemes and ZK-SNARKs in [MN24]. Therefore, they need the random oracle model or the non-falsifiable knowledge assumptions. Even worse, these strong assumptions seem essential, because the security definition requires the existence of an extraction algorithm that can extract a long "witness" from a compact ciphertext.

Motivation. To summarize, from (circular-secure) LWE assumptions, we can construct FHE schemes with either CPA^D security or FuncCPA security, which are weaker than the more natural CCA1 security. We can also construct CCA1 secure FHE schemes from LWE, but with ciphertexts that are not fully compact. If we hope to have both CCA1 security and compact ciphertexts, the only known solutions require either random oracle or non-falsifiable knowledge assumptions. Therefore, it is natural to ask

Can we construct compact FHE schemes with CCA1 security (or even more) from (circular-secure) LWE assumptions in the standard model?

1.1 Our Results

In this work, we affirmatively answer the above question and construct CCA-secure FHE schemes from the following four building blocks:

- A CPA-secure FHE scheme, which can be constructed from circular-secure LWE assumptions as discussed above;
- A simulation-sound non-interactive zero-knowledge (NIZK) proof, which can be instantiated from the standard LWE assumption by applying the general construction in [Sah99] to the LWE-based NIZK proofs [PS19, Wat24];
- A non-interactive index batch argument (BARG), which is constructed from the standard LWE assumption in a recent work [CJJ21];
- A predicate extractable commitment, which is a new primitive presented in this work and is constructed from the standard LWE assumption in Sec. 3.

As all building blocks can be constructed from (circular-secure) LWE assumptions, we can instantiate our CCA-secure FHE scheme solely from circular-secure LWE. We illustrate how to instantiate it in Figure 1.

The CCA-security achieved by our FHE scheme is a new security notion called input-verifiable CCA (IV-CCA) security. Similar to the vCCA security

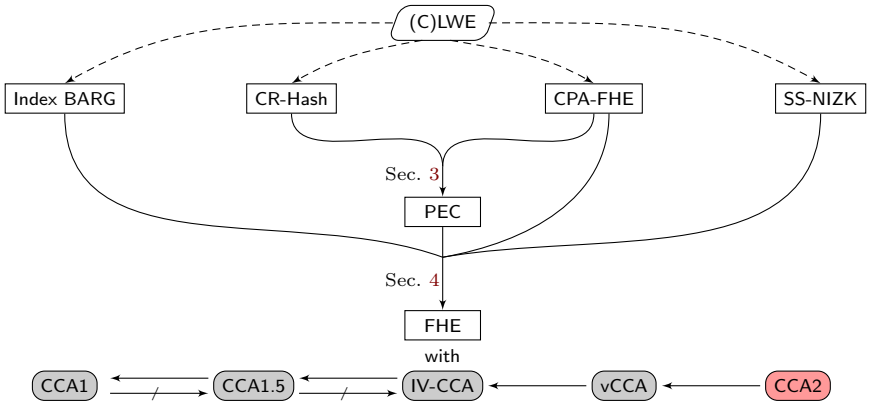


Fig. 1 The roadmap for constructing IV-CCA secure FHE from circular-secure LWE (top part). Here, the dotted lines denote results from previous work; "(C)LWE" denotes "(circular-secure) LWE", "CR-Hash" denotes "collision resistant hash functions", 'SS-NIZK" denotes "simulation-sound NIZK proofs", and "PEC" denotes "predicate extractable commitment". Our FHE scheme achieves IV-CCA security and the bottom part of the figure summarizes its relation with other CCA security notions. Note that the CCA2 security is not achievable for FHE schemes.

proposed in [MN24], the IV-CCA security allows the adversary to submit arbitrary ciphertexts to the decryption oracle, both before and after seeing the challenge ciphertext, and the query will be answered only if it is not generated from the challenge ciphertext. We enable the check by asking the adversary to submit the ciphertexts used to generate the queried ciphertext for each post-challenge query. Nonetheless, we do *not* assume the adversary to be honest in submitting the additional ciphertexts; in contrast, we will construct a verification algorithm that checks whether the provided input ciphertexts can be evaluated to the queried ciphertext. We note that our IV-CCA security is implied by the vCCA security, since their extraction algorithm can be used to construct our verification algorithm. On the other hand, our IV-CCA security is strictly stronger than the CCA1.5 security [DDA13, MN24] and the CCA1 security as discussed in Remark 4.1. We summarize the relations between the IV-CCA security and other CCA security notions discussed in this work in Figure 1.

On the downside, our FHE scheme is only single-hop homomorphic, i.e., outputs of the evaluation algorithm cannot be evaluated again. In addition, similar to previous constructions of CCA-secure FHE schemes from proof systems [BSW12, CRRV17, MN24], the evaluation algorithm of our FHE scheme will attach an additional proof to the evaluated ciphertext, whose size is poly-logarithmically in the size of the evaluated circuit. We discuss these limitation in more detail in Sec. 1.3. We also compare the features of CCA-secure FHE schemes constructed via different approaches in Table 1.

	Security	Hop	$\lvert ct_{\mathtt{Eval}}\rvert - \lvert ct_{enc}\rvert$	Assumptions
FHE + PA-1	CCA1	Multi	0	Knowledge
CHK	CCA1	Multi	$poly(\lambda, l)$ 0	LWE iO
FHE + ZK-SNARK	vCCA	Multi	$poly(\lambda, \log l, \log\lvert\mathtt{C}\rvert)$ $poly(\lambda)$	LWE + RO Knowledge
FHE + BARG + PEC	IV-CCA	Single	$poly(\lambda, \log l, \log\lvert\mathtt{C}\rvert)$	LWE

Table 1: Features of CCA-secure FHE schemes constructed via known approaches. We will consider the following four approaches: (1) adding PA-1 security to CPA-secure FHE [LMSV11], (2) using the CHK transform [CRRV17, YKT18, WWX18], (3) using the double encryption paradigm with general ZK-SNARKs [BSW12, CRRV17, MN24], and (4) our method of using the double encryption paradigm with index BARG and predicate extractable commitment (PEC). The column "Security" denotes the strongest security notion that has been achieved via each approach. The column "Hop" denotes whether the FHE scheme is multi-hop/single-hop, i.e., if the outputs of the evaluation algorithm can/cannot be used as the input of the evaluation algorithm again. The next column denotes the difference between the sizes of the evaluated ciphertext and the fresh ciphertext; here, we assume that the evaluated ciphertext is generated by evaluating a circuit $\mathtt{C}$ on l ciphertexts; we use λ to denote the security parameter and the value 0 means that the evaluation algorithm will not change the ciphertext size. For the assumptions, we only consider the main assumptions used; here "Knowledge" denotes any knowledge-type assumptions, "LWE" denotes the (circular-secure) LWE assumption, "iO" denotes the existence of sub-exponentially secure indistinguishability obfuscations, and "RO" denotes the random oracle model.

1.2 Technical Overview

In this section, we provide an overview of our main techniques. Our starting point is the well-known double encryption paradigm [NY90, Sah99], which leads to CCA secure FHE schemes if we use a general ZK-SNARK in the construction. Then we show how to replace the general ZK-SNARK with a special succinct argument, which can be constructed from batch arguments. We identify the main barriers to prove the security of the construction and then explain how to overcome the barriers.

The Starting Point. Our starting point is the Naor-Yung double encryption paradigm. In particular, given a FHE scheme with CPA security, the encryption algorithm will encrypt the message twice under different public keys, and a NIZK proof is attached to prove that the two ciphertexts encrypt the same message. The decryption algorithm will decrypt exactly one ciphertext after checking the validity of the proof. The double encryption technique allows us to answer the decrypt oracle with one secret key while guaranteeing the security of the ciphertexts under the other key. In addition, since the two ciphertext components of each queried ciphertext are proved to encrypt the same message, it is safe to switch between the two secret keys when answering the decryption queries. This guarantees security of ciphertexts under both keys.

When proving CCA security of a FHE scheme, we need to answer decryption queries for both fresh ciphertexts outputted by the encryption algorithm and evaluated ciphertexts outputted by the evaluation algorithm. Therefore, we need to generate NIZK proofs for both types of ciphertexts. It is easy to prove the validity of a fresh ciphertext, but this needs the encryption randomness as the witness, which is not known by the evaluation algorithm. To prove the validity of an evaluated ciphertext, we can argue that it is evaluated from valid fresh ciphertexts attached with valid NIZK proofs honestly. However, the proof size of a normal NIZK proof is polynomial in the size of the proved statement, which further depends on the size of the evaluated circuit; this ruins the compactness of the FHE scheme. The issue can be solved by using a succinct argument, and [BSW12, CRRV17, MN24] showed how to construct FHE schemes with CCA security from a CPA secure FHE and a general ZK-SNARK. Nonetheless, these general ZK-SNARKs rely on the random oracle model or some non-falsifiable knowledge assumptions, and significant barriers are known to construct them from standard assumptions in the standard model [GW11].

Avoiding the General ZK-SNARK. Recently, there are a number of works that construct succinct arguments from standard assumptions in the standard model, but only for restricted class of languages: including polynomial time deterministic computations (i.e., the class P) [CJJ21, KVZ21], batch NP [CJJ21, KLVW23, CGJ+23], and monotone policy batch NP [BBK+23, NWW24, Goy24].

It is appealing to see if we can use them to prove the validity of the evaluated ciphertext. However, these restricted languages only cover deterministic computations or simple non-deterministic computation. In contrast, the above construction of CCA-secure FHE needs a succinct argument for complex non-deterministic computations, i.e., proving that a target ciphertext can be evaluated from some input ciphertexts via some circuit, where the the input ciphertexts and the circuit are not known to the verifier[4]. Nonetheless, we notice that the ideas of using batch argument to prove circuit satisfiability [CJJ21, BBK+23] is potentially useful for our purpose, and we explain how to apply their techniques to the FHE construction below.

The First Attempt. Next, we describe how to prove the validity of an evaluated ciphertext via an index batch argument [CJJ21]. Recall that the index batch argument proves N statements simultaneously, where these N statements can be viewed as the same statement parameterized by indices from 1 to N[5], and the proof size is only polylogarithmic in N.

Let $(ct_{1,0}, ct_{1,1}, \rho_1), \ldots, (ct_{l,0}, ct_{l,1}, \rho_l)$ be a vector of l input ciphertexts, where $ct_{i,0}, ct_{i,1}$ are two ciphertexts of the underlying CPA-secure FHE scheme that encrypt the same message under different public keys pk_0 and pk_1, and ρ_i is a NIZK proof for this. Also, let $\mathsf{C} : \{0,1\}^l \to \{0,1\}$ be a circuit containing N

[4] This is because the verifier, which is the decryptor in the FHE scheme, can only access the target ciphertext, which cannot contain the input ciphertexts and the circuit due to the compactness requirement.

[5] For example, the i-th statement can be described as follows: there exists a witness w_i s.t. $f(i, w_i) = 1$ for a fixed f.

wires. The evaluation algorithm first evaluates $\mathtt{C}$ homomorphically over $(ct_{1,0}, \ldots, ct_{l,0})$ and $(ct_{1,1}, \ldots, ct_{l,1})$ respectively, and let $(ct_{i,0})_{i\in[1,N]}$ and $(ct_{i,1})_{i\in[1,N]}$ denote the ciphertexts of all the wires in the two evaluations. It then commits each wire $(ct_{i,0}, ct_{i,1})$ to position i of a hash tree [Mer87], and employs the batch argument to prove the following N indexed statements:

- **If the index $i \le l$.** There exists $(ct_{i,0}, ct_{i,1}, \rho_i)$ s.t. $(ct_{i,0}, ct_{i,1})$ is committed at position i of the hash tree, and ρ_i proves that $ct_{i,0}, ct_{i,1}$ encrypt the same message.
- **If the index $i > l$.** There exists $(ct_{i,0}, ct_{i,1}; L_i, ct_{L_i,0}, ct_{L_i,1}; R_i, ct_{R_i,0}, ct_{R_i,1})$ s.t. $(ct_{i,0}, ct_{i,1})$ / $(ct_{L_i,0}, ct_{L_i,1})$ / $(ct_{R_i,0}, ct_{R_i,1})$ are committed at positions i / L_i / R_i of the hash tree, $L_i, R_i < i$, $ct_{i,0}$ is evaluated from $ct_{L_i,0}$ and $ct_{R_i,0}$, and $ct_{i,1}$ is evaluated from $ct_{L_i,1}$ and $ct_{R_i,1}$.

The final ciphertext includes $(ct_{N,0}, ct_{N,1})$ and a proof, which contains the root of the hash tree, a path from $(ct_{N,0}, ct_{N,1})$ to the root, and the proof of the index BARG.

The Challenges. At first glance, our initial attempt successfully proves the validity of the evaluated ciphertext. More precisely, if the final ciphertext $(ct_{N,0}, ct_{N,1})$ is invalid, the N-th statement guarantees that there exists an index j s.t. $(ct_{j,0}, ct_{j,1})$ is also invalid, where j is either L_N or R_N and thus $j < N$.[6] Then by the j-th statement, we can find an invalid ciphertext with a smaller label. We will finally reach an input wire with invalid ciphertext, but the attached proof will prove the validity of the located input ciphertext, making a contradiction.

The above "security proof" works only if the batch argument has *global soundness*, i.e., it can guarantee the soundness of all statements simultaneously. Unfortunately, known batch argument can only achieve a much weaker *local soundness*: the common reference string of the batch argument is embedded with an index i^* and the proof only guarantees the soundness of the i^*-th statement. The common reference string can hide the index i^*, so one may hope to check every wire via a hybrid strategy. However, once the hybrid changes the index of the common reference string (possibly from N to j), the adversary can submit a completely different ciphertext and proof, and we cannot hope $(ct_{j,0}, ct_{j,1})$ is still invalid. Therefore, we need some "global properties" to hold in different hybrids.

Global Properties from Local Soundness. The problem of local soundness also appears in previous works and it is mainly solved via a layer-by-layer hybrid strategy. Assume that we include all ciphertexts $(ct_{i,0}, ct_{i,1}, d_i)_{i\in[1,N]}$ in the proof, where d_i denotes the layer of the i-th wire in the circuit.[7] Then for $h \in [1, d_N]$, the Hybrid h proves that an invalid ciphertext will appear at the $(h-1)$-th layer assuming that an invalid ciphertext appears at the h-th layer, where the

[6] The N-th statement states that $ct_{N,0}$ is evaluated from $ct_{L_N,0}$ and $ct_{R_N,0}$, and $ct_{N,1}$ is evaluated from $ct_{L_N,1}$ and $ct_{R_N,1}$, thus $(ct_{N,0}, ct_{N,1})$ must be a valid ciphertext if both $(ct_{L_N,0}, ct_{L_N,1})$ and $(ct_{R_N,0}, ct_{R_N,1})$ are valid, due to the evaluation correctness of FHE. That means, if $(ct_{N,0}, ct_{N,1})$ is invalid, then either $(ct_{L_N,0}, ct_{L_N,1})$ or $(ct_{R_N,0}, ct_{R_N,1})$ (or both) must be invalid.

[7] In particular, input wires are at layer 0 and a wire evaluated from two wires at layer d_L and layer d_R will be at layer $\max(d_L, d_R) + 1$.

presumption was proved in a previous hybrid. The hybrids work now because they can check if the target event happens or not given all the ciphertexts.

The above solution needs to include all the internal states of the computation in the proof and thus has a large proof size. Then to compress the proof, existing works will hash the states (layer-by-layer). A normal hash function is used in the SNARG for P constructed in [CJJ21], and this is sufficient in their construction because the deterministic computation allows the hybrids to reconstruct all the internal states and compare the hashes of the states. However, we cannot use this approach because our computation is non-deterministic. In [BBK+23], a new primitive called *predicate extractable hash for bit fixing* is used, which will check if the values of the internal wires (at each layer) matches a given string at some predefined positions. The idea of checking states by the hash function looks promising, but the predicate extractable hash for bit fixing is too restricted.

Our Solution. Inspired by [BBK+23], we will check the existence of an invalid ciphertext while committing it. To support this, we present *predicate extractable commitment for general predicates*, which extends the predicate extractable hash. In a nutshell, it is a vector commitment [Mer87,CFM08,LY10,CF13] that checks if each committed value satisfies a predefined predicate. More precisely, the scheme will *secretly* encode a predicate P into its public key. The commit algorithm will commit a vector of values $(x_i)_{i\in[1,N]}$ into a short commitment; meanwhile, it will record the position i in the commitment once it finds an x_i satisfying P. The scheme also has an extraction algorithm that allows one to extract the position from the commitment given the trapdoor. Its security requires that, given a commitment com, if the extraction algorithm cannot extract anything, it should be hard to open com to any x s.t. $\mathsf{P}(x) = 1$ at any position, and if the extraction algorithm obtains a position i, it should be hard to open com to an input x s.t. $\mathsf{P}(x) = 0$ at position i.

Now, we use the predicate extractable commitment instead of the hash tree in the construction. In more detail, the evaluation algorithm also first evaluates the circuit C on the input ciphertexts $(ct_{i,0}, ct_{i,1}, \rho_i)_{i\in[1,l]}$ and let $(ct_{i,0}, ct_{i,1})_{i\in[1,N]}$ denote the ciphertexts of all wires. Also, define d_i as the layer of the i-th wire. The evaluation algorithm then commits $(ct_{i,0}, ct_{i,1}, d_i)_{i\in[1,N]}$ using a predicate extractable commitment. For technical reason, we will commit the same vector twice using different public keys $ck^{(1)}$ and $ck^{(2)}$, and we denote the commitments as $com^{(1)}$ and $com^{(2)}$. Then the algorithm employs the batch argument to prove the following N indexed statements:

- **If the index $i \le l$.** There exists $(ct_{i,0}, ct_{i,1}, d_i, \rho_i)$ s.t.
  1. $(ct_{i,0}, ct_{i,1}, d_i)$ is committed at position i of both $com^{(1)}$ and $com^{(2)}$;
  2. $d_i = 0$;
  3. ρ_i proves that $ct_{i,0}, ct_{i,1}$ encrypt the same message.
- **If the index $i > l$.** There exists $(ct_{i,0}, ct_{i,1}, d_i; L_i, ct_{L_i,0}, ct_{L_i,1}, d_{L_i}; R_i, ct_{R_i,0}, ct_{R_i,1}, d_{R_i})$ s.t.
  1. $(ct_{i,0}, ct_{i,1}, d_i)$ / $(ct_{L_i,0}, ct_{L_i,1}, d_{L_i})$ / $(ct_{R_i,0}, ct_{R_i,1}, d_{R_i})$ are committed at positions i / L_i / R_i of both $com^{(1)}$ and $com^{(2)}$;

2. $d_{L_i}, d_{R_i} < d_i$;
3. $ct_{i,0}$ is evaluated from $ct_{L_i,0}$ and $ct_{R_i,0}$;
4. $ct_{i,1}$ is evaluated from $ct_{L_i,1}$ and $ct_{R_i,1}$.

The final ciphertext includes $(ct_{N,0}, ct_{N,1}, d_N)$ and a proof, which contains the commitments, the openings of $(ct_{N,0}, ct_{N,1}, d_N)$ for both commitments, and the proof of the index BARG.

Next, to prove the soundness of the proof described above, we will use the hybrid strategy. First, for any integer i, we define $\mathtt{P}_i(ct_0, ct_1, d)$ be a function that outputs 1 iff (1) $d \leq i$ and (2) the decryption results of ct_0 and ct_1 are different, i.e., it will pick invalid ciphertexts (on a wire) with layer not exceeding i. Then at Hybrid h, we will encode $\mathtt{P}_h$ and $\mathtt{P}_{h-1}$ into the commitment public keys $ck^{(1)}$ and $ck^{(2)}$ respectively. Assume that we can extract an index j from $com^{(1)}$, then the j-th ciphertext $(ct_{j,0}, ct_{j,1}, d_j)$ should be an invalid ciphertext and $d_j \leq h$ due to the security of the predicate extractable commitment. We also set the BARG to be sound at the j-th statement[8], then the correctness of the j-th statement guarantees that there exists an invalid ciphertext $(ct_{j',0}, ct_{j',1}, d_{j'})$ s.t. $d_{j'} < d_j \leq h$, i.e., $d_{j'} \leq h - 1$. Note that $\mathtt{P}_{h-1}(ct_{j',0}, ct_{j',1}, d_{j'}) = 1$. This further implies that we can extract an index (not necessarily equal to j') from $com^{(2)}$ due to the security of the predicate extractable commitment. Next, we switch the encoded predicates to be $(\mathtt{P}_{h-1}, \mathtt{P}_{h-1})$ and then to be $(\mathtt{P}_{h-1}, \mathtt{P}_{h-2})$. The modifications cannot be detected since the commitment public keys hide the embedded predicates. Now, we have moved to Hybrid $h-1$. The previous hybrid ensures that there exist invalid ciphertexts with layer not exceeding $h-1$, and this guarantees the presumption for Hybrid $h-1$; then we can further prove that there exists invalid ciphertexts with layer not exceeding $h-2$.

To summarize, if the final ciphertext $(ct_{N,0}, ct_{N,1}, d_N)$ is invalid, then we can show that there exists some invalid ciphertext at layer 0 via the above hybrids. We can also extract its index j and set the BARG to be sound at j. This implies that there exists a NIZK proof to certificate the validity of the invalid ciphertext $(ct_{j,0}, ct_{j,1})$, making a contradiction.

On Achieving IV-CCA Security. We have discussed how to construct the succinct argument for the validity of an evaluated ciphertext. Roughly, this is because if the evaluated ciphertext is invalid, then the argument ensures that at least one of its input ciphertext, which has a NIZK proof certificating its validity, is invalid. The argument implies the validity of the queried ciphertexts in the pre-challenge phase, because the adversary cannot forge a valid NIZK proof for an invalid ciphertext. For the post-challenge queries, while the challenge ciphertext can be an invalid ciphertext with a valid proof (in some hybrids), it is excluded from being used to generate a queried ciphertext. Then the IV-CCA security of the FHE scheme can be proved following the double encryption paradigm.

[8] We need to specify the index j in the beginning, but this issue can be solved by guessing j in the beginning and checking if the guess is correct after extracting j from $com^{(1)}$. This will introduce a polynomial reduction loss but only affects the current hybrid.

To deny ciphertexts evaluated from the challenge ciphertexts, the IV-CCA security requires the adversary to submit all the input ciphertexts used to generate the queried ciphertext in a post-challenge decryption query, and needs a verification algorithm to check the correctness of the submitted input ciphertexts. We enable the check by committing all the *input ciphertexts* into a third commitment $com^{(3)}$ in the evaluation algorithm and additionally proving that each input ciphertext is committed to $com^{(3)}$ via the BARG. The verification algorithm then commits all the input ciphertexts provided by the adversary into a commitment com' and examines if it is identical to $com^{(3)}$ included in the queried ciphertext. We require the predicate extractable commitment to have a determinist commit algorithm, then we have $com' = com^{(3)}$ if all the input ciphertexts are correct; otherwise, com' will be different from $com^{(3)}$ due to the binding property of the commitment scheme.

Constructing Predicate Extractable Commitment. It remains to show how to construct a predicate extractable commitment, which is a vector commitment that allows one to extract the position of a value in the vector that satisfies a (hidden) predicate $\mathtt{P}$. Our construction is inspired by previous constructions of similar primitives, including somewhere extractable hash [HW15] and predicate extractable hash [BBK+23].

In more detail, the predicate $\mathtt{P}$ is encrypted by a FHE scheme. The ciphertext $ct_{\mathtt{P}}$ is set to be the commitment public key, and the secret key of the FHE scheme is the trapdoor. The construction also needs a collision resistant hash function. Then given a vector of values $(x_0, \ldots, x_{N-1})$, the commit algorithm first build a hash tree [Mer87] for the vector. Recall that, the hash tree will contain $n+1$ levels, where $n = \lceil \log N \rceil$; the root node is at level 0 and the leaf nodes are at level n; each node at level k is associated with a k-bit string as its label; the label for the i-th leaf node is the n-bit binary representation of i, and the label for an internal node is the prefix of its two child nodes. Besides, the i-th leaf node contains x_i, and each internal node contains the hash of the values contained in its two child nodes. Next, the commit algorithm adds some extra information to each node of the hash tree as follows:

- The i-th leaf node additionally stores its position i if $\mathtt{P}(x_i) = 1$ and it stores $\perp$ if $\mathtt{P}(x_i) = 0$.
- For an internal node with label v, let $z_{v\|0}$, $z_{v\|1}$ be the additional values stored in its two child nodes, which have labels $v\|0$ and $v\|1$ respectively, then the algorithm set the value z_v stored at node v as follows:
  1. If $z_{v\|0} \neq \perp$ and $v\|0$ is a prefix of $z_{v\|0}$: $z_v = z_{v\|0}$;
  2. Otherwise, if $z_{v\|1} \neq \perp$ and $v\|1$ is a prefix of $z_{v\|1}$: $z_v = z_{v\|1}$;
  3. Otherwise, $z_v = \perp$.

As the predicate $\mathtt{P}$ is needed to compute the extract information, the whole commit algorithm will be run homomorphically on $ct_{\mathtt{P}}$, and each node in the hash tree will have a ciphertext that encrypts the hash output and the extra information (i.e., a position or a failure symbol $\perp$). The commitment will be the ciphertext associated with the root node.

To open the above commitment at a position i, the opening algorithm first identify the path from the i-th leaf node to the root, then it picks the ciphertext for the sibling node of each node in this path. Given x_i and the openings, the verification algorithm first reconstructs the ciphertext for the leaf node i from x_i and $\mathtt{ct}_{\mathtt{P}}$, then it continues to rebuild the path to the root of the hash tree and compares the reconstructed root ciphertext with the commitment. Here, to guarantee the verification correctness, we need the underlying FHE to have a deterministic evaluation algorithm. Moreover, since we only add some tiny information to a standard hash tree, its binding property and succinctness is not affected. Besides, the predicate is hidden due to the CPA security of FHE.

We finally explain how to extract the correct information from the commitment. The extraction algorithm simply decrypt the commitment, i.e., the ciphertext on the root node, and gets either a position i or a failure symbol $\perp$. Here, we say a node with label v contains a valid position z if v is a prefix of z, then it is easy to see a parent node contains a valid position iff at least one of its child node contains a valid position. Now assume that the extraction algorithm gets $\perp$ from the root node, then none of its child nodes should contain a valid position, and this will finally imply that none of the nodes in the hash tree should contain a valid position; as a leaf node i will contain i if $\mathtt{P}(x_i) = 1$, that means $\mathtt{P}(x_i) = 0$ for all x_i. On the other hand, if the extraction algorithm gets a position i and let i_1 be the first bit of the binary representation of i, then the node i_1 must contain i, because i is not a valid position for the node $\overline{i_1}$; for the same reason, we can finally show that the node i must contain i and this implies that $\mathtt{P}(x_i) = 1$.

The above analysis assumes that the commitment is constructed honestly. Nonetheless, the way the verification algorithm works ensures that the commitment can be reconstructed honestly from a value x_j as long as a valid opening for x_j is provided. Then we can at least guarantee: if the extraction algorithm gets $\perp$, then x_j must not satisfy the predicate $\mathtt{P}$; and if the extraction algorithm gets j, then $\mathtt{P}(x_j) = 1$. That is sufficient for our construction of FHE. Another concern is that the adversary may compromise the homomorphic computations by including some ill-formed FHE ciphertexts (e.g., ciphertexts with huge noises) in the opening. The issue can be solved using the bootstrapping technique [Gen09b, Gen09a], where ill-formed FHE ciphertexts can be refreshed by decrypting them homomorphically with an encryption of the secret key. We can either use a pure FHE that supports bootstrapping directly, or use a leveled FHE but prepare different key pairs at each level of the hash tree. In Sec. 3.2, we employ the second approach and the scheme can be instantiated from the standard LWE assumption.

1.3 Discussions

CCA Security vs. Bootstrapping. The bootstrapping technique [Gen09b, Gen09a] allows one to "refresh" evaluated ciphertexts by decrypting them homomorphically. This requires the scheme to publish ct_{sk} that encrypts the secret key of the FHE scheme in the beginning, and it was thought to be incompatible

with even CCA1 security, because the adversary may submit ct_{sk} (or ciphertexts related to it) to the decryption oracle. The issue, however, does not exist in constructions from NIZK proofs. More precisely, in these constructions (including ours), we only attach proofs to ciphertexts outputted by the encryption algorithm, and ct_{sk} does not have a valid proof. In addition, the decryption algorithm/oracle only accepts a fresh ciphertext with a valid proof or an evaluated ciphertext from fresh ciphertexts with valid proofs, where the latter is proved by a succinct argument. Therefore, the adversary cannot submit ct_{sk} or any ciphertexts that are evaluated directly from ct_{sk} to the decryption oracle, i.e., it cannot obtain information about sk from the decryption queries.

Note that the decryption algorithm/oracle can still decrypt bootstrapped ciphertext. To see this, we briefly describe what happens after performing the bootstrapping procedure in a NIZK-based construction. For simplicity, suppose that the bootstrapping operation is incorporated into every gate evaluation. That means, for any ciphertexts (ct_1, π_1) and (ct_2, π_2), we will evaluate a gate $\mathtt{G}$ over them by running the following circuit (with constant ct_1 and ct_2) on ct_{sk} homomorphically: first the circuit decrypts ct_1, ct_2 with the input sk and gets m_1, m_2; then it outputs $\mathtt{G}(m_1, m_2)$. This gives us an encryption of the output of the gate. Then we can create a proof to show the validity of this new ciphertext, proving that it is evaluated from ciphertexts (i.e., ct_1 and ct_2) that are associated with valid proofs (i.e., π_1 and π_2). Therefore, the ciphertext is acceptable to the decryption algorithm/oracle. But this will not compromise security of the scheme because the bootstrapped ciphertext does not contain any information about sk.

On Compactness of Our Construction. In our FHE scheme, if we evaluate a circuit $\mathtt{C}$ on l ciphertexts, the size of the evaluated ciphertext ct grows polylogarithmically in $|\mathtt{C}|$ and l. This is "less compact" than previous CPA-secure FHE schemes, where the evaluation algorithm does not change the size of the ciphertext inherently. Nonetheless, we can bound the size of $|ct|$ by a pre-defined polynomial $L(\lambda, \log|\mathtt{C}|, \log l)$. In addition, both $|\mathtt{C}|$ and l are polynomial in λ, and we can safely assume that $l, |\mathtt{C}| \leq 2^{\lambda}$. Therefore, $L(\lambda, \log|\mathtt{C}|, \log l) \leq L(\lambda, \lambda, \lambda)$, i.e., we can still bound $|ct|$ by a pre-defined polynomial $L(\lambda, \lambda, \lambda)$.

On Achieving Multi-Hop Homomorphism. In this work, we only construct CCA secure FHE schemes with single-hop homomorphism, i.e., the evaluation algorithm only takes as input fresh ciphertexts outputted by the encryption algorithm. It is preferable to have a multi-hop FHE scheme, which allows the outputs of the evaluation algorithm to be used as the inputs of the evaluation algorithm again. However, there are a few barriers to adapt our construction to support multi-hop evaluations.

First, multi-hop evaluations need the evaluation algorithm to run the batch argument recursively, i.e., the statements proved in the $(i+1)$-th evaluation needs to verify the proof in the ciphertexts outputted by the i-th evaluation. This might lead to a large blow-up of the proof size. Even if we use rate-1 batch arguments [DGKV22, PP22], whose proof size is close to the size of the witness for a single statement, the final proof size would still be polynomial in the number of hops. Moreover, in our security proof, we need to use the guess-

and-check trick when applying the local soundness of the batch argument; if there are k hops and we need to use the soundness of the BARG proof in the initial evaluation, a reduction loss that is exponential in k will be introduced. Besides, the evaluation algorithm needs to commit all (fresh) input ciphertexts to support the input ciphertexts verification required by the IV-CCA security, and it looks like that in the final ciphertext outputted by multi-hop evaluations, we need to include all these commitments in previous evaluations. The total size of the commitments will also be polynomial in the number of hops. In addition, a recent work [CHL+25] uncovers a few gaps when proving CCA security of multi-hop homomorphic encryption schemes from succinct arguments. Those gaps may also arise when extending our constructions and proofs to the multi-hop setting.

It is a fascinating open problem to construct a CCA secure multi-hop FHE scheme, ideally with a compact ciphertext whose size is only polylogarithmic in the number of hops, from standard assumption in the standard model.[9]

On Achieving Stronger CCA Security. In this work, we define and construct FHE schemes with input-verifiable chosen-ciphertext security. In its definition, the challenger will check if a post-challenge decryption query is evaluated from the challenge ciphertext, and it requires the adversary to submit all input ciphertexts used to generate the queried ciphertext to enable the check. The security definition is weaker than the vCCA security defined in [MN24]. In the vCCA security definition, the adversary only needs to submit the queried ciphertext to the post-challenge decryption oracle, then an additional extraction algorithm will extract all input ciphertexts (and the evaluation circuit) from the queried ciphertext and checks if the challenge ciphertext is included. However, the vCCA security needs a very powerful extraction algorithm that is able to obtain (potentially) unbounded information from a compact ciphertext, and it seems impossible to implement it from standard assumptions in the standard model. Therefore, it is interesting to find some intermediate security notions between the IV-CCA security and the vCCA security, and construct FHE schemes satisfying them from standard assumption in the standard model.

2 Preliminaries

We write $negl(\cdot)$ to denote a negligible function, and write $poly(\cdot)$ to denote a polynomial. All polynomials are positive unless otherwise specified. For integers $a \leq b$, we write $[a, b]$ to denote all integers from a to b. Let s be a string, we use $|s|$ to denote the length of s. For integers $a \leq |s|$, we use $s[a]$ to denote the a-th character of s and for integers $a \leq b \leq |s|$, we use $s[a : b]$ to denote the substring $(s[a], s[a+1], \ldots, s[b])$. Let $\mathcal{S}$ be a finite set, we use $|\mathcal{S}|$ to denote the size of $\mathcal{S}$, and use $s \xleftarrow{\$} \mathcal{S}$ to denote sampling an element s uniformly from set $\mathcal{S}$. Let $\mathcal{D}$ be a distribution, we use $d \leftarrow \mathcal{D}$ to denote sampling d according to $\mathcal{D}$.

[9] In a subsequent work [BBPS26], Brakerski et al. construct a multi-hop FHE scheme with CCA1 security from LWE. They solve the exponential reduction loss issue by using a functional batch argument [PP26], but the ciphertext size of their construction is still linear in the number of hops.

Following the syntax in [BLW17], for a circuit family $\mathtt{C}$ indexed by a few, say m, constants, we write $\mathtt{C}[c_1, \ldots, c_m]$ to denote a circuit with constants c_1, $\ldots, c_m$. We use $\overline{\wedge}$ to denote the NAND gate and suppose that all circuits are composed exclusively by NAND gates unless otherwise specified. In particular, given a circuit $\mathtt{C} : \{0,1\}^l \to \{0,1\}^k$ with N wires, we can label its input wires with numbers in $[1, l]$, label its output wires with numbers in $[N-k+1, N]$, and label all other internal wires with numbers in $[l+1, N-k]$, where each wire has a larger label than its children; in addition, we can write the circuit as $(1^l, 1^k, (L_i, R_i)_{i\in[l+1,N]})$, where $L_i \leq R_i < i$, and for $i \in [l+1, N]$, the value of the i-th wire in the circuit is evaluated by performing the NAND gate on values of the L_i-th and the R_i-th wires. Below we recall the definitions of cryptographic primitives employed in this work.

2.1 Collision Resistant Hash

A collision resistant hash $\mathsf{H} = (\mathtt{KeyGen}, \mathtt{Eval})$ with input space $\{0,1\}^*$ and output space $\{0,1\}^n$ consists of the following two probabilistic polynomial-time (PPT) algorithms:

- $\mathtt{KeyGen}(1^\lambda) \to hk$: On input the security parameter 1^λ, the key generation algorithm outputs the hash key hk.
- $\mathtt{Eval}(hk, x) \to y$: On input the hash key hk and an input x, the *deterministic* evaluation algorithm outputs an output $y \in \{0,1\}^n$.

In addition, it satisfies the following conditions:

- **Collision Resistance.** For all PPT adversaries $\mathcal{A}$, we have

$$\Pr\left[\begin{matrix} hk \leftarrow \mathtt{KeyGen}(1^\lambda); \\ (x_0, x_1) \leftarrow \mathcal{A}(1^\lambda, hk); \end{matrix} : \begin{matrix} x_0 \neq x_1 \wedge \\ \mathtt{Eval}(hk, x_0) = \mathtt{Eval}(hk, x_1) \end{matrix}\right] \leq negl(\lambda)$$

2.2 Simulation-Sound NIZK Proof

A simulation-sound non-interactive zero-knowledge proof [BFM88, Sah99] for a language $\mathcal{L}$ consists of the following PPT algorithms:

- $\mathtt{Setup}(1^\lambda) \to crs$. On input the security parameter 1^λ, the setup algorithm outputs a common reference string crs.
- $\mathtt{Prove}(crs, x, w) \to \pi$. On input the common reference string crs, a statement $x \in \mathcal{L}$, and a witness w for x, the proving algorithm outputs a proof π.
- $\mathtt{Verify}(crs, x, \pi) \to b$. On input the common reference string crs, a statement x, and a proof π, the *deterministic* verification algorithm outputs a bit indicating whether the proof is valid.
- $\mathtt{S}_1(1^\lambda) \to (crs, td)$. On input the security parameter 1^λ, the simulated setup algorithm outputs a common reference string crs and a trapdoor td.
- $\mathtt{S}_2(crs, td, x) \to \pi$. On input the common reference string crs, a trapdoor td and a statement x, the simulated proving algorithm outputs a proof π.

Also, it satisfies the following conditions:

- **Completeness.** For any $x \in \mathcal{L}$ and any valid witness w for x,

$$\Pr[crs \leftarrow \mathtt{Setup}(1^\lambda), \pi \leftarrow \mathtt{Prove}(crs, x, w) : \mathtt{Verify}(crs, x, \pi) = 1] = 1$$

- **Zero-Knowledge.** For any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, we have:

$$\Pr\left[\begin{array}{l} b \leftarrow \{0,1\}; \\ crs_0 \leftarrow \mathtt{Setup}(1^\lambda); \\ (crs_1, td) \leftarrow \mathtt{S}_1(1^\lambda); \\ (x, w, state) \leftarrow \mathcal{A}_1(crs_b); \\ \pi_0 \leftarrow \mathtt{Prove}(crs_0, x, w); \\ \pi_1 \leftarrow \mathtt{S}_2(crs_1, td, x); \\ b' \leftarrow \mathcal{A}_2(\pi_b, state) \end{array} : \; b = b'\right] \leq 1/2 + negl(\lambda)$$

  Here $\mathcal{A}_1$ is required to output a *valid* statement/witness pair.
- **Simulation-Soundness.** For any unbounded adversary $\mathcal{A}$, we have:

$$\Pr\left[\begin{array}{l} (crs, td) \leftarrow \mathtt{S}_1(1^\lambda); \\ (x', \pi') \leftarrow \mathcal{A}^{\mathtt{S}_2(crs, td, \cdot)}(crs) \end{array} : \begin{array}{l} (x', \pi') \notin \mathcal{Q} \wedge x' \notin \mathcal{L} \\ \wedge\, \mathtt{Verify}(crs, x', \pi') = 1 \end{array}\right] \leq \mathsf{negl}(\lambda)$$

  where the oracle $\mathtt{S}_2(crs, td, \cdot)$ outputs $\mathtt{S}_2(crs, td, x)$ on input a query x, we use $\mathcal{Q}$ to denote the list of input/output pairs of the oracle, and the adversary is only allowed to query the oracle for at most once.

Simulation-Sound NIZK from LWE. Simulation-sound NIZK can be constructed from a standard NIZK and a one-way function [Sah99]. Moreover, standard NIZK can be constructed from LWE [PS19, Wat24]. Therefore, simulation-sound NIZK can be instantiated from the LWE assumption.

2.3 Fully Homomorphic Encryption

A fully homomorphic encryption scheme [Gen09b] with message space $\{0,1\}$ and ciphertext space $\mathcal{C}$ consists of the following PPT algorithms:

- $\mathtt{KeyGen}(1^\lambda) \to (pk, sk)$. On input the security parameter 1^λ, the key generation algorithm outputs a public key pk and a secret key sk.
- $\mathtt{Enc}(pk, \mu) \to ct$. On input the public key pk and a message $\mu \in \{0,1\}$, the encryption algorithm outputs a ciphertext $ct \in \mathcal{C}$.
- $\mathtt{Dec}(sk, ct) \to \mu$. On input the secret key sk and a ciphertext $ct \in \mathcal{C}$, the *deterministic* decryption algorithm outputs a message $\mu \in \{0,1\}$.
- $\mathtt{Eval}(pk, ct_1, ct_2) \to ct$. On input the public key pk and two ciphertexts ct_1, $ct_2 \in \mathcal{C}$, the *deterministic* evaluation algorithm (for NAND gate) outputs a ciphertext $ct \in \mathcal{C}$.

Besides, we define the following algorithms for ease of notation in applications, where $\texttt{EncV}/\texttt{DecV}$ will encrypt/decrypt a vector of messages/ciphertexts and $\texttt{EvalC}$ will evaluate a circuit rather than a NAND gate:

- $\texttt{EncV}(pk, \boldsymbol{\mu}) \to \boldsymbol{ct}$. For any polynomial m, on input the public key pk and an m-bit string $\boldsymbol{\mu} = (\mu_1, \ldots, \mu_m) \in \{0,1\}^m$, the algorithm computes $ct_i \leftarrow \texttt{Enc}(pk, \mu_i)$ for $i \in [1, m]$, and outputs $\boldsymbol{ct} = (ct_1, \ldots, ct_m)$.
- $\texttt{DecV}(sk, \boldsymbol{ct}) \to \boldsymbol{\mu}$. For any polynomial m, on input the secret key sk and m ciphertexts $\boldsymbol{ct} = (ct_1, \ldots, ct_m) \in \mathcal{C}^m$, the algorithm computes $\mu_i = \texttt{Dec}(sk, ct_i)$ for $i \in [1, m]$, and outputs $\boldsymbol{\mu} = (\mu_1, \ldots, \mu_m)$.
- $\texttt{EvalC}(pk, \texttt{C}, \boldsymbol{ct}) \to \boldsymbol{ct'}$. For any polynomial l, k, N, the algorithm takes as input the public key pk, a circuit $\texttt{C} = (1^l, 1^k, (L_i, R_i)_{i \in [l+1, N]})$ (i.e., a circuit with l-bit input, k-bit output, and N wires), and a vector of l ciphertexts $\boldsymbol{ct} = (ct_1, \ldots, ct_l) \in \mathcal{C}^l$. Then it computes $ct_i = \texttt{Eval}(pk, ct_{L_i}, ct_{R_i})$ for $i \in [l+1, N]$, and outputs $\boldsymbol{ct'} = (ct_{N-k+1}, \ldots, ct_N)$.

Also, it satisfies the following properties:

- **Standard Correctness.** For any messages $\mu \in \{0,1\}$,

$$\Pr[(pk, sk) \leftarrow \texttt{KeyGen}(1^\lambda), ct \leftarrow \texttt{Enc}(pk, \mu) : \texttt{Dec}(sk, ct) = \mu] = 1$$

- **Evaluation Correctness.** For any messages $\mu_1, \mu_2 \in \{0,1\}$,

$$\Pr[(pk, sk) \leftarrow \texttt{KeyGen}(1^\lambda), ct_1 \leftarrow \texttt{Enc}(pk, \mu_1), ct_2 \leftarrow \texttt{Enc}(pk, \mu_2) : \texttt{Dec}(sk, \texttt{Eval}(pk, ct_1, ct_2)) = \mu_1 \barwedge \mu_2] = 1$$

  Also, for any polynomial l, k, N, any vector of messages $\boldsymbol{\mu} = (\mu_1, \ldots, \mu_l) \in \{0,1\}^l$, and any circuit $\texttt{C} = (1^l, 1^k, (L_i, R_i)_{i \in [l+1, N]})$,

$$\Pr[(pk, sk) \leftarrow \texttt{KeyGen}(1^\lambda), \boldsymbol{ct} \leftarrow \texttt{EncV}(pk, \boldsymbol{\mu}) : \texttt{DecV}(sk, \texttt{EvalC}(pk, \texttt{C}, \boldsymbol{ct})) = \texttt{C}(\mu_1, \ldots, \mu_l)] = 1$$

- **Strong Evaluation Correctness.** For any $ct_1, ct_2 \in \mathcal{C}$,

$$\Pr[(pk, sk) \leftarrow \texttt{KeyGen}(1^\lambda), ct = \texttt{Eval}(pk, ct_1, ct_2) : \texttt{Dec}(sk, ct) = \texttt{Dec}(sk, ct_1) \barwedge \texttt{Dec}(sk, ct_2)] = 1$$

- **Compactness.** For any $(pk, sk) \leftarrow \texttt{KeyGen}(1^\lambda)$, any $ct_1, ct_2 \in \mathcal{C}$, let $ct = \texttt{Eval}(pk, ct_1, ct_2)$, then we have $|ct| = |ct_1| = |ct_2|$ and the running time for decrypting ct, ct_1, ct_2 are identical.
- **IND-CPA Security.** For any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, we have:

$$\Pr\left[\begin{array}{l} (pk, sk) \leftarrow \texttt{KeyGen}(1^\lambda); \\ (\mu_0^*, \mu_1^*, state) \leftarrow \mathcal{A}_1(pk); \\ b \xleftarrow{\$} \{0,1\}; \\ ct^* \leftarrow \texttt{Enc}(pk, \mu_b^*) \end{array} : \mathcal{A}_2(ct^*, state) = b \right] \leq 1/2 + negl(\lambda)$$

Remark 2.1. Previous FHE schemes (e.g., the FHE scheme given in [GSW13]) are proven to have the compactness and the IND-CPA security, assuming the circular-secure LWE assumption. For the correctness properties, existing schemes are usually only proved to have the statistical standard/evaluation correctness, where the decryption may fail with a negligible probability; in contrast, we require perfect correctness properties in the above definition. Nonetheless, it is easy to adapt these schemes to have the perfect correctness by replacing the underlying discrete Gaussian distribution with the truncated discrete Gaussian distribution [LATV12]. In a nutshell, the truncated discrete Gaussian distribution will first sample elements from the standard discrete Gaussian distribution, but it outputs 0 directly if the sampled value exceeds a pre-defined threshold; this distribution is statistically indistinguishable from the standard discrete Gaussian distribution if we set a large enough threshold; on the other hand, we can bound the norm of noises sampled from this truncated distribution exactly, which allows us to bound the size of the error term in the decryption exactly and achieve perfect correctness. Besides, although the strong evaluation correctness is not considered in previous work, it can be achieved if we slightly modify their evaluation algorithms. In more detail, in previous constructions of pure FHE schemes, a ciphertext $\boldsymbol{ct}_{sk}$ that encrypts the secret key of the FHE scheme should be included in the public key to enable the bootstrapping procedure. This allows us to run the algorithm $\mathtt{Eval}(pk, ct_1, ct_2)$ by evaluating the circuit $\mathtt{DE}_{ct_1,ct_2}$ on $\boldsymbol{ct}_{sk}$, where $\mathtt{DE}_{ct_1,ct_2}(sk) = \mathtt{Dec}(sk, ct_1) \barwedge \mathtt{Dec}(sk, ct_2)$; then the strong evaluation correctness follows. To recap, FHE schemes that satisfy all properties defined above exist assuming the hardness of the circular-secure LWE assumption.

Remark 2.2. A leveled FHE scheme is a FHE scheme that only supports evaluations on circuits of bounded depth d that is specified when generating the keys. That is, the key generation algorithm should additionally take as input a parameter 1^d, and its evaluation correctness only holds for circuits of depth at most d. Besides, we do not require the strong evaluation correctness for a leveled FHE scheme. Leveled FHE scheme can be constructed from the standard LWE assumption as shown in previous works such as [BV11a, GSW13].

2.4 Somewhere Extractable Non-Interactive Batch Arguments

Let $W = W(\lambda)$ be a polynomial and let $\mathcal{L}$ be the following language:

$$\mathcal{L} = \{(\mathtt{C}, ind) : \exists w \in \{0,1\}^W, \mathtt{C}(ind, w) = 1\}$$

where $\mathtt{C} : \{0,1\}^\lambda \times \{0,1\}^W \to \{0,1\}$ is a circuit, ind is an index, and w is the witness. A non-interactive index batch argument for $\mathcal{L}$ [CJJ21] consists of the following PPT algorithms:

- $\mathtt{Setup}(1^\lambda, 1^C, ind) \to (crs, td)$. On input the security parameter 1^λ, the circuit size C, and an index ind, the setup algorithm outputs a common reference string crs and a trapdoor td.

- $\texttt{Prove}(crs, N, \texttt{C}, w_1, \ldots, w_N) \to \pi$. On input the common reference string crs, the number of instances N, a circuit $\texttt{C} : \{0,1\}^\lambda \times \{0,1\}^W \to \{0,1\}$, and N witnesses $w_1, \ldots, w_N$, the proving algorithm outputs a proof π.
- $\texttt{Verify}(crs, N, \texttt{C}, \pi) \to b$. On input the common reference string crs, the number of instances N, a circuit $\texttt{C} : \{0,1\}^\lambda \times \{0,1\}^W \to \{0,1\}$, and a proof π, the verification algorithm outputs a bit indicating if the proof is valid.
- $\texttt{Extract}(td, N, \texttt{C}, \pi) \to w$. On input the trapdoor td, the number of instances N, a circuit $\texttt{C} : \{0,1\}^\lambda \times \{0,1\}^W \to \{0,1\}$, and a proof π, the extraction algorithm outputs a witness w.

Also, it satisfies the following properties:

- **Completeness.** For any polynomials $N = N(\lambda), C = C(\lambda), W = W(\lambda)$, any circuit $\texttt{C} : \{0,1\}^\lambda \times \{0,1\}^W \to \{0,1\}$ of size at most C, any sequence of witnesses $w_1, \ldots, w_N \in \{0,1\}^W$ s.t. $\texttt{C}(i, w_i) = 1$ for $i \in [1, N]$, and any index $ind \in [1, N]$:

$$\Pr\left[\begin{array}{l}(crs, td) \leftarrow \texttt{Setup}(1^\lambda, 1^C, ind); \\ \pi \leftarrow \texttt{Prove}(crs, N, \texttt{C}, w_1, \ldots, w_N)\end{array} : \quad \texttt{Verify}(crs, N, \texttt{C}, \pi) = 1\right] = 1$$

- **Succinctness.** For any polynomials $N = N(\lambda), C = C(\lambda), W = W(\lambda)$, any circuit $\texttt{C} : \{0,1\}^\lambda \times \{0,1\}^W \to \{0,1\}$ of size at most C, any sequence of witnesses $w_1, \ldots, w_N \in \{0,1\}^W$ s.t. $\texttt{C}(i, w_i) = 1$ for $i \in [1, N]$, and any index $ind \in [1, N]$, let $(crs, td) \leftarrow \texttt{Setup}(1^\lambda, 1^C, ind)$, $\pi \leftarrow \texttt{Prove}(crs, N, \texttt{C}, w_1, \ldots, w_N)$. Then we have

$$|crs| \leq poly(\lambda, C), \quad |\pi| \leq poly(\lambda, C, \log N)$$

  and the running time of the verification algorithm on input $(crs, N, \texttt{C}, \pi)$ is bounded by $poly(\lambda, C, \log N)$.
- **Index Hiding.** For any polynomials $C = C(\lambda)$ and for all PPT adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, we have:

$$\Pr\left[\begin{array}{l}(ind_0, ind_1, state) \leftarrow \mathcal{A}_1(1^\lambda, 1^C); \\ b \xleftarrow{\$} \{0,1\}; \\ (crs, td) \leftarrow \texttt{Setup}(1^\lambda, 1^C, ind_b); \\ b' \leftarrow \mathcal{A}_2(crs, state);\end{array} : \quad b = b'\right] \leq 1/2 + negl(\lambda)$$

- **Somewhere Argument of Knowledge.** For any polynomials $C = C(\lambda)$ and for all PPT adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, we have:

$$\Pr\left[\begin{array}{l}(ind, state) \leftarrow \mathcal{A}_1(1^\lambda, 1^C); \\ (crs, td) \leftarrow \texttt{Setup}(1^\lambda, 1^C, ind); \\ (N, \texttt{C}, \pi) \leftarrow \mathcal{A}_2(crs, state); \\ w \leftarrow \texttt{Extract}(td, N, \texttt{C}, \pi); \\ b \leftarrow \texttt{Verify}(crs, N, \texttt{C}, \pi)\end{array} : \begin{array}{c} ind \in [1, N] \wedge b = 1 \\ \wedge\, \texttt{C}(ind, w) = 0 \end{array}\right] \leq negl(\lambda)$$

An index BARG can be constructed from the standard LWE assumption as shown in [CJJ21]. However, the scheme only works when the number of instances is determined in the setup phase. It is easy to remove this restriction using standard techniques and we show the concrete construction below for completeness.

Index BARG from LWE. Given an index BARG $\mathsf{BI}' = (\mathtt{Setup}', \mathtt{Prove}', \mathtt{Verify}', \mathtt{Extract}')$ that fixes the batch size N in the beginning, it is easy to construct an index BARG that can choose the batch size N after generating the common reference string:

- $\mathtt{Setup}$. On input the security parameter 1^λ, the circuit size C, and an index ind, the setup algorithm generates

  $$(crs_i, td_i) \leftarrow \mathtt{Setup}'(1^\lambda, 2^i, 1^{C'}, \min(ind, 2^i))$$

  for $i \in [1, \lambda]$, where C' is the upper bound of the size of the circuit $\mathtt{C}'$ defined below. Then it outputs

  $$CRS = (crs_i)_{i\in[1,\lambda]}, \quad TD = (td_i)_{i\in[1,\lambda]}$$

- $\mathtt{Prove}$. On input the common reference string $CRS = (crs_i)_{i\in[1,\lambda]}$, the number of instances N, a circuit $\mathtt{C} : \{0,1\}^\lambda \times \{0,1\}^W \to \{0,1\}$, and N witnesses $w_1, \ldots, w_N$, the proving algorithm computes $n = \lceil \log N \rceil$. Then it sets $\mathtt{C}' : \{0,1\}^\lambda \times \{0,1\}^W \to \{0,1\}$ be a circuit that

  $$\mathtt{C}'(i, w_i) = \begin{cases} \mathtt{C}(i, w_i) & \text{If } 1 \le i \le N \\ 1 & \text{Otherwise} \end{cases}$$

  sets $w_i = 0^W$ for $i \in [N+1, 2^n]$, and outputs $\pi \leftarrow \mathtt{Prove}'(crs_n, \mathtt{C}', w_1, \ldots, w_{2^n})$.
- $\mathtt{Verify}$. On input the common reference string $CRS = (crs_i)_{i\in[1,\lambda]}$, the number of instances N, a circuit $\mathtt{C} : \{0,1\}^\lambda \times \{0,1\}^W \to \{0,1\}$, and a proof π, the verification algorithm outputs 0 if N is not polynomial in λ. Otherwise, it computes $n = \lceil \log N \rceil$, sets $\mathtt{C}' : \{0,1\}^\lambda \times \{0,1\}^W \to \{0,1\}$ be a circuit that

  $$\mathtt{C}'(i, w_i) = \begin{cases} \mathtt{C}(i, w_i) & \text{If } 1 \le i \le N \\ 1 & \text{Otherwise} \end{cases}$$

  and outputs $\mathtt{Verify}'(crs_n, \mathtt{C}', \pi)$.
- $\mathtt{Extract}$. On input the trapdoor $TD = (td_i)_{i\in[1,\lambda]}$, the number of instances N, a circuit $\mathtt{C} : \{0,1\}^\lambda \times \{0,1\}^W \to \{0,1\}$, and a proof π, the extraction algorithm outputs $\perp$ if N is not polynomial in λ. Otherwise, it computes $n = \lceil \log N \rceil$, sets $\mathtt{C}' : \{0,1\}^\lambda \times \{0,1\}^W \to \{0,1\}$ be a circuit that

  $$\mathtt{C}'(i, w_i) = \begin{cases} \mathtt{C}(i, w_i) & \text{If } 1 \le i \le N \\ 1 & \text{Otherwise} \end{cases}$$

  and outputs $\mathtt{Extract}'(td_n, \mathtt{C}', \pi)$.

Security and efficiency of the index BARG constructed above comes from the security and efficiency of BI' directly. Note that here we further require that the running time of $\mathtt{Setup}'(1^\lambda, 2^n, 1^{C'}, \min(ind, 2^n))$ is bounded by $poly(\lambda, n, C')$ and this is satisfied by the index BARG constructed in [CJJ21].

3 Predicate Extractable Commitment

3.1 The Definition

In this section, we provide the definition of predicate extractable commitment. The notion combines a vector commitment [Mer87, CFM08, LY10, CF13] and a predicate extractable hash [BBK+23]: it allows one to commit a vector of values into a compact commitment and open the commitment at any positions; in addition, it checks if any of the values satisfy a predicate and enables one to extract its position from the commitment given a trapdoor.

Formally, a predicate extractable commitment for a predicate family $\mathcal{F}$ over a message space $\{0,1\}^m$ consists of the following PPT algorithms:

- $\mathtt{Setup}(1^\lambda, f) \to (crs, td)$: On input the security parameter 1^λ and a predicate circuit $f : \{0,1\}^m \to \{0,1\}$, the setup algorithm outputs a common reference string crs and a trapdoor td.
- $\mathtt{Commit}(crs, N, \boldsymbol{x}) \to (com, st)$: On input the common reference string crs, the vector length N, and a vector $\boldsymbol{x} \in (\{0,1\}^m)^N$, the *deterministic* commit algorithm outputs a commitment com and a state st.
- $\mathtt{Open}(crs, N, st, j) \to \rho$: On input the common reference string crs, the vector length N, a commitment state st, and an index $j \in [1, N]$, the *deterministic* open algorithm outputs a proof ρ.
- $\mathtt{Verify}(crs, N, com, j, x, \rho) \to b$: On input the common reference string crs, the vector length N, a commitment com, an index $j \in [1, N]$, a message $x \in \{0,1\}^m$, and a proof ρ, the *deterministic* verification algorithm outputs a bit $b \in \{0,1\}$.
- $\mathtt{Extract}(td, N, com) \to (a, j)$: On input the trapdoor td, the vector length N, and a commitment com, the *deterministic* extraction algorithm outputs a tuple $(a, j) \in \{(0,0)\} \cup \{(1,j)\}_{j \in [1,N]}$.

Besides, it should satisfy the following properties:

Definition 3.1 (Correctness). *For any predicate circuit $f \in \mathcal{F}$, any polynomial $N = N(\lambda)$, any vector $\boldsymbol{x} = (x_1, \ldots, x_N) \in (\{0,1\}^m)^N$, and any index $j \in [1, N]$:*

$$\Pr\left[\begin{array}{l}(crs, td) \leftarrow \mathtt{Setup}(1^\lambda, f);\\ (com, st) = \mathtt{Commit}(crs, N, \boldsymbol{x});: \quad \mathtt{Verify}(crs, N, com, j, x_j, \rho) = 1\\ \rho = \mathtt{Open}(crs, N, st, j)\end{array}\right] = 1$$

Definition 3.2 (Succinctness). *For any predicate circuit $f \in \mathcal{F}$, any polynomial $N = N(\lambda)$, any vector $\boldsymbol{x} = (x_1, \ldots, x_N) \in (\{0,1\}^m)^N$, and any index $j \in [1, N]$, let*

$$(crs, td) \leftarrow \mathtt{Setup}(1^\lambda, f),\ (com, st) = \mathtt{Commit}(crs, N, \boldsymbol{x}),\ \rho = \mathtt{Open}(crs, N, st, j)$$

Then we have

$$|crs| \leq poly(\lambda, m, |f|), \quad |com| + |\rho| \leq poly(\lambda, m, |f|, \log N)$$

and the running time of the verification algorithm on input $(crs, N, com, j, x_j, \rho)$ is bounded by $poly(\lambda, m, |f|, \log N)$.

Definition 3.3 (Binding). *For any predicate circuit $f \in \mathcal{F}$ and for all PPT adversaries $\mathcal{A}$, we have*

$$\Pr\left[\begin{array}{l}(crs, td) \leftarrow \mathtt{Setup}(1^\lambda, f);\\ (N, com, j, x, x', \rho, \rho') \leftarrow \mathcal{A}(crs);\\ b = \mathtt{Verify}(crs, N, com, j, x, \rho);\\ b' = \mathtt{Verify}(crs, N, com, j, x', \rho')\end{array} : \quad x \neq x' \wedge b = 1 \wedge b' = 1\right] \leq negl(\lambda)$$

Definition 3.4 (Predicate Hiding). *For all PPT adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, we have*

$$\Pr\left[\begin{array}{l}(f_0, f_1, state) \leftarrow \mathcal{A}_1(1^\lambda);\\ b \xleftarrow{\$} \{0,1\};\\ (crs, td) \leftarrow \mathtt{Setup}(1^\lambda, f_b);\\ b' \leftarrow \mathcal{A}_2(crs, state);\end{array} : \quad b = b'\right] \leq 1/2 + negl(\lambda)$$

where the adversary is required to submit f_0, f_1 s.t. $|f_0| = |f_1|$.

Definition 3.5 (Extractability). *For any predicate circuit $f \in \mathcal{F}$ and for all PPT adversaries $\mathcal{A}$, we have*

$$\Pr\left[\begin{array}{l}(crs, td) \leftarrow \mathtt{Setup}(1^\lambda, f);\\ (N, com, j, x, \rho) \leftarrow \mathcal{A}(crs);\\ (a, j') = \mathtt{Extract}(td, N, com);\\ b = \mathtt{Verify}(crs, N, com, j, x, \rho)\end{array} : \quad a = 0 \wedge f(x) = 1 \wedge b = 1\right] \leq negl(\lambda)$$

and

$$\Pr\left[\begin{array}{l}(crs, td) \leftarrow \mathtt{Setup}(1^\lambda, f);\\ (N, com, j, x, \rho) \leftarrow \mathcal{A}(crs);\\ (a, j') = \mathtt{Extract}(td, N, com);\\ b = \mathtt{Verify}(crs, N, com, j, x, \rho)\end{array} : \begin{array}{l}a = 1 \wedge j = j' \wedge\\ f(x) = 0 \wedge b = 1\end{array}\right] \leq negl(\lambda)$$

3.2 The Construction

In this section, we present our construction of predicate extractable commitment. Let λ be the security parameter. Let m, k, c, d, F be positive integers that are polynomial in λ. Let $\mathcal{F}$ be a family of circuits that have m-bit input, 1-bit output, and F wires. Our construction is built on the following building blocks:

- A collision resistant hash $\mathsf{H} = (\mathsf{H}.\mathtt{KeyGen}, \mathsf{H}.\mathtt{Eval})$ with output space $\{0,1\}^k$.
- A leveled fully homomorphic encryption scheme $\mathsf{HE} = (\mathsf{HE}.\mathtt{KeyGen}, \mathsf{HE}.\mathtt{Enc}, \mathsf{HE}.\mathtt{Dec}, \mathsf{HE}.\mathtt{Eval}, \mathsf{HE}.\mathtt{EncV}, \mathsf{HE}.\mathtt{DecV}, \mathsf{HE}.\mathtt{EvalC})$ with message space $\{0,1\}$ and ciphertext space $\{0,1\}^c$.

The construction also relies on the two circuits $\mathtt{C}_0$ and $\mathtt{C}_1$ defined in Figure 2 and Figure 3 respectively. Both $\mathtt{C}_0$ and $\mathtt{C}_1$ will output a tuple $(b, j, t) \in \{0,1\} \times \{0,1\}^\lambda \times \{0,1\}^k$. Suppose that the predicate circuit f is selected from $\mathcal{F}$ and let d be the upper bound of the depths of $\mathtt{C}_0$ and $\mathtt{C}_1$, then d can be bounded by an a priori bounded polynomials in λ, m and F.[10] Besides, the sizes of the two circuits can be bounded by $poly(\lambda, d, m, F)$, which can be further bounded by an a priori bounded polynomials in λ, m and F.

$\mathtt{C}_0$

Constant: $hk, x \in \{0,1\}^m, v$

Input: f

1. $t = \mathsf{H}.\mathtt{Eval}(hk, x)$.
2. If $f(x) = 0$: Output $(0, 0, t)$.
3. Output $(1, v+1, t)$.

Fig. 2 The circuit $\mathtt{C}_0$.

We construct the predicate extractable commitment $\mathsf{PC} = (\mathtt{Setup}, \mathtt{Commit}, \mathtt{Open}, \mathtt{Verify}, \mathtt{Extract})$, which has message space $\{0,1\}^m$ and supports predicate circuits from $\mathcal{F}$, as follows:

- $\mathtt{Setup}$. On input the security parameter 1^λ and a circuit $f \in \mathcal{F}$, the setup algorithm generates

$$(pk_i, sk_i) \leftarrow \mathsf{HE}.\mathtt{KeyGen}(1^\lambda, 1^d)$$

for $i \in [0, \lambda]$ and

$$\boldsymbol{c}_i \leftarrow \mathsf{HE}.\mathtt{EncV}(pk_i, sk_{i-1})$$

for $i \in [1, \lambda]$. Then it computes

$$\boldsymbol{c}_f \leftarrow \mathsf{HE}.\mathtt{EncV}(pk_0, f), \quad hk \leftarrow \mathsf{H}.\mathtt{KeyGen}(1^\lambda)$$

[10] The circuit $\mathtt{C}_1$ needs to run the decryption algorithm of HE, whose parameters depend on the depth d here. Nonetheless, the depth of the decryption algorithm of previous LWE-based leveled FHE schemes (e.g., [BV11a, GSW13]) is in $O(\log \lambda)$. Therefore, it is safe to assume that the depth of the decryption circuit can be bounded by λ.

$\mathtt{C}_1$

Constant: $hk, \boldsymbol{ct}_L \in (\{0,1\}^c)^{1+\lambda+k}, \boldsymbol{ct}_R \in (\{0,1\}^c)^{1+\lambda+k}, v$

Input: sk

1. $(b_L, ind_L, t_L) = \mathsf{HE}.\mathtt{DecV}(sk, \boldsymbol{ct}_L)$.
2. $(b_R, ind_R, t_R) = \mathsf{HE}.\mathtt{DecV}(sk, \boldsymbol{ct}_R)$.
3. $t = \mathsf{H}.\mathtt{Eval}(hk, t_L \| t_R)$.
4. If $b_L = 1$ and $v\|0$ is a prefix of $ind_L - 1$:
   (a) Output $(1, ind_L, t)$.
5. If $b_R = 1$ and $v\|1$ is a prefix of $ind_R - 1$:
   (a) Output $(1, ind_R, t)$.
6. Output $(0, 0, t)$.

Fig. 3 The circuit $\mathtt{C}_1$.

and outputs

$$crs = ((pk_i)_{i\in[0,\lambda]}, (\boldsymbol{c}_i)_{i\in[1,\lambda]}, \boldsymbol{c}_f, hk), \quad td = (sk_i)_{i\in[0,\lambda]}$$

- `Commit`. On input the common reference string

$$crs = ((pk_i)_{i\in[0,\lambda]}, (\boldsymbol{c}_i)_{i\in[1,\lambda]}, \boldsymbol{c}_f, hk)$$

a polynomial $N = N(\lambda)$, and a vector of N messages

$$\boldsymbol{x} = (x_1, \ldots, x_N) \in (\{0,1\}^m)^N$$

the commit algorithm proceeds as follows:

1. $n = \lceil \log N \rceil$.
2. For $v \in [0, N-1]$:
   (a) $\boldsymbol{ct}_{0,v} = \mathsf{HE}.\mathtt{EvalC}(pk_0, \mathtt{C}_0[hk, x_{v+1}, v], \boldsymbol{c}_f)$.
3. For $v \in [N, 2^n - 1]$:
   (a) $\boldsymbol{ct}_{0,v} = \mathsf{HE}.\mathtt{EncV}(pk_0, 0^{1+\lambda+k}; \mathbf{0})$.
4. For $i \in [1, n-1]$:
   (a) For $v \in \{0,1\}^{n-i}$:
      i. $\boldsymbol{ct}_{i,v} = \mathsf{HE}.\mathtt{EvalC}(pk_i, \mathtt{C}_1[hk, \boldsymbol{ct}_{i-1,v\|0}, \boldsymbol{ct}_{i-1,v\|1}, v], \boldsymbol{c}_i)$.
5. $\boldsymbol{ct}_n = \mathsf{HE}.\mathtt{EvalC}(pk_n, \mathtt{C}_1[hk, \boldsymbol{ct}_{n-1,0}, \boldsymbol{ct}_{n-1,1}, \epsilon], \boldsymbol{c}_n)$.

where we use ϵ to denote an empty string. Finally, it outputs

$$com = \boldsymbol{ct}_n, \quad st = (\boldsymbol{ct}_{i,v})_{i\in[0,n-1], v\in\{0,1\}^{n-i}}$$

- `Open`. On input the common reference string crs, a polynomial $N = N(\lambda)$, a commitment state

$$st = (\boldsymbol{ct}_{i,v})_{i\in[0,n-1], v\in\{0,1\}^{n-i}}$$

where $n = \lceil \log N \rceil$, and an index $j \in [1, N]$, the open algorithm outputs $\perp$ if N is not polynomial in λ or if $j \notin [1, N]$. Otherwise, let $u = j - 1$ and let $(u_1, \ldots, u_n)$ be the binary representation of u (i.e., $u_i \in \{0, 1\}$ for $i \in [1, n]$ and $u = \sum_{i=1}^{n} u_i \cdot 2^{n-i}$). Also define

$$u^{(i)} = (u_1, u_2, \ldots, u_{n-i-1}, u_{n-i}), \quad \bar{u}^{(i)} = (u_1, u_2, \ldots, u_{n-i-1}, 1 - u_{n-i})$$

for $i \in [0, n-1]$. Then the algorithm outputs

$$\rho = (\boldsymbol{ct}_{i,u^{(i)}}, \boldsymbol{ct}_{i,\bar{u}^{(i)}})_{i \in [0,n-1]}$$

- `Verify`. On input the common reference string

$$crs = ((pk_i)_{i \in [0,\lambda]}, (\boldsymbol{c}_i)_{i \in [1,\lambda]}, \boldsymbol{c}_f, hk)$$

a polynomial $N = N(\lambda)$, a commitment $com \in (\{0,1\}^c)^{1+\lambda+k}$, an index $j \in [1, N]$, an input $x \in \{0,1\}^m$, and a proof $\rho \in (\{0,1\}^c)^{2n \cdot (1+\lambda+k)}$, where $n = \lceil \log N \rceil$, the verification algorithm **outputs** 0 if N is not polynomial in λ or if $j \notin [1, N]$. Otherwise, the algorithm parses

$$com = \boldsymbol{ct}_n, \quad \rho = (\boldsymbol{ct}_i, \overline{\boldsymbol{ct}}_i)_{i \in [0,n-1]}$$

Then, let $u = j - 1$ and let $(u_1, \ldots, u_n)$ be the binary representation of u; also, let $u^{(i)} = (u_1, u_2, \ldots, u_{n-i-1}, u_{n-i})$ for $i \in [0, n-1]$ and let $u^{(n)}$ be the empty string ϵ. The algorithm proceeds as follows:

1. If $\boldsymbol{ct}_0 \neq \mathsf{HE}.\mathtt{EvalC}(pk_0, \mathtt{C}_0[hk, x, u], \boldsymbol{c}_f)$: **Output** 0.
2. For $i \in [1, n]$:
   (a) $\boldsymbol{ct}_{i-1,u_{n-i+1}} = \boldsymbol{ct}_{i-1}$.
   (b) $\boldsymbol{ct}_{i-1,1-u_{n-i+1}} = \overline{\boldsymbol{ct}}_{i-1}$.
   (c) If $\boldsymbol{ct}_i \neq \mathsf{HE}.\mathtt{EvalC}(pk_i, \mathtt{C}_1[hk, \boldsymbol{ct}_{i-1,0}, \boldsymbol{ct}_{i-1,1}, u^{(i)}], \boldsymbol{c}_i)$: **Output** 0.

Finally, it **outputs** 1 if the algorithm does not abort in above procedures.

- `Extract`. On input the trapdoor $td = (sk_i)_{i \in [0,\lambda]}$, a polynomial $N = N(\lambda)$, and a commitment $com = \boldsymbol{ct} \in (\{0,1\}^c)^{1+\lambda+k}$, the extraction algorithm outputs $(0, 0)$ if N is not polynomial in λ. Then it computes $n = \lceil \log N \rceil$ and computes

$$(b, ind, t) = \mathsf{HE}.\mathtt{DecV}(sk_n, \boldsymbol{ct})$$

It outputs $(0, 0)$ if $b = 0$ or if $ind \notin [1, N]$; otherwise, it outputs (b, ind).

Theorem 3.1. *If* H *is a collision resistant hash function and* HE *is a leveled FHE scheme with IND-CPA security, then* PC *is a secure predicate extractable commitment scheme.*

We present proof of Theorem 3.1 in Appendix A. The binding property and the predicate hiding property comes from the collision resistance of H and the IND-CPA security of HE directly. Next, we briefly explain why the extractability holds. Given an input x committed at position j of a commitment $com = \boldsymbol{ct}_n$ with a valid proof $\rho = (\boldsymbol{ct}_i, \overline{\boldsymbol{ct}}_i)_{i \in [0,n-1]}$, we will consider the following two cases:

- First, if $f(x) = 1$, then $\boldsymbol{ct}_0$ will encrypt $(1, j, *)$. This valid position information (or another valid position information contained in $\overline{\boldsymbol{ct}}_0$) will be transmitted to $\boldsymbol{ct}_1$, thus $\boldsymbol{ct}_1$ will also contain a valid position information. We can continue to argue this for $i \in [2, n]$ and prove that $\boldsymbol{ct}_n$ will contain valid position information, i.e., the extraction algorithm will not get $(0, 0)$.
- On the other hand, if $f(x) = 0$, then $\boldsymbol{ct}_0$ will encrypt $(0, 0, *)$. That means, $\boldsymbol{ct}_1$ cannot get $(1, j, *)$ from $\boldsymbol{ct}_0$; in addition, even if the adversary sets $\overline{\boldsymbol{ct}}_0$ as an encryption of $(1, j, *)$, it will not be selected by $\boldsymbol{ct}_1$, because this information cannot pass the prefix check in the circuit $\mathtt{C}_1$. we can continue to argue this for $i \in [2, n]$ and prove that $\boldsymbol{ct}_n$ will not encrypt $(1, j, *)$, i.e., the extraction algorithm can never get a "false positive" report.

Note that both the collision resistant hash and the leveled FHE scheme can be instantiated from the standard LWE assumption, thus we have

Corollary 3.1. *Assuming the standard LWE assumption, there exist predicate extractable commitment schemes.*

4 Fully Homomorphic Encryption with Chosen-Ciphertext Security

4.1 The Definition

In this section, we provide the definition of FHE with chosen-ciphertext security. We consider a new security notion called *input-verifiable chosen-ciphertext* security. It allow the adversary to query the decryption oracle both before and after seeing the challenge ciphertext. However, in the second phase, the adversary should also provide the original input ciphertexts that are homomorphically evaluated to the queried ciphertext, and the query will be answered only if the input ciphertexts do not include the challenge ciphertext. We do *not* assume the adversary to be honest when submitting the input ciphertexts, and we need a verification algorithm that can publicly verify if the input ciphertexts are correct.

Formally, an IV-CCA secure fully homomorphic encryption with message space $\{0, 1\}$ and ciphertext space $\mathcal{C}, \mathcal{C}'$ consists of the following PPT algorithms:

- $\mathtt{KeyGen}(1^\lambda) \to (pk, sk)$. On input the security parameter 1^λ, the key generation algorithm outputs a public key pk and a secret key sk.
- $\mathtt{Enc}(pk, \mu) \to ct$. On input the public key pk and a message $\mu \in \{0, 1\}$, the encryption algorithm outputs a ciphertext $ct \in \mathcal{C}$.
- $\mathtt{Dec}(sk, ct) \to \mu$. On input the secret key sk and a ciphertext $ct \in \mathcal{C} \cup \mathcal{C}'$, the decryption algorithm outputs a message $\mu \in \{0, 1, \perp\}$, where we use $\perp$ to denote that the ciphertext is illegal.
- $\mathtt{Eval}(pk, \mathtt{C}, \boldsymbol{ct}) \to ct'$. For any polynomial l, on input the public key pk, a circuit $\mathtt{C} : \{0, 1\}^l \to \{0, 1\}$, and a vector of l ciphertexts $\boldsymbol{ct} = (ct_1, \ldots, ct_l) \in \mathcal{C}^l$, the evaluation algorithm outputs a ciphertext $ct' \in \mathcal{C}'$.

- $\texttt{Verify}(pk, \boldsymbol{ct}, ct') \to b$. For any polynomial l, on input the public key pk, a vector of l ciphertexts $\boldsymbol{ct} = (ct_1, \ldots, ct_l) \in \mathcal{C}^l$, and a ciphertext $ct' \in \mathcal{C} \cup \mathcal{C}'$, the verification algorithm outputs a bit $b \in \{0,1\}$.

Besides, it should satisfy the correctness, compactness, and security properties defined as follows.

Correctness. First, we require that a freshly encrypted ciphertext can be decrypted correctly and it can pass the verification algorithm if we set itself as the input ciphertext.

Definition 4.1 (Standard Correctness). *For any messages $\mu \in \{0,1\}$,*

$$\Pr[(pk, sk) \leftarrow \texttt{KeyGen}(1^\lambda), ct \leftarrow \texttt{Enc}(pk, \mu) : \texttt{Dec}(sk, ct) = \mu \wedge \texttt{Verify}(pk, ct, ct) = 1] = 1$$

We also require that if we evaluate a circuit over a vector of input ciphertexts and decrypt the resulting output ciphertext, we will get the result of evaluating the circuit over the plaintexts encrypted in the input ciphertexts. In addition, the verification algorithm will output 1 given the correct input/output ciphertexts.

Definition 4.2 (Evaluation Correctness). *For any polynomials l, any messages $\mu_1, \ldots, \mu_l \in \{0,1\}$, and any circuit $\texttt{C} : \{0,1\}^l \to \{0,1\}$ of polynomial size,*

$$\Pr\left[\begin{array}{l|l} (pk, sk) \leftarrow \texttt{KeyGen}(1^\lambda); & \texttt{Dec}(sk, ct') = \texttt{C}(\mu_1, \ldots, \mu_l) \wedge \\ \forall i \in [1,l], ct_i \leftarrow \texttt{Enc}(pk, \mu_i); & \texttt{Verify}(pk, (ct_i)_{i\in[1,l]}, ct') = 1 \\ ct' \leftarrow \texttt{Eval}(pk, \texttt{C}, (ct_i)_{i\in[1,l]}) & \end{array}\right] = 1$$

Compactness. The compactness requires that the size of an evaluated ciphertext and the runtime needed to decrypt it only grow polylogarithmically in the size of the circuit and the number of input ciphertexts.

Definition 4.3 (Compactness). *For any polynomial l, N, any messages $\mu_1, \ldots, \mu_l \in \{0,1\}$, and any circuit $\texttt{C} : \{0,1\}^l \to \{0,1\}$ with N wires, let*

$$(pk, sk) \leftarrow \texttt{KeyGen}(1^\lambda), \forall i \in [1,l], ct_i \leftarrow \texttt{Enc}(pk, \mu_i), ct' \leftarrow \texttt{Eval}(pk, \texttt{C}, (ct_i)_{i\in[1,l]})$$

Then both the size of ct' and the running time of the decryption algorithm on input (sk, ct') are bounded by $poly(\lambda, \log N, \log l)$.

Security. Next, we define the IV-CCA security, which relaxes the standard CCA2 security by rejecting queries evaluated from the challenge ciphertext.

Definition 4.4 (IV-CCA Security). *For any PPT adversary $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$, we have:*

$$\Pr\left[\begin{array}{l} (pk, sk) \leftarrow \texttt{KeyGen}(1^\lambda); \\ (\mu_0^*, \mu_1^*, state) \leftarrow \mathcal{A}_1^{\texttt{D}_1(\cdot)}(pk); \\ b \xleftarrow{\$} \{0,1\}; \\ ct^* \leftarrow \texttt{Enc}(pk, \mu_b^*) \end{array} : \mathcal{A}_2^{\texttt{D}_2(\cdot,\cdot)}(ct^*, state) = b\right] \leq 1/2 + negl(\lambda)$$

Here, the oracle $\mathtt{D}_1$ *takes as input a ciphertext* $ct \in \mathcal{C} \cup \mathcal{C}'$ *and returns* $\mathtt{Dec}(sk, ct)$*. The oracle* $\mathtt{D}_2$ *takes as input a vector of ciphertexts* $\boldsymbol{ct} \in \mathcal{C}^l$ *for some positive polynomial* l *and a ciphertext* $ct' \in \mathcal{C} \cup \mathcal{C}'$*, and proceeds as follows:*

1. $\mu \leftarrow \mathtt{Dec}(sk, ct')$.
2. *If* $\mu = \perp$*: Return* $\perp$.
3. *If* $\mathtt{Verify}(pk, \boldsymbol{ct}, ct') = 0 \vee ct^* \in \boldsymbol{ct}$*: Return* $\top$.
4. *Return* μ

Remark 4.1. Our definition of IV-CCA security implies several CCA security notions in the literature. In particular, it is easy to see it implies the standard CCA1 security, which only allows the adversary to query $\mathtt{D}_1$ before seeing the challenge ciphertext, and the so-called CCA1.5 security [DDA13, MN24], where the adversary can query $\mathtt{D}_1$ before seeing the challenge ciphertext and can learn if its queried ciphertext is valid or not after seeing the challenge ciphertext[11]. In addition, our definition is strictly stronger than the CCA1/CCA1.5 security. To see this, assume that we have an IV-CCA secure FHE scheme and replace its verification algorithm with an algorithm that always outputs 1. The scheme is still CCA1/CCA1.5 secure since the verification algorithm is not used to achieve them; but it is not IV-CCA secure because now the adversary can simply submit (ct, ct^*) to $\mathtt{D}_2$ and obtain decryption of ct^*, where ct can be any ciphertext that is not equal to ct^*.

Remark 4.2. Compared to the definition of the CPA-secure FHE scheme, which is recalled in Sec. 2.3, we make the following modifications in addition to defining IV-CCA security:

- We need an additional verification algorithm that checks if a target ciphertext is evaluated from a vector of input ciphertexts. We also need the verification algorithm to output 1 if the inputs are honestly generated.
- There are two types of ciphertexts: ciphertexts that are encrypted freshly by the encryption algorithm and ciphertexts that are output by the evaluation algorithm. In addition, the evaluation algorithm only admits fresh ciphertexts. That is, we only consider single-hop evaluations here.
- Our decryption algorithm will reject some illegal ciphertexts, i.e., it will output $\perp$ on some ciphertext.
- Our evaluation algorithm will expand the ciphertext size. More precisely, if we run a circuit with N wires on l input ciphertexts, the size of the resulting ciphertext ct (and the costs for decrypting ct) will grow polylogarithmically in N and l. Nonetheless, the ciphertext size and the decryption time are still bounded by a pre-defined polynomial $L(\lambda, \log N, \log l) \leq L(\lambda, \lambda, \lambda)$, where the inequality comes from the fact that $l, N \leq 2^\lambda$.

[11] More precisely, in the second phase, the adversary is given access to an oracle that takes as input a ciphertext ct and returns if $\mathtt{Dec}(sk, ct) = \perp$ or not.

4.2 The Construction

In this section, we present our construction of FHE with IV-CCA security. Let λ be the security parameter. Let c, m, C be positive integers that are polynomial in λ s.t. $m = 2c + \lambda$.

Our construction is built on the following building blocks:

- A fully homomorphic encryption scheme $\mathsf{HE} = (\mathsf{HE.KeyGen}, \mathsf{HE.Enc}, \mathsf{HE.Dec}, \mathsf{HE.Eval})$ with message space $\{0,1\}$, ciphertext space $\{0,1\}^c$, and encryption randomness space $\mathcal{R}_{\mathtt{Enc}}$.
- A simulation sound NIZK proof $\mathsf{ZK} = (\mathsf{ZK.Setup}, \mathsf{ZK.Prove}, \mathsf{ZK.Verify}, \mathsf{ZK.S1}, \mathsf{ZK.S2})$ for a language $\mathcal{L}$, where

$$\mathcal{L} = \{(pk_0, pk_1, ct_0, ct_1) : \exists(\mu, r_0, r_1) \in \{0,1\} \times \mathcal{R}_{\mathtt{Enc}} \times \mathcal{R}_{\mathtt{Enc}}, \\ ct_0 = \mathsf{HE.Enc}(pk_0, \mu; r_0), ct_1 = \mathsf{HE.Enc}(pk_1, \mu; r_1)\}$$

- A predicate extractable commitment $\mathsf{PC} = (\mathsf{PC.Setup}, \mathsf{PC.Commit}, \mathsf{PC.Open}, \mathsf{PC.Verify}, \mathsf{PC.Extract})$, which has message space $\{0,1\}^m$ and supports predicate circuits from $\mathcal{F}$ defined below.
- A somewhere extractable index BARG $\mathsf{BI} = (\mathsf{BI.Setup}, \mathsf{BI.Prove}, \mathsf{BI.Verify}, \mathsf{BI.Extract})$ for the (satisfiability of circuits in the) circuit family $\mathtt{C}_e$ defined in Figure 5.

$\mathtt{C}_c$

Constant: $sk_0, sk_1, level$

Input: ct_0, ct_1, d

1. If $d \leq level \wedge \mathsf{HE.Dec}(sk_0, ct_0) \neq \mathsf{HE.Dec}(sk_1, ct_1)$:
   (a) Output 1.
2. Output 0.

Fig. 4 The circuit $\mathtt{C}_c$.

Let $\mathtt{C}_c : \{0,1\}^c \times \{0,1\}^c \times \{0,1\}^\lambda \to \{0,1\}$ be the circuit defined in Figure 4. Here, d and *level* will denote the layer in a circuit and is polynomial in λ, so we can encode them into λ-bit binary strings. It is easy to see, the size of $\mathtt{C}_c$ is bounded by $poly(\lambda)$. Let $\mathtt{C}_d$ be a dummy circuit that always output 0 and is padded to have the same size as $\mathtt{C}_c$. Then we define $\mathcal{F}$ as a family of circuits that contains $\mathtt{C}_d$ and all possible instances of $\mathtt{C}_c$.

Let $\mathtt{C}_e$ be the circuit defined in Figure 5. In the construction below, we will use $\mathtt{C}_e$ to check the correctness of each wire in the homomorphic evaluation. Here, l and N are the number of input wires and the number of wires of the evaluated circuit respectively. Then for an input wire, the circuit will check that the wire is properly committed in the commitments, the associated NIZK proof for the ciphertext is valid, and the wire is recognized as at layer 0 (i.e., the input layer); for an internal wire, which is the output of a NAND gate, the circuit will confirm that all the related wires are properly committed in the commitments, and the

$\mathtt{C}_e$

Constant: $pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, com^{(1)}, com^{(2)}, com^{(3)}, l, N$

Input: ind, w

1. Parse $w = (\mathcal{C} = (ct_0, ct_1, d), \sigma, \rho^{(1)}, \rho^{(2)}, \rho^{(3)},$
   $L, \mathcal{C}_L = (ct_{L,0}, ct_{L,1}, d_L), \rho^{(1)}_L, \rho^{(2)}_L,$
   $R, \mathcal{C}_R = (ct_{R,0}, ct_{R,1}, d_R), \rho^{(1)}_R, \rho^{(2)}_R)$
2. If $ind \leq l$: Output 1 iff all conditions below are satisfied:
   (a) $\mathsf{PC}.\mathtt{Verify}(crs^{(1)}_{\mathsf{PC}}, N, com^{(1)}, ind, \mathcal{C}, \rho^{(1)}) = 1$;
   (b) $\mathsf{PC}.\mathtt{Verify}(crs^{(2)}_{\mathsf{PC}}, N, com^{(2)}, ind, \mathcal{C}, \rho^{(2)}) = 1$;
   (c) $\mathsf{PC}.\mathtt{Verify}(crs^{(3)}_{\mathsf{PC}}, l, com^{(3)}, ind, \mathcal{C}, \rho^{(3)}) = 1$;
   (d) $\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_0, ct_1), \sigma) = 1$;
   (e) $d = 0$.
3. If $l < ind \leq N$: Output 1 iff all conditions below are satisfied:
   (a) $\mathsf{PC}.\mathtt{Verify}(crs^{(1)}_{\mathsf{PC}}, N, com^{(1)}, ind, \mathcal{C}, \rho^{(1)}) = 1$;
   (b) $\mathsf{PC}.\mathtt{Verify}(crs^{(2)}_{\mathsf{PC}}, N, com^{(2)}, ind, \mathcal{C}, \rho^{(2)}) = 1$;
   (c) $\mathsf{PC}.\mathtt{Verify}(crs^{(1)}_{\mathsf{PC}}, N, com^{(1)}, L, \mathcal{C}_L, \rho^{(1)}_L) = 1$;
   (d) $\mathsf{PC}.\mathtt{Verify}(crs^{(2)}_{\mathsf{PC}}, N, com^{(2)}, L, \mathcal{C}_L, \rho^{(2)}_L) = 1$;
   (e) $\mathsf{PC}.\mathtt{Verify}(crs^{(1)}_{\mathsf{PC}}, N, com^{(1)}, R, \mathcal{C}_R, \rho^{(1)}_R) = 1$;
   (f) $\mathsf{PC}.\mathtt{Verify}(crs^{(2)}_{\mathsf{PC}}, N, com^{(2)}, R, \mathcal{C}_R, \rho^{(2)}_R) = 1$;
   (g) $ct_0 = \mathsf{HE}.\mathtt{Eval}(pk_0, ct_{L,0}, ct_{R,0})$;
   (h) $ct_1 = \mathsf{HE}.\mathtt{Eval}(pk_1, ct_{L,1}, ct_{R,1})$;
   (i) $d_L \leq d - 1$;
   (j) $d_R \leq d - 1$;
   (k) $d \geq 1$.

Fig. 5 The circuit $\mathtt{C}_e$.

wire and the layer are correctly set[12]. The size of the circuit can be bounded by $poly(\lambda, |\mathtt{C}_c|, \log N)$, due to the succinctness of PC. The size of $\mathtt{C}_c$ is bounded by a polynomial in λ. In addition, as N is polynomial in λ, we can assume that $N \leq 2^\lambda$ and have $\log N \leq \lambda$. Therefore, the size of $\mathtt{C}_e$ can be bounded by a predefined polynomial $C = C(\lambda)$.

We construct the FHE scheme $\mathsf{FHE} = (\mathtt{KeyGen}, \mathtt{Enc}, \mathtt{Dec}, \mathtt{Eval}, \mathtt{Verify})$ with message space $\{0, 1\}$ as follows:

- $\mathtt{KeyGen}$. On input the security parameter 1^λ, the key generation algorithm generates:
  1. $(pk_0, sk_0) \leftarrow \mathsf{HE}.\mathtt{KeyGen}(1^\lambda)$.
  2. $(pk_1, sk_1) \leftarrow \mathsf{HE}.\mathtt{KeyGen}(1^\lambda)$.
  3. $crs_{\mathsf{ZK}} \leftarrow \mathsf{ZK}.\mathtt{Setup}(1^\lambda)$.

[12] We only check if the layer of the output wire of a NAND gate is higher than the layers of the input wires, and that is sufficient for our construction.

4. $(crs^{(1)}_{\mathsf{PC}}, td^{(1)}_{\mathsf{PC}}) \leftarrow \mathsf{PC}.\mathtt{Setup}(1^\lambda, \mathtt{C}_d)$.
5. $(crs^{(2)}_{\mathsf{PC}}, td^{(2)}_{\mathsf{PC}}) \leftarrow \mathsf{PC}.\mathtt{Setup}(1^\lambda, \mathtt{C}_d)$.
6. $(crs^{(3)}_{\mathsf{PC}}, td^{(3)}_{\mathsf{PC}}) \leftarrow \mathsf{PC}.\mathtt{Setup}(1^\lambda, \mathtt{C}_d)$.
7. $(crs_{\mathsf{BI}}, td_{\mathsf{BI}}) \leftarrow \mathsf{BI}.\mathtt{Setup}(1^\lambda, 1^C, 1)$.

Then it outputs the public key

$$PK = (pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, crs_{\mathsf{BI}})$$

and the secret key

$$SK = (sk_0, PK)$$

- $\mathtt{Enc}$. On input the public key $PK = (pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, crs_{\mathsf{BI}})$ and a message $\mu \in \{0,1\}$, the encryption algorithm first samples $r_0, r_1 \xleftarrow{\$} \mathcal{R}_{\mathtt{Enc}}$. Then it computes

$$ct_0 = \mathsf{HE}.\mathtt{Enc}(pk_0, \mu; r_0), \quad ct_1 = \mathsf{HE}.\mathtt{Enc}(pk_1, \mu; r_1)$$

$$\sigma \leftarrow \mathsf{ZK}.\mathtt{Prove}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_0, ct_1), (\mu, r_0, r_1))$$

and outputs

$$CT = (0, ct_0, ct_1, \sigma)$$

- $\mathtt{Eval}$. For any polynomial l, N, on input the public key

$$PK = (pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, crs_{\mathsf{BI}})$$

a circuit $\mathtt{C} = (1^l, 1, (L_i, R_i)_{i\in[l+1,N]})$ (i.e., a circuit with l-bit input, 1-bit output, and N wires), and a vector of l ciphertexts $CT_1, \ldots, CT_l$, where $CT_i = (0, ct_{i,0}, ct_{i,1}, \sigma_i)$, the evaluation algorithm proceeds as follows:

1. For $i \in [1, l]$:
   (a) If $\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_{i,0}, ct_{i,1}), \sigma_i) = 0$:
      i. **Output** $\perp$.
   (b) $d_i = 0$.
   (c) $\mathcal{C}_i = (ct_{i,0}, ct_{i,1}, d_i)$.
2. For $i \in [l+1, N]$:
   (a) $ct_{i,0} = \mathsf{HE}.\mathtt{Eval}(pk_0, ct_{L_i,0}, ct_{R_i,0})$.
   (b) $ct_{i,1} = \mathsf{HE}.\mathtt{Eval}(pk_1, ct_{L_i,1}, ct_{R_i,1})$.
   (c) $d_i = \max(d_{L_i}, d_{R_i}) + 1$.
   (d) $\mathcal{C}_i = (ct_{i,0}, ct_{i,1}, d_i)$.
3. $(com^{(1)}, st^{(1)}) = \mathsf{PC}.\mathtt{Commit}(crs^{(1)}_{\mathsf{PC}}, N, (\mathcal{C}_i)_{i\in[1,N]})$.
4. $(com^{(2)}, st^{(2)}) = \mathsf{PC}.\mathtt{Commit}(crs^{(2)}_{\mathsf{PC}}, N, (\mathcal{C}_i)_{i\in[1,N]})$.
5. For $i \in [1, N]$:
   (a) $\rho^{(1)}_i = \mathsf{PC}.\mathtt{Open}(crs^{(1)}_{\mathsf{PC}}, N, st^{(1)}, i)$.
   (b) $\rho^{(2)}_i = \mathsf{PC}.\mathtt{Open}(crs^{(2)}_{\mathsf{PC}}, N, st^{(2)}, i)$.

6. $(com^{(3)}, st^{(3)}) = \mathsf{PC}.\mathtt{Commit}(crs_{\mathsf{PC}}^{(3)}, l, (\mathcal{C}_i)_{i\in[1,l]})$.
7. For $i \in [1, l]$:
   (a) $\rho_i^{(3)} = \mathsf{PC}.\mathtt{Open}(crs_{\mathsf{PC}}^{(3)}, l, st^{(3)}, i)$.
8. Define $ct_\perp = 0^c$, $\sigma_\perp = 0^{|\sigma_1|}$, $\rho_\perp = 0^{|\rho_1^{(1)}|}$, $\rho'_\perp = 0^{|\rho_1^{(3)}|}$.
9. For $i \in [1, l]$:
   (a) $w_i = (\mathcal{C}_i, \sigma_i, \rho_i^{(1)}, \rho_i^{(2)}, \rho_i^{(3)},$
   $0, \mathcal{C}_L = (ct_\perp, ct_\perp, 0), \rho_\perp, \rho_\perp,$
   $0, \mathcal{C}_R = (ct_\perp, ct_\perp, 0), \rho_\perp, \rho_\perp)$
10. For $i \in [l + 1, N]$:
   (a) $w_i = (\mathcal{C}_i, \sigma_\perp, \rho_i^{(1)}, \rho_i^{(2)}, \rho'_\perp,$
   $L_i, \mathcal{C}_{L_i}, \rho_{L_i}^{(1)}, \rho_{L_i}^{(2)},$
   $R_i, \mathcal{C}_{R_i}, \rho_{R_i}^{(1)}, \rho_{R_i}^{(2)})$
11. $\mathtt{P} = \mathtt{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs_{\mathsf{PC}}^{(1)}, crs_{\mathsf{PC}}^{(2)}, crs_{\mathsf{PC}}^{(3)}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$
12. $\Pi \leftarrow \mathsf{BI}.\mathtt{Prove}(crs_{\mathsf{BI}}, N, \mathtt{P}, w_1, \ldots, w_N)$.

Finally, the algorithm **outputs** the ciphertext

$$CT = (1, \mathcal{C}_N, com^{(1)}, com^{(2)}, com^{(3)}, \rho_N^{(1)}, \rho_N^{(2)}, \Pi, l, N)$$

- $\mathtt{Dec}$. On input the secret key $SK = (sk_0; pk_0, pk_1, crs_{\mathsf{ZK}}, crs_{\mathsf{PC}}^{(1)}, crs_{\mathsf{PC}}^{(2)}, crs_{\mathsf{PC}}^{(3)}, crs_{\mathsf{BI}})$ and a ciphertext CT, the algorithm proceeds as follows:
  1. If $CT = (0, ct_0, ct_1, \sigma)$:
     (a) If $\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_0, ct_1), \sigma) = 0$:
        i. **Output** $\perp$.
     (b) **Output** $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0)$.
  2. If $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$:
     (a) If N is not polynomial in λ:
        i. **Output** $\perp$.
     (b) If $l > N \vee d > N$:
        i. **Output** $\perp$.
     (c) $\mathtt{P} = \mathtt{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs_{\mathsf{PC}}^{(1)}, crs_{\mathsf{PC}}^{(2)}, crs_{\mathsf{PC}}^{(3)}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$
     (d) If $\mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathtt{P}, \Pi) = 0$:
        i. **Output** $\perp$.
     (e) If $\mathsf{PC}.\mathtt{Verify}(crs_{\mathsf{PC}}^{(1)}, N, com^{(1)}, N, \mathcal{C}, \rho^{(1)}) = 0$:
        i. **Output** $\perp$.
     (f) If $\mathsf{PC}.\mathtt{Verify}(crs_{\mathsf{PC}}^{(2)}, N, com^{(2)}, N, \mathcal{C}, \rho^{(2)}) = 0$:
        i. **Output** $\perp$.
     (g) **Output** $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0)$.

- $\mathtt{Verify}$. On input the public key $PK = (pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, crs_{\mathsf{BI}})$, a vector of L ciphertexts $CT_1, \ldots, CT_L$, where $CT_i = (0, ct_{i,0}, ct_{i,1}, \sigma_i)$, and a ciphertext CT, the verification algorithm proceeds as follows:
  1. For $i \in [1, L]$:
     (a) If $\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_{i,0}, ct_{i,1}), \sigma_i) = 0$:
        i. **Output** 0.
  2. If $CT = (0, ct_0, ct_1, \sigma)$:
     (a) If $L = 1 \wedge CT_1 = CT$:
        i. **Output** 1.
     (b) Otherwise: **Output** 0.
  3. If $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$:
     (a) $(com, st) = \mathsf{PC}.\mathtt{Commit}(crs^{(3)}_{\mathsf{PC}}, L, (ct_{i,0}, ct_{i,1}, 0)_{i \in [1,L]})$.
     (b) If $L = l \wedge com = com^{(3)}$:
        i. **Output** 1.
     (c) Otherwise: **Output** 0.

Theorem 4.1. *If* HE *is a secure FHE scheme with IND-CPA security and strong evaluation correctness,* ZK *is a simulation sound NIZK proof,* PC *is a secure predicate extractable commitment, and* BI *is a secure somewhere extractable index BARG with dynamic batch size, then* FHE *is a secure fully homomorphic encryption scheme with IV-CCA security.*

We present proof of Theorem 4.1 in Appendix B. At high-level, the security proof follows the security proofs of standard double encryption schemes [NY90, Sah99]. Our main task is to argue that an evaluated ciphertext

$$CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$$

generated by the adversary must satisfy $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0) = \mathsf{HE}.\mathtt{Dec}(sk_1, ct_1)$. This is guaranteed by the special succinct argument for the validity of ciphertexts, which is designed implicitly in above construction from the simulation sound NIZK proof, the predicate extractable commitment, and the somewhere extractable index BARG. We refer the readers to Sec. 1.2 for an overview of our main ideas for designing this special succinct argument and proving its security.

Note that all building blocks of the construction can be instantiated from (circular-secure) LWE, thus we have

Corollary 4.1. *Assuming the circular-secure LWE assumption, there exist FHE schemes with IV-CCA security.*

Acknowledgement. We appreciate the anonymous reviewers for their valuable comments. Rupeng Yang is partially supported by the Australian Research Council (ARC) Discovery Early Career Research (DECRA) DE250101041. Zuoxia Yu is partially supported by ARC DECRA DE240100282. Willy Susilo is supported by ARC Laureate Fellowship FL230100033.

References

[AGHV22] Adi Akavia, Craig Gentry, Shai Halevi, and Margarita Vald. Achievable CCA2 relaxation for homomorphic encryption. In *TCC*, pages 70–99, 2022.

[ASP14] Jacob Alperin-Sheriff and Chris Peikert. Faster bootstrapping with polynomial error. In *CRYPTO*, pages 297–314. Springer, 2014.

[BBK+23] Zvika Brakerski, Maya Farber Brodsky, Yael Tauman Kalai, Alex Lombardi, and Omer Paneth. SNARGs for monotone policy batch NP. In *CRYPTO*, pages 252–283. Springer, 2023.

[BBPS26] Zvika Brakerski, Maya Farber Brodsky, Omer Paneth, and Tomer Solomon. Mergeable SNARGs for trapdoor languages and their applications. Cryptology ePrint Archive, Report 2026/678, 2026. https://eprint.iacr.org/2026/678.

[BCF+25] Chris Brzuska, Sbastien Canard, Caroline Fontaine, Duong Hieu Phan, David Pointcheval, Marc Renard, and Renaud Sirdey. Relations among new CCA security notions for approximate FHE. *IACR CIC*, 2(1), 2025.

[BCHK07] Dan Boneh, Ran Canetti, Shai Halevi, and Jonathan Katz. Chosen-ciphertext security from identity-based encryption. *SIAM Journal on Computing*, 36(5):1301–1328, 2007.

[BEF+17] Jean-François Biasse, Thomas Espitau, Pierre-Alain Fouque, Alexandre Gélin, and Paul Kirchner. Computing generator in cyclotomic integer rings: A subfield algorithm for the principal ideal problem in $L_{|\Delta_{\mathbb{K}}|}(\frac{1}{2})$ and application to the cryptanalysis of a fhe scheme. In *EUROCRYPT*, pages 60–88. Springer, 2017.

[BFM88] Manuel Blum, Paul Feldman, and Silvio Micali. Non-interactive zero-knowledge and its applications. In *STOC*, pages 103–112. ACM, 1988.

[BGV12] Zvika Brakerski, Craig Gentry, and Vinod Vaikuntanathan. (leveled) fully homomorphic encryption without bootstrapping. In *ITCS*, pages 309–325, 2012.

[BLW17] Dan Boneh, Kevin Lewi, and David J Wu. Constraining pseudorandom functions privately. In *PKC*, pages 494–524. Springer, 2017.

[BP04] Mihir Bellare and Adriana Palacio. Towards plaintext-aware public-key encryption without random oracles. In *ASIACRYPT*, pages 48–62. Springer, 2004.

[Bra12] Zvika Brakerski. Fully homomorphic encryption without modulus switching from classical GapSVP. In *CRYPTO*, pages 868–886. Springer, 2012.

[BSW12] Dan Boneh, Gil Segev, and Brent Waters. Targeted malleability: homomorphic encryption for restricted computations. In *ITCS*, pages 350–366, 2012.

[BV11a] Zvika Brakerski and Vinod Vaikuntanathan. Efficient fully homomorphic encryption from (standard) LWE. In *FOCS*, pages 97–106, 2011.

[BV11b] Zvika Brakerski and Vinod Vaikuntanathan. Fully homomorphic encryption from ring-LWE and security for key dependent messages. In *CRYPTO*, pages 505–524. Springer, 2011.

[BV14] Zvika Brakerski and Vinod Vaikuntanathan. Lattice-based FHE as secure as PKE. In *ITCS*, pages 1–12, 2014.

[CCP+24] Jung Hee Cheon, Hyeongmin Choe, Alain Passelègue, Damien Stehlé, and Elias Suvanto. Attacks against the IND-CPA^D security of exact FHE schemes. In *CCS*, pages 2505–2519, 2024.

[CDPR16] Ronald Cramer, Léo Ducas, Chris Peikert, and Oded Regev. Recovering short generators of principal ideals in cyclotomic rings. In *EUROCRYPT*, pages 559–585. Springer, 2016.

[CF13] Dario Catalano and Dario Fiore. Vector commitments and their applications. In *PKC*, pages 55–72. Springer, 2013.

[CFM08] Dario Catalano, Dario Fiore, and Mariagrazia Messina. Zero-knowledge sets with short proofs. In *EUROCRYPT*, pages 433–450. Springer, 2008.

[CGGI20] Ilaria Chillotti, Nicolas Gama, Mariya Georgieva, and Malika Izabachène. TFHE: fast fully homomorphic encryption over the torus. *Journal of Cryptology*, 33(1):34–91, 2020.

[CGJ+23] Arka Rai Choudhuri, Sanjam Garg, Abhishek Jain, Zhengzhong Jin, and Jiaheng Zhang. Correlation intractability and SNARGs from sub-exponential DDH. In *CRYPTO*, pages 635–668. Springer, 2023.

[CHK04] Ran Canetti, Shai Halevi, and Jonathan Katz. Chosen-ciphertext security from identity-based encryption. In *EUROCRYPT*, pages 207–222. Springer, 2004.

[CHL+25] Suvradip Chakraborty, Dennis Hofheinz, Roman Langrehr, Jesper Buus Nielsen, Christoph Striecks, and Daniele Venturi. Malleable SNARKs and their applications. In *EUROCRYPT*, pages 184–213. Springer, 2025.

[CJJ21] Arka Rai Choudhuri, Abhihsek Jain, and Zhengzhong Jin. SNARGs for $\mathcal{P}$ from LWE. In *FOCS*, pages 68–79. IEEE, 2021.

[CKKS17] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. Homomorphic encryption for arithmetic of approximate numbers. In *ASIACRYPT*, pages 409–437. Springer, 2017.

[CM15] Michael Clear and Ciaran McGoldrick. Multi-identity and multi-key leveled FHE from learning with errors. In *CRYPTO*, pages 630–656. Springer, 2015.

[CRRV17] Ran Canetti, Srinivasan Raghuraman, Silas Richelson, and Vinod Vaikuntanathan. Chosen-ciphertext secure fully homomorphic encryption. In *PKC*, pages 213–240. Springer, 2017.

[CSBB24] Marina Checri, Renaud Sirdey, Aymen Boudguiga, and Jean-Paul Bultel. On the practical CPA^D security of exact and threshold FHE schemes and libraries. In *CRYPTO*, pages 3–33. Springer, 2024.

[DDA13] Angsuman Das, Sabyasachi Dutta, and Avishek Adhikari. Indistinguishability against chosen ciphertext verification attack revisited: The complete picture. In *ProvSec*, pages 104–120. Springer, 2013.

[DDN91] Danny Dolev, Cynthia Dwork, and Moni Naor. Non-malleable cryptography. In *STOC*, pages 542–552, 1991.

[DGKV22] Lalita Devadas, Rishab Goyal, Yael Kalai, and Vinod Vaikuntanathan. Rate-1 non-interactive arguments for batch-NP and applications. In *FOCS*, pages 1057–1068. IEEE, 2022.

[DHW23] Yevgeniy Dodis, Shai Halevi, and Daniel Wichs. Security with functional re-encryption from CPA. In *TCC*, pages 279–305. Springer, 2023.

[DM15] Léo Ducas and Daniele Micciancio. FHEW: bootstrapping homomorphic encryption in less than a second. In *EUROCRYPT*, pages 617–640. Springer, 2015.

[FHR22] Prastudy Fauzi, Martha Norberg Hovd, and Håvard Raddum. On the IND-CCA1 security of FHE schemes. *Cryptography*, 6(1):13, 2022.

[FV12] Junfeng Fan and Frederik Vercauteren. Somewhat practical fully homomorphic encryption. Cryptology ePrint Archive, Report 2012/144, 2012.

[Gen09a] Craig Gentry. *A FULLY HOMOMORPHIC ENCRYPTION SCHEME*. PhD thesis, STANFORD UNIVERSITY, 2009.

[Gen09b] Craig Gentry. Fully homomorphic encryption using ideal lattices. In *STOC*, pages 169–178, 2009.

[Goy24] Rishab Goyal. Mutable batch arguments and applications. Cryptology ePrint Archive, Report 2024/737, 2024.

[Gro10] Jens Groth. Short non-interactive zero-knowledge proofs. In *ASIACRYPT*, pages 341–358. Springer, 2010.

[GSW13] Craig Gentry, Amit Sahai, and Brent Waters. Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based. In *CRYPTO*, pages 75–92. Springer, 2013.

[GW11] Craig Gentry and Daniel Wichs. Separating succinct non-interactive arguments from all falsifiable assumptions. In *STOC*, pages 99–108, 2011.

[HW15] Pavel Hubacek and Daniel Wichs. On the communication complexity of secure function evaluation with long output. In *ITCS*, pages 163–172, 2015.

[Kil92] Joe Kilian. A note on efficient zero-knowledge proofs and arguments. In *STOC*, pages 723–732, 1992.

[KLVW23] Yael Kalai, Alex Lombardi, Vinod Vaikuntanathan, and Daniel Wichs. Boosting batch arguments and RAM delegation. In *STOC*, pages 1545–1552, 2023.

[KVZ21] Yael Tauman Kalai, Vinod Vaikuntanathan, and Rachel Yun Zhang. Somewhere statistical soundness, post-quantum security, and SNARGs. In *TCC*, pages 330–368. Springer, 2021.

[LATV12] Adriana López-Alt, Eran Tromer, and Vinod Vaikuntanathan. On-the-fly multiparty computation on the cloud via multikey fully homomorphic encryption. In *STOC*, pages 1219–1234, 2012.

[LM21] Baiyu Li and Daniele Micciancio. On the security of homomorphic encryption on approximate numbers. In *EUROCRYPT*, pages 648–677. Springer, 2021.

[LMSV11] Jake Loftus, Alexander May, Nigel P Smart, and Frederik Vercauteren. On CCA-secure somewhat homomorphic encryption. In *SAC*, pages 55–72. Springer, 2011.

[LY10] Benoît Libert and Moti Yung. Concise mercurial vector commitments and independent zero-knowledge sets with short proofs. In *TCC*, pages 499–517. Springer, 2010.

[Mer87] Ralph C Merkle. A digital signature based on a conventional encryption function. In *CRYPTO*, pages 369–378. Springer, 1987.

[Mic94] Silvio Micali. Cs proofs. In *FOCS*, pages 436–453. IEEE, 1994.

[MN24] Mark Manulis and Jérôme Nguyen. Fully homomorphic encryption beyond IND-CCA1 security: Integrity through verifiability. In *EUROCRYPT*, pages 63–93. Springer, 2024.

[NWW24] Shafik Nassar, Brent Waters, and David J Wu. Monotone policy BARGs from BARGs and additively homomorphic encryption. In *TCC*, pages 399–430. Springer, 2024.

[NY90] Moni Naor and Moti Yung. Public-key cryptosystems provably secure against chosen ciphertext attacks. In *STOC*, pages 427–437, 1990.

[PP22] Omer Paneth and Rafael Pass. Incrementally verifiable computation via rate-1 batch arguments. In *FOCS*, pages 1045–1056. IEEE, 2022.

[PP26] Omer Paneth and Rafael Pass. Verifiable divide-and-conquer. Cryptology ePrint Archive, Report 2026/662, 2026. https://eprint.iacr.org/2026/662.

[PR08] Manoj Prabhakaran and Mike Rosulek. Homomorphic encryption with CCA security. In *ICALP*, pages 667–678. Springer, 2008.

[PS19] Chris Peikert and Sina Shiehian. Noninteractive zero knowledge for NP from (plain) learning with errors. In *CRYPTO*, pages 89–114. Springer, 2019.

[RAD78] Ronald L Rivest, Len Adleman, and Michael L Dertouzos. On data banks and privacy homomorphisms. *Foundations of secure computation*, 4(11):169–180, 1978.

[Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In *STOC*, pages 84–93. ACM, 2005.

[RS91] Charles Rackoff and Daniel R Simon. Non-interactive zero-knowledge proof of knowledge and chosen ciphertext attack. In *CRYPTO*, pages 433–444. Springer, 1991.

[Sah99] Amit Sahai. Non-malleable non-interactive zero knowledge and adaptive chosen-ciphertext security. In *FOCS*, pages 543–553. IEEE, 1999.

[Sho98] Victor Shoup. Why chosen ciphertext security matters. IBM TJ Watson Research Center, 1998.

[STTY24] Takumi Shinozaki, Keisuke Tanaka, Masayuki Tezuka, and Yusuke Yoshida. On the relationship between FuncCPA and FuncCPA+. Cryptology ePrint Archive, Report 2024/1166, 2024.

[SV10] Nigel P Smart and Frederik Vercauteren. Fully homomorphic encryption with relatively small key and ciphertext sizes. In *PKC*, pages 420–443. Springer, 2010.

[Wat24] Brent Waters. A new approach for non-interactive zero-knowledge from learning with errors. In *STOC*, pages 399–410, 2024.

[WWX18] Biao Wang, Xueqing Wang, and Rui Xue. CCA1 secure FHE from PIO, revisited. *Cybersecurity*, 1:1–8, 2018.

[YKT18] Satoshi Yasuda, Fuyuki Kitagawa, and Keisuke Tanaka. Constructions for the IND-CCA1 secure fully homomorphic encryption. *Mathematical Modelling for Next-Generation Cryptography: CREST Crypto-Math Project*, pages 331–347, 2018.

A Security Analysis of Predicate Extractable Commitment

We present proof of Theorem 3.1 in this section. More precisely, we will prove the correctness, succinctness, binding, predicate hiding, and extractability of PC.

Correctness. For any predicate circuit $f \in \mathcal{F}$, any polynomial N, any $\boldsymbol{x} = (x_1, \ldots, x_N) \in (\{0,1\}^m)^N$, and any $j \in [1, N]$, let $n = \lceil \log N \rceil$.

Let $crs = ((pk_i)_{i\in[0,\lambda]}, (\boldsymbol{c}_i)_{i\in[1,\lambda]}, \boldsymbol{c}_f, hk)$, where

$$\forall i \in [0,\lambda], (pk_i, sk_i) \leftarrow \mathsf{HE}.\mathtt{KeyGen}(1^\lambda, 1^d)$$

$$\forall i \in [1,\lambda], \boldsymbol{c}_i \leftarrow \mathsf{HE}.\mathtt{EncV}(pk_i, sk_{i-1})$$

$$\boldsymbol{c}_f \leftarrow \mathsf{HE}.\mathtt{EncV}(pk_0, f), \quad hk \leftarrow \mathsf{H}.\mathtt{KeyGen}(1^\lambda)$$

Let $com = \boldsymbol{ct}_n$ and $\rho = (\boldsymbol{ct}_{i,u^{(i)}}, \boldsymbol{ct}_{i,\bar{u}^{(i)}})_{i\in[0,n-1]}$, where $u = j-1$, $(u_1, \ldots, u_n)$ is the binary representation of u, and

$$\forall i \in [0,n-1], u^{(i)} = (u_1, u_2, \ldots, u_{n-i-1}, u_{n-i}),\ \bar{u}^{(i)} = (u_1, u_2, \ldots, u_{n-i-1}, 1-u_{n-i})$$

$$\forall v \in [0, N-1], \boldsymbol{ct}_{0,v} = \mathsf{HE}.\mathtt{EvalC}(pk_0, \mathtt{C}_0[hk, x_{v+1}, v], \boldsymbol{c}_f) \tag{1}$$
$$\forall v \in [N, 2^n-1], \boldsymbol{ct}_{0,v} = \mathsf{HE}.\mathtt{EncV}(pk_0, 0^{1+\lambda+k}; \mathbf{0})$$

$$\forall i \in [1, n-1], v \in \{0,1\}^{n-i}:$$
$$\boldsymbol{ct}_{i,v} = \mathsf{HE}.\mathtt{EvalC}(pk_i, \mathtt{C}_1[hk, \boldsymbol{ct}_{i-1,v\|0}, \boldsymbol{ct}_{i-1,v\|1}, v], \boldsymbol{c}_i) \tag{2}$$

$$\boldsymbol{ct}_n = \mathsf{HE}.\mathtt{EvalC}(pk_n, \mathtt{C}_1[hk, \boldsymbol{ct}_{n-1,0}, \boldsymbol{ct}_{n-1,1}, \epsilon], \boldsymbol{c}_n) \tag{3}$$

Now, if we run the algorithm $\mathtt{Verify}(crs, N, com, j, x_j, \rho)$, the algorithm will output 0 only if

$$\boldsymbol{ct}_{0,u} \neq \mathsf{HE}.\mathtt{EvalC}(pk_0, \mathtt{C}_0[hk, x_j, u], \boldsymbol{c}_f)$$

which contradicts Equation (1), or there exists $i \in [1, n-1]$ s.t.

$$\boldsymbol{ct}_{i,u^{(i)}} \neq \mathsf{HE}.\mathtt{EvalC}(pk_i, \mathtt{C}_1[hk, \boldsymbol{ct}_{i-1,u^{(i)}\|0}, \boldsymbol{ct}_{i-1,u^{(i)}\|1}, u^{(i)}], \boldsymbol{c}_i)$$

which contradicts Equation (2), or

$$\boldsymbol{ct}_n \neq \mathsf{HE}.\mathtt{EvalC}(pk_n, \mathtt{C}_1[hk, \boldsymbol{ct}_{n-1,0}, \boldsymbol{ct}_{n-1,1}, \epsilon], \boldsymbol{c}_n)$$

which contradicts Equation (3). Therefore, the verification algorithm will always output 1.

Succinctness. For any predicate circuit $f \in \mathcal{F}$, any polynomial N, any $\boldsymbol{x} = (x_1, \ldots, x_N) \in (\{0,1\}^m)^N$, and any $j \in [1, N]$, let $n = \lceil \log N \rceil$. Let l_{pk}, l_{sk}, l_{hk} be the lengths of the public key of HE, the secret key of HE, and the hash key of H respectively. Note that

$$l_{pk} = poly(\lambda, d) = poly(\lambda, m, F), \quad l_{sk} = poly(\lambda, d) = poly(\lambda, m, F)$$

$$c = poly(\lambda, d) = poly(\lambda, m, F), \quad l_{hk} = poly(\lambda), \quad k = poly(\lambda)$$

Also, let

$$(crs, td) \leftarrow \mathtt{Setup}(1^\lambda, f),\ (com, st) = \mathtt{Commit}(crs, N, \boldsymbol{x}),\ \rho = \mathtt{Open}(crs, N, st, j)$$

Then we can write

$$crs = ((pk_i)_{i\in[0,\lambda]}, (\boldsymbol{c}_i)_{i\in[1,\lambda]}, \boldsymbol{c}_f, hk), \quad com = \boldsymbol{ct}_n, \quad \rho = (\boldsymbol{ct}_i, \overline{\boldsymbol{ct}}_i)_{i\in[0,n-1]}$$

That is

$$|crs| = (\lambda+1)\cdot l_{pk} + \lambda \cdot l_{sk} \cdot c + poly(F)\cdot c + l_{hk} = poly(\lambda, m, F)$$

$$|com| = (1 + \lambda + k) \cdot c = poly(\lambda, m, F)$$
$$|\rho| = 2n \cdot (1 + \lambda + k) \cdot c = poly(\lambda, m, F, \log N)$$

In addition, the verification algorithm will mainly perform one homomorphic evaluation of $\mathtt{C}_0$ and $n = \lceil \log N \rceil$ homomorphic evaluations of $\mathtt{C}_1$ over the FHE ciphertexts. The running time of each homomorphic evaluation will be bounded by $poly(\lambda, m, F)$ and therefore, the overall runtime of the verification algorithm can be bounded by $poly(\lambda, m, F, \log N)$.

Binding. For any predicate circuit $f \in \mathcal{F}$ and for all PPT adversaries $\mathcal{A}$, let $(crs, td) \leftarrow \mathtt{Setup}(1^\lambda, f)$ and let $(N, com, j, x, x', \rho, \rho')$ be the output of the adversary. Our goal is to prove that

$$\Pr[x \neq x' \wedge \mathtt{Verify}(crs, N, com, j, x, \rho) = 1 \wedge \mathtt{Verify}(crs, N, com, j, x', \rho') = 1] \leq negl(\lambda) \quad (4)$$

Here, we only need to consider the case that N is a polynomial in λ and $j \in [1, N]$, since otherwise the verification algorithm will output 0 directly.

First, let $n = \lceil \log N \rceil$, $u = j - 1$, $(u_1, \ldots, u_n)$ be the binary representation of u, $u^{(i)} = (u_1, u_2, \ldots, u_{n-i-1}, u_{n-i})$ for $i \in [0, n-1]$, and $u^{(n)} = \epsilon$. Then parse

$$crs = ((pk_i)_{i\in[0,\lambda]}, (\boldsymbol{c}_i)_{i\in[1,\lambda]}, \boldsymbol{c}_f, hk), \quad td = (sk_i)_{i\in[0,\lambda]}$$

$$com = \boldsymbol{ct}_n, \quad \rho = (\boldsymbol{ct}_i, \overline{\boldsymbol{ct}}_i)_{i\in[0,n-1]}, \quad \rho' = (\boldsymbol{ct}'_i, \overline{\boldsymbol{ct}}'_i)_{i\in[0,n-1]}$$

In addition, for $i \in [0, n-1]$, compute

$$(b_i, ind_i, t_i) = \mathsf{HE}.\mathtt{DecV}(sk_i, \boldsymbol{ct}_i), \quad (\bar{b}_i, \overline{ind}_i, \bar{t}_i) = \mathsf{HE}.\mathtt{DecV}(sk_i, \overline{\boldsymbol{ct}}_i)$$

$$(b'_i, ind'_i, t'_i) = \mathsf{HE}.\mathtt{DecV}(sk_i, \boldsymbol{ct}'_i), \quad (\bar{b}'_i, \overline{ind}'_i, \bar{t}'_i) = \mathsf{HE}.\mathtt{DecVs}(sk_i, \overline{\boldsymbol{ct}}'_i)$$

and compute

$$(b_n, ind_n, t_n) = \mathsf{HE}.\mathtt{DecV}(sk_n, \boldsymbol{ct}_n)$$

Now, assume that

$$\mathtt{Verify}(crs, N, com, j, x, \rho) = 1$$

then we have

$$(b_0, ind_0, t_0) = \mathtt{C}_0[hk, x, u](f)$$

due to the evaluation correctness of HE, i.e.,

$$t_0 = \mathsf{H}.\mathtt{Eval}(hk, x)$$

Moreover, for $i \in [1, n]$, we have

$$(b_i, ind_i, t_i) = \mathtt{C}_1[hk, \boldsymbol{ct}_{i-1,0}, \boldsymbol{ct}_{i-1,1}, u^{(i)}](sk_{i-1})$$

due to the evaluation correctness of HE, where

$$\boldsymbol{ct}_{i-1,u_{n-i+1}} = \boldsymbol{ct}_{i-1}, \quad \boldsymbol{ct}_{i-1,1-u_{n-i+1}} = \overline{\boldsymbol{ct}}_{i-1}$$

That is, for $i \in [1, n]$

$$t_i = \mathsf{H}.\mathtt{Eval}(hk, t_{i-1,0} \| t_{i-1,1})$$

where

$$t_{i-1,u_{n-i+1}} = t_{i-1}, \quad t_{i-1,1-u_{n-i+1}} = \bar{t}_{i-1}$$

To summarize, if $\mathtt{Verify}(crs, N, com, j, x, \rho) = 1$, then we have

$$\begin{aligned} & t_0 = \mathsf{H}.\mathtt{Eval}(hk, x) \wedge \\ & \forall i \in [1, n], t_i = \begin{cases} \mathsf{H}.\mathtt{Eval}(hk, t_{i-1} \| \bar{t}_{i-1}) & \text{If } u_{n-i+1} = 0 \\ \mathsf{H}.\mathtt{Eval}(hk, \bar{t}_{i-1} \| t_{i-1}) & \text{If } u_{n-i+1} = 1 \end{cases} \end{aligned} \tag{5}$$

Now, define $t'_n = t_n$. Similarly, we can prove that if $\mathtt{Verify}(crs, N, com, j, x', \rho') = 1$, then we have

$$\begin{aligned} & t'_0 = \mathsf{H}.\mathtt{Eval}(hk, x') \wedge \\ & \forall i \in [1, n], t'_i = \begin{cases} \mathsf{H}.\mathtt{Eval}(hk, t'_{i-1} \| \bar{t}'_{i-1}) & \text{If } u_{n-i+1} = 0 \\ \mathsf{H}.\mathtt{Eval}(hk, \bar{t}'_{i-1} \| t'_{i-1}) & \text{If } u_{n-i+1} = 1 \end{cases} \end{aligned} \tag{6}$$

Next, assume that both verifications can pass. Then we have both Equation (5) and Equation (6) hold. Moreover, since $t'_n = t_n$, we have $t'_{n-1} \neq t_{n-1}$ with only negligible probability, due to the collision resistance of H; then continuing the above argument we have

$$\Pr[t_0 \neq t'_0] \leq negl(\lambda)$$

Again, by the collision resistance of H, we have

$$\Pr[x \neq x'] \leq negl(\lambda)$$

if both verifications pass. This implies Equation (4) and that completes the proof.

Predicate Hiding. Next, we prove the predicate hiding property. We first define the following games between a challenger and a PPT adversary $\mathcal{A}$.

- **Game 0.** This is the real experiment. Let f_0, f_1 be the predicate circuits submitted by the adversary. The challenger samples $b \xleftarrow{\$} \{0, 1\}$ and computes

$$\forall i \in [0, \lambda], (pk_i, sk_i) \leftarrow \mathsf{HE}.\mathtt{KeyGen}(1^\lambda, 1^d)$$

$$\forall i \in [1, \lambda], \boldsymbol{c}_i \leftarrow \mathsf{HE}.\mathtt{EncV}(pk_i, sk_{i-1})$$

$$\boldsymbol{c}_f \leftarrow \mathsf{HE}.\mathtt{EncV}(pk_0, f_b), \quad hk \leftarrow \mathsf{H}.\mathtt{KeyGen}(1^\lambda)$$

  Then it returns $crs = ((pk_i)_{i \in [0,\lambda]}, (\boldsymbol{c}_i)_{i \in [1,\lambda]}, \boldsymbol{c}_f, hk)$ to the adversary. Finally, after the adversary submits a guess b', the challenger outputs 1 iff $b = b'$.

- **Game h.** For $h \in [1, \lambda]$, Game h is identical to Game $h-1$ except that, the challenger computes
$$\boldsymbol{c}_\ell \leftarrow \mathsf{HE}.\,\mathtt{EncV}(pk_\ell, 0^{l_{sk}})$$
where $\ell = \lambda + 1 - h$ and l_{sk} is the secret key length of HE.
- **Game $\lambda+1$.** In Game $\lambda+1$, the challenger computes
$$\boldsymbol{c}_f \leftarrow \mathsf{HE}.\,\mathtt{EncV}(pk_0, 0^{l_f})$$
where $l_f = |f_0| = |f_1|$.

Let $\mathcal{E}_h$ be the output of Game h for $h \in [0, \lambda+1]$. We next show that $\Pr[\mathcal{E}_0 = 1] \leq 1/2 + negl(\lambda)$ via proving the following lemmas.

Lemma A.1. *For $h \in [1, \lambda]$, $|\Pr[\mathcal{E}_{h-1} = 1] - \Pr[\mathcal{E}_h = 1]| \leq negl(\lambda)$.*

Proof. Let $\ell = \lambda + 1 - h$. In Game $h-1$, sk_ℓ does not appear in the adversary's view, thus, indistinguishability between $\mathsf{HE}.\,\mathtt{EncV}(pk_\ell, sk_{\ell-1})$ and $\mathsf{HE}.\,\mathtt{EncV}(pk_\ell, 0^{l_{sk}})$ comes from the IND-CPA security of HE directly. □

Lemma A.2. $|\Pr[\mathcal{E}_\lambda = 1] - \Pr[\mathcal{E}_{\lambda+1} = 1]| \leq negl(\lambda)$.

Proof. In Game λ, sk_0 does not appear in the adversary's view, thus, indistinguishability between $\mathsf{HE}.\,\mathtt{EncV}(pk_0, f_b)$ and $\mathsf{HE}.\,\mathtt{EncV}(pk_0, 0^{l_f})$ comes from the IND-CPA security of HE directly. □

Lemma A.3. $\Pr[\mathcal{E}_{\lambda+1} = 1] = 1/2$.

Proof. In Game $\lambda+1$, the bit b does not appear in the adversary's view, thus, the probability that the adversary can output b is exactly $1/2$. □

Combining Lemma A.1 to Lemma A.3, we have $\Pr[\mathcal{E}_0 = 1] \leq 1/2 + negl(\lambda)$, and that completes the proof.

Extractability. For any predicate circuit $f \in \mathcal{F}$ and for all PPT adversaries $\mathcal{A}$, let $(crs, td) \leftarrow \mathtt{Setup}(1^\lambda, f)$ and let (N, com, j, x, ρ) be the output of the adversary. Also, let $(a, j') = \mathtt{Extract}(td, N, com)$. Our goal is to prove that

$$\Pr[a = 0 \wedge f(x) = 1 \wedge \mathtt{Verify}(crs, N, com, j, x, \rho) = 1] \leq negl(\lambda) \quad (7)$$

$$\Pr[(a, j') = (1, j) \wedge f(x) = 0 \wedge \mathtt{Verify}(crs, N, com, j, x, \rho) = 1] \leq negl(\lambda) \quad (8)$$

Here, we only need to consider the case that N is a polynomial in λ and $j \in [1, N]$, since otherwise the verification algorithm will output 0 directly.

First, let $n = \lceil \log N \rceil$, $u = j - 1$, $(u_1, \ldots, u_n)$ be the binary representation of u, $u^{(i)} = (u_1, u_2, \ldots, u_{n-i-1}, u_{n-i})$ for $i \in [0, n-1]$, and $u^{(n)} = \epsilon$. Then parse

$$crs = ((pk_i)_{i\in[0,\lambda]}, (\boldsymbol{c}_i)_{i\in[1,\lambda]}, \boldsymbol{c}_f, hk), \quad td = (sk_i)_{i\in[0,\lambda]}$$

$$com = \boldsymbol{ct}_n, \quad \rho = (\boldsymbol{ct}_i, \overline{\boldsymbol{ct}}_i)_{i\in[0,n-1]}$$

In addition, for $i \in [0, n-1]$, compute

$$(b_i, ind_i, t_i) = \mathsf{HE}.\mathtt{DecV}(sk_i, \boldsymbol{ct}_i), \quad (\bar{b}_i, \overline{ind}_i, \bar{t}_i) = \mathsf{HE}.\mathtt{DecV}(sk_i, \overline{\boldsymbol{ct}}_i)$$

and compute

$$(b_n, ind_n, t_n) = \mathsf{HE}.\mathtt{DecV}(sk_n, \boldsymbol{ct}_n)$$

Next, we prove Equation (7) and (8).

Lemma A.4. $\Pr[a = 0 \wedge f(x) = 1 \wedge \mathtt{Verify}(crs, N, com, j, x, \rho) = 1] \leq negl(\lambda)$

Proof. We prove Lemma A.4 by showing that

$$\Pr[a = 0 \mid f(x) = 1 \wedge \mathtt{Verify}(crs, N, com, j, x, \rho) = 1] = 0$$

First, assume that $\mathtt{Verify}(crs, N, com, j, x, \rho) = 1$ and $f(x) = 1$, then we have the following claims

Claim A.1. $(b_0, ind_0) = (1, j)$.

Proof. First, as the verification passes, we have

$$(b_0, ind_0, t_0) = \mathsf{C}_0[hk, x, u](f)$$

due to the evaluation correctness of HE. Then, since $f(x) = 1$, the circuit will output $(1, j, *)$ and therefore $(b_0, ind_0) = (1, j)$. □

Claim A.2. *For $i \in [1, n]$, if*

$$b_{i-1} = 1 \quad \wedge \quad u^{(i-1)} \text{ is a prefix of } (ind_{i-1} - 1) \quad \wedge \quad 1 \leq ind_{i-1} \leq j$$

then we have

$$b_i = 1 \quad \wedge \quad u^{(i)} \text{ is a prefix of } (ind_i - 1) \quad \wedge \quad 1 \leq ind_i \leq j$$

Proof. First, as the verification passes, we have

$$(b_i, ind_i, t_i) = \mathsf{C}_1[hk, \boldsymbol{ct}_{i-1,0}, \boldsymbol{ct}_{i-1,1}, u^{(i)}](sk_{i-1})$$

due to the evaluation correctness of HE, where

$$\boldsymbol{ct}_{i-1,u_{n-i+1}} = \boldsymbol{ct}_{i-1}, \quad \boldsymbol{ct}_{i-1,1-u_{n-i+1}} = \overline{\boldsymbol{ct}}_{i-1}$$

We next consider the following two cases:

- **Case I:** $u_{n-i+1} = 0$**.** In this case, we have

  $$(b_i, ind_i, t_i) = \mathsf{C}_1[hk, \boldsymbol{ct}_{i-1}, \overline{\boldsymbol{ct}}_{i-1}, u^{(i)}](sk_{i-1})$$

  In addition, we have $u^{(i-1)} = u^{(i)} \| u_{n-i+1} = u^{(i)} \| 0$, thus $u^{(i)} \| 0$ is a prefix of $ind_{i-1} - 1$. Therefore, the circuit will output $(b_{i-1}, ind_{i-1}, *)$, i.e., $b_i = b_{i-1} = 1$, $1 \leq ind_i = ind_{i-1} \leq j$, and $u^{(i)}$, which is a prefix of $u^{(i-1)}$, is a prefix of $ind_i - 1$.

- **Case II:** $u_{n-i+1} = 1$**.** In this case, we have

$$(b_i, ind_i, t_i) = \mathtt{C}_1[hk, \overline{\boldsymbol{ct}}_{i-1}, \boldsymbol{ct}_{i-1}, u^{(i)}](sk_{i-1})$$

We further consider the following two cases:

- **Case 1:** $\bar{b}_{i-1} = 1$ **and** $u^{(i)}\|0$ **is a prefix of** $\overline{ind}_{i-1} - 1$**.** Then we have $(b_i, ind_i) = (\bar{b}_{i-1}, \overline{ind}_{i-1})$. This implies that $b_i = 1$ and $u^{(i)}$, which is a prefix of $u^{(i)}\|0$, is a prefix of $ind_i - 1$. Besides, since $u^{(i)}\|0$ is a prefix of $ind_i - 1$ and $u^{(i)}\|1$ is a prefix of $ind_{i-1} - 1$ (note that $u^{(i-1)} = u^{(i)}\|u_{n-i+1} = u^{(i)}\|1$), $ind_i \leq ind_{i-1} \leq j$. Finally, since $u^{(i)}\|0$ is a prefix of $ind_i - 1$, $ind_i - 1 \geq 0$ and $ind_i \geq 1$.
- **Case 2:** $\bar{b}_{i-1} = 0$ **or** $u^{(i)}\|0$ **is not a prefix of** $\overline{ind}_{i-1}$**.** That means, the first check in the circuit does not pass. Then similar to the proof in Case I, we have $(b_i, ind_i) = (b_{i-1}, ind_{i-1})$ and all conditions for (b_i, ind_i) are satisfied.

As in all cases, we always have

$$b_i = 1 \quad \wedge \quad u^{(i)} \text{ is a prefix of } (ind_i - 1) \quad \wedge \quad 1 \leq ind_i \leq j$$

Proof of Claim A.2 follows. □

Combining Claim A.1 and Claim A.2, we have $b_n = 1$ and $1 \leq ind_n \leq j \leq N$, i.e., $(a, j') = (b_n, ind_n)$ and therefore $a \neq 0$ if $f(x) = 1$ and the verification passes. That completes the proof. □

Lemma A.5. $\Pr[(a, j') = (1, j) \wedge f(x) = 0 \wedge \mathtt{Verify}(crs, N, com, j, x, \rho) = 1] \leq negl(\lambda)$

Proof. We prove Lemma A.5 by showing that

$$\Pr[(a, j') = (1, j) \mid f(x) = 0 \wedge \mathtt{Verify}(crs, N, com, j, x, \rho) = 1] = 0$$

First, assume that $\mathtt{Verify}(crs, N, com, j, x, \rho) = 1$ and $f(x) = 0$, then we have the following claims

Claim A.3. $(b_0, ind_0) = (0, 0)$.

Proof. First, as the verification passes, we have

$$(b_0, ind_0, t_0) = \mathtt{C}_0[hk, x, u](f)$$

due to the evaluation correctness of HE. Then, since $f(x) = 0$, the circuit will output $(0, 0, *)$ and therefore $(b_0, ind_0) = (0, 0)$. □

Claim A.4. *For* $i \in [1, n]$*, if* $ind_{i-1} \neq j$*, then* $ind_i \neq j$.

Proof. First, as the verification passes, we have

$$(b_i, ind_i, t_i) = \mathtt{C}_1[hk, \boldsymbol{ct}_{i-1,0}, \boldsymbol{ct}_{i-1,1}, u^{(i)}](sk_{i-1})$$

due to the evaluation correctness of HE, where

$$\boldsymbol{ct}_{i-1,u_{n-i+1}} = \boldsymbol{ct}_{i-1}, \quad \boldsymbol{ct}_{i-1,1-u_{n-i+1}} = \overline{\boldsymbol{ct}}_{i-1}$$

Therefore, we have $ind_i \in \{ind_{i-1}, \overline{ind}_{i-1}, 0\}$. Since $ind_{i-1} \neq j$, $ind_i = j$ only if $\overline{ind}_{i-1} = j$.

We next show that if $\overline{ind}_{i-1} = j$ then $ind_i \neq \overline{ind}_{i-1}$. First, if $\overline{ind}_{i-1} = j$, then we have $\overline{ind}_{i-1} - 1 = j - 1 = u$. Moreover, if the circuit outputs $\overline{ind}_{i-1}$, we must have $u^{(i)} \| (1 - u_{n-i+1})$ is a prefix of $\overline{ind}_{i-1} - 1$, i.e., $u^{(i)} \| (1 - u_{n-i+1})$ is a prefix of u, which is not possible. Therefore, if $\overline{ind}_{i-1} = j$, then the circuit will not output it.

To summarize, we have $ind_i \neq j$ if $ind_{i-1} \neq j$ and that completes the proof. □

Combining Claim A.3 and Claim A.4, we have $ind_n \neq j$. Since $j' \in \{0, ind_n\}$, we have $j' \neq j$ if $f(x) = 0$ and the verification passes. That completes the proof. □

B Security Analysis of Fully Homomorphic Encryption

We present proof of Theorem 4.1 in this section. More precisely, we will prove the standard correctness, evaluation correctness, compactness, and IV-CCA security of FHE.

Standard Correctness. The standard decryption correctness comes from the standard correctness of HE and the completeness of ZK directly; the standard verification correctness comes from the completeness of ZK directly.

Evaluation Correctness. For any polynomials l, N, any messages $\mu_1, \ldots, \mu_l \in \{0,1\}$, and any circuit $\mathtt{C} : \{0,1\}^l \to \{0,1\}$ with N wires, let

$$(PK, SK) \leftarrow \mathtt{KeyGen}(1^\lambda), \quad \forall i \in [1, l], CT_i \leftarrow \mathtt{Enc}(PK, \mu_i)$$

$$CT' \leftarrow \mathtt{Eval}(PK, \mathtt{C}, (CT_i)_{i\in[1,l]})$$

Next, we prove that $\mathtt{Dec}(SK, CT') = \mathtt{C}(\mu_1, \ldots, \mu_l)$ and $\mathtt{Verify}(PK, (CT_i)_{i\in[1,l]}, CT') = 1$.

First, by the completeness of ZK, the initial check in the evaluation algorithm can pass and $CT' \neq \perp$. Next, when decrypting

$$CT' = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$$

with

$$SK = (sk_0; pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, crs_{\mathsf{BI}})$$

the algorithm proceeds as follows

1. Checking if N is polynomial in λ or not. The check will pass since N is a polynomial here.
2. Checking if both l and d do not exceed N. We have $l \leq N$ since the circuit is correctly described; also, if we run the evaluation algorithm honestly, we have $d \leq N - l \leq N$.
3. Checking the validity of Π. By the completeness of BI, the proof can be accepted if the circuit $\mathtt{P} = \mathtt{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$ is satisfied for any index $i \in [1, N]$. We argue this for the following two cases:
   - **Case I:** $i \leq l$**.** In this case, the circuit can be satisfied due to the correctness of PC, the completeness of ZK, and the fact that we set $d_i = 0$ for $i \in [1, l]$.
   - **Case II:** $i > l$**.** In this case, the circuit can be satisfied due to the correctness of PC, and the fact that we compute $ct_{i,0} = \mathsf{HE}.\mathtt{Eval}(pk_0, ct_{L_i,0}, ct_{R_i,0})$, $ct_{i,1} = \mathsf{HE}.\mathtt{Eval}(pk_1, ct_{L_i,1}, ct_{R_i,1})$, and $d_i = \max(d_{L_i}, d_{R_i}) + 1$.
4. Checking if $\mathcal{C}$ is committed in both $com^{(1)}$ and $com^{(2)}$ at the N-th position. The check passes due to the correctness of PC.
5. Finally, if all checks pass, the algorithm decrypt ct_0 using sk_0. Note that ct_0 is generated by evaluating $\mathtt{C}$ honestly on $ct_{1,0}, \ldots, ct_{l,0}$ gate-by-gate. Thus, by the evaluation correctness of HE, the decryption will return the correct result $\mathtt{C}(\mu_1, \ldots, \mu_l)$.

Therefore, we have $\mathtt{Dec}(SK, CT') = \mathtt{C}(\mu_1, \ldots, \mu_l)$.
Next, we prove that $\mathtt{Verify}(PK, (CT_i)_{i\in[1,l]}, CT') = 1$. Let $CT_i = (0, ct_{i,0}, ct_{i,1}, \sigma_i)$ for $i \in [1, l]$, then the verification algorithm will check if each σ_i is valid and if $com = com^{(3)}$, where $(com, st) = \mathsf{PC}.\mathtt{Commit}(crs^{(3)}_{\mathsf{PC}}, l, (ct_{i,0}, ct_{i,1}, 0)_{i\in[1,l]})$. The first check passes due to the completeness of ZK; the second check passes since both com and $com^{(3)}$ will commit the same set and the commit algorithm of PC is deterministic.

Compactness. For any polynomials l, N, any messages $\mu_1, \ldots, \mu_l \in \{0, 1\}$, and any circuit $\mathtt{C} : \{0, 1\}^l \to \{0, 1\}$ with N wires, let

$$(PK, SK) \leftarrow \mathtt{KeyGen}(1^\lambda), \quad \forall i \in [1, l], CT_i \leftarrow \mathtt{Enc}(PK, \mu_i)$$

$$CT' \leftarrow \mathtt{Eval}(PK, \mathtt{C}, (CT_i)_{i\in[1,l]})$$

We can parse

$$CT' = (1, \mathcal{C}, com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$$

which contains two ciphertexts of HE, three commitments and two proofs of PC, one proof of BI, and some strings whose lengths can be bounded by $O(\lambda)$. We have $|CT|' \leq poly(\lambda, \log N, \log l, |\mathtt{C}_c|, |\mathtt{C}_e|)$ due to the compactness/succinctness of HE, PC, and BI. Then, since the sizes of $\mathtt{C}_c$ and $\mathtt{C}_e$ are bounded by polynomials of λ, we have

$$|CT|' \leq poly(\lambda, \log N, \log l)$$

In addition, to decrypt CT', the decryption algorithm mainly needs to verify a proof of BI, verify two proofs of PC, and decrypt one ciphertext of HE. The running time can be bounded by $poly(\lambda, \log N)$ due to the compactness/succinctness of HE, PC, and BI.

Security. Next, we prove the IV-CCA security of FHE. We first define the following games between a challenger and a PPT adversary $\mathcal{A}$. Here we use $\mathcal{Q} = \mathcal{Q}(\lambda)$ to denote the upper bound of the decryption queries made by $\mathcal{A}$. In addition, for each ciphertext $CT = (1, *, *, *, *, *, *, *, *, , N)$ queried by $\mathcal{A}$, the decryption oracle will output $\perp$ immediately if N is not a polynomial, and we use $\mathcal{M} = \mathcal{M}(\lambda)$ to denote the upper bound of N when N is a polynomial.

- **Game 0.** This is the real experiment defined in Definition 4.4. More precisely, the challenger proceeds as follows.
  1. First, the challenger generates:
     (a) $(pk_0, sk_0) \leftarrow \mathsf{HE}.\mathtt{KeyGen}(1^\lambda)$.
     (b) $(pk_1, sk_1) \leftarrow \mathsf{HE}.\mathtt{KeyGen}(1^\lambda)$.
     (c) $crs_{\mathsf{ZK}} \leftarrow \mathsf{ZK}.\mathtt{Setup}(1^\lambda)$.
     (d) $(crs^{(1)}_{\mathsf{PC}}, td^{(1)}_{\mathsf{PC}}) \leftarrow \mathsf{PC}.\mathtt{Setup}(1^\lambda, \mathtt{C}_d)$.
     (e) $(crs^{(2)}_{\mathsf{PC}}, td^{(2)}_{\mathsf{PC}}) \leftarrow \mathsf{PC}.\mathtt{Setup}(1^\lambda, \mathtt{C}_d)$.
     (f) $(crs^{(3)}_{\mathsf{PC}}, td^{(3)}_{\mathsf{PC}}) \leftarrow \mathsf{PC}.\mathtt{Setup}(1^\lambda, \mathtt{C}_d)$.
     (g) $(crs_{\mathsf{BI}}, td_{\mathsf{BI}}) \leftarrow \mathsf{BI}.\mathtt{Setup}(1^\lambda, 1^C, 1)$.

     and returns $PK = (pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, crs_{\mathsf{BI}})$ to the adversary.
  2. Then it answers $\mathcal{A}$'s decryption oracle queries. In particular, for each queried ciphertext CT, it proceeds as follows if $CT = (0, ct_0, ct_1, \sigma)$:
     (a) If $\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_0, ct_1), \sigma) = 0$: Return $\perp$.
     (b) Return $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0)$.

     On the other hand, it proceeds as follows if $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$:
     (a) If N is not polynomial in λ: Return $\perp$.
     (b) If $l > N \vee d > N$: Return $\perp$.
     (c) $\mathtt{P} = \mathtt{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$
     (d) If $\mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathtt{P}, \Pi) = 0$: Return $\perp$.
     (e) If $\mathsf{PC}.\mathtt{Verify}(crs^{(1)}_{\mathsf{PC}}, N, com^{(1)}, N, \mathcal{C}, \rho^{(1)}) = 0$: Return $\perp$.
     (f) If $\mathsf{PC}.\mathtt{Verify}(crs^{(2)}_{\mathsf{PC}}, N, com^{(2)}, N, \mathcal{C}, \rho^{(2)}) = 0$: Return $\perp$.
     (g) Return $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0)$.
  3. Next, after $\mathcal{A}$ submits two challenge messages μ^*_0 and μ^*_1, the challenger samples $b \xleftarrow{\$} \{0, 1\}$, generates:

(a) $r_0^*, r_1^* \xleftarrow{\$} \mathcal{R}_{\mathtt{Enc}}$.
(b) $ct_0^* = \mathsf{HE}.\mathtt{Enc}(pk_0, \mu_b^*; r_0^*)$.
(c) $ct_1^* = \mathsf{HE}.\mathtt{Enc}(pk_1, \mu_b^*; r_1^*)$.
(d) $\sigma^* \leftarrow \mathsf{ZK}.\mathtt{Prove}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_0^*, ct_1^*), (\mu_b^*, r_0^*, r_1^*))$.

and returns $CT^* = (0, ct_0^*, ct_1^*, \sigma^*)$ to the adversary.

4. Then the challenger continues to answer $\mathcal{A}$'s decryption oracle queries. In particular, for each query $((CT_1, \ldots, CT_L), CT)$, it proceeds as follows if $CT = (0, ct_0, ct_1, \sigma)$:
   (a) If $\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_0, ct_1), \sigma) = 0$: Return $\bot$.
   (b) If $L \neq 1 \vee CT_1 \neq CT \vee CT^* = CT$: Return $\top$.
   (c) Return $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0)$.

   On the other hand, it proceeds as follows if $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$:
   (a) If N is not polynomial in λ: Return $\bot$.
   (b) If $l > N \vee d > N$: Return $\bot$.
   (c) $\mathtt{P} = \mathtt{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs_{\mathsf{PC}}^{(1)}, crs_{\mathsf{PC}}^{(2)}, crs_{\mathsf{PC}}^{(3)}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$
   (d) If $\mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathtt{P}, \Pi) = 0$: Return $\bot$.
   (e) If $\mathsf{PC}.\mathtt{Verify}(crs_{\mathsf{PC}}^{(1)}, N, com^{(1)}, N, \mathcal{C}, \rho^{(1)}) = 0$: Return $\bot$.
   (f) If $\mathsf{PC}.\mathtt{Verify}(crs_{\mathsf{PC}}^{(2)}, N, com^{(2)}, N, \mathcal{C}, \rho^{(2)}) = 0$: Return $\bot$.
   (g) For $i \in [1, L]$:
      i. If $CT^* = CT_i$: Return $\top$.
      ii. Parse $CT_i = (0, ct_{i,0}, ct_{i,1}, \sigma_i)$.
      iii. If $\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_{i,0}, ct_{i,1}), \sigma_i) = 0$: Return $\top$.
   (h) $(com, st) = \mathsf{PC}.\mathtt{Commit}(crs_{\mathsf{PC}}^{(3)}, L, (ct_{i,0}, ct_{i,1}, 0)_{i \in [1,L]})$.
   (i) If $L \neq l \vee com \neq com^{(3)}$: Return $\top$.
   (j) Return $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0)$.
5. Finally, after $\mathcal{A}$ submits a bit b', the challenger outputs 1 if $b = b'$ and it outputs 0 otherwise.

- **Game 1.** This is identical to Game 0 except that σ^* is a simulated proof. More precisely, in the beginning, the challenger generates

$$(crs_{\mathsf{ZK}}, td_{\mathsf{ZK}}) \leftarrow \mathsf{ZK}.\mathsf{S}_1(1^\lambda)$$

  then in Phase 3, the challenger generates

$$\sigma^* \leftarrow \mathsf{ZK}.\mathsf{S}_2(crs_{\mathsf{ZK}}, td_{\mathsf{ZK}}, (pk_0, pk_1, ct_0^*, ct_1^*))$$

- **Game 2.** This is identical to Game 1 except that the challenger generates

$$ct_1^* \leftarrow \mathsf{HE}.\mathtt{Enc}(pk_1, 0)$$

- **Game 3.** This is identical to Game 2 except that the challenger changes the way to answer the decryption oracle given a fresh ciphertext. In more detail, at Phase 2, on input a ciphertext $CT = (0, ct_0, ct_1, \sigma)$, it proceeds as follows:
  1. If $\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_0, ct_1), \sigma) = 0$: Return $\perp$.
  2. If $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_1)$: Abort the game and output 2.
  3. Return $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0)$.

  Then at Phase 4, on input a query $((CT_1, \ldots, CT_L), CT)$, where $CT = (0, ct_0, ct_1, \sigma)$, it proceeds as follows:
  1. If $\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_0, ct_1), \sigma) = 0$: Return $\perp$.
  2. If $L \neq 1 \vee CT_1 \neq CT \vee CT^* = CT$: Return $\top$.
  3. If $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_1)$: Abort the game and output 2.
  4. Return $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0)$.
- **Game 4.** This is identical to Game 3 except that the challenger changes the way to answer the decryption oracle given an evaluated ciphertext. In more detail, at Phase 2, on input a ciphertext $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$, it proceeds as follows:
  1. If N is not polynomial in λ: Return $\perp$.
  2. If $l > N \vee d > N$: Return $\perp$.
  3. $\mathsf{P} = \mathsf{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$
  4. If $\mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathsf{P}, \Pi) = 0$: Return $\perp$.
  5. If $\mathsf{PC}.\mathtt{Verify}(crs^{(1)}_{\mathsf{PC}}, N, com^{(1)}, N, \mathcal{C}, \rho^{(1)}) = 0$: Return $\perp$.
  6. If $\mathsf{PC}.\mathtt{Verify}(crs^{(2)}_{\mathsf{PC}}, N, com^{(2)}, N, \mathcal{C}, \rho^{(2)}) = 0$: Return $\perp$.
  7. If $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_1)$: Abort the game and output 3.
  8. Return $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0)$.

  Then at Phase 4, on input a query $((CT_1, \ldots, CT_L), CT)$, where $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$, it proceeds as follows:
  1. If N is not polynomial in λ: Return $\perp$.
  2. If $l > N \vee d > N$: Return $\perp$.
  3. $\mathsf{P} = \mathsf{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$
  4. If $\mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathsf{P}, \Pi) = 0$: Return $\perp$.
  5. If $\mathsf{PC}.\mathtt{Verify}(crs^{(1)}_{\mathsf{PC}}, N, com^{(1)}, N, \mathcal{C}, \rho^{(1)}) = 0$: Return $\perp$.
  6. If $\mathsf{PC}.\mathtt{Verify}(crs^{(2)}_{\mathsf{PC}}, N, com^{(2)}, N, \mathcal{C}, \rho^{(2)}) = 0$: Return $\perp$.
  7. For $i \in [1, L]$:
     (a) If $CT^* = CT_i$: Return $\top$.
     (b) Parse $CT_i = (0, ct_{i,0}, ct_{i,1}, \sigma_i)$.
     (c) If $\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_{i,0}, ct_{i,1}), \sigma_i) = 0$: Return $\top$.
  8. $(com, st) = \mathsf{PC}.\mathtt{Commit}(crs^{(3)}_{\mathsf{PC}}, L, (ct_{i,0}, ct_{i,1}, 0)_{i \in [1,L]})$.

9. If $L \neq l \vee com \neq com^{(3)}$: Return $\top$.
10. If $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_1)$: Abort the game and output 3.
11. Return $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0)$.

- **Game 5.** This is identical to Game 4 except that for each decryption query CT (at Phase 2) or $((CT_1, \ldots, CT_L), CT)$ (at Phase 4), let $CT = (0, ct_0, ct_1, \sigma)$ or $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$, then the challenger returns

$$\mathsf{HE}.\mathtt{Dec}(sk_1, ct_1)$$

  instead of $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0)$ if all checks pass.
- **Game 6.** This is identical to Game 5 except that the challenger removes all checks added in Game 3 and Game 4.
- **Game 7.** This is identical to Game 6 except that the challenger generates

$$ct_0^* \leftarrow \mathsf{HE}.\mathtt{Enc}(pk_0, 0)$$

Let $\mathcal{E}_i$ be the output of Game i. We next show that $\Pr[\mathcal{E}_0 = 1] \leq 1/2 + negl(\lambda)$ via proving the following lemmas.

Lemma B.1. $\mid \Pr[\mathcal{E}_0 = 1] - \Pr[\mathcal{E}_1 = 1] \mid \leq negl(\lambda)$.

Proof. This comes from the zero-knowledge property of ZK directly. □

Lemma B.2. $\mid \Pr[\mathcal{E}_1 = 1] - \Pr[\mathcal{E}_2 = 1] \mid \leq negl(\lambda)$.

Proof. This comes from the IND-CPA security of HE directly. □

Lemma B.3. *For* $z \in \{1, 2, 3\}$, $\mid \Pr[\mathcal{E}_2 = z] - \Pr[\mathcal{E}_3 = z] \mid \leq negl(\lambda)$.

Proof. The two games are identical unless for a pre-challenge query $CT = (0, ct_0, ct_1, \sigma)$,

$$\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_1) \wedge \mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_0, ct_1), \sigma) = 1 \quad (9)$$

or for a post-challenge query $((CT_1, \ldots, CT_L), CT)$ s.t. $CT = (0, ct_0, ct_1, \sigma)$,

$$\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_1) \wedge \mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_0, ct_1), \sigma) = 1 \wedge CT \neq CT^* \quad (10)$$

If $(pk_0, pk_1, ct_0, ct_1) \in \mathcal{L}$, then we always have $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0) = \mathsf{HE}.\mathtt{Dec}(sk_1, ct_1)$ due to the standard correctness of HE. That means, if $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_1)$, then $(pk_0, pk_1, ct_0, ct_1) \notin \mathcal{L}$, i.e., if Equation (9) or Equation (10) occurs, then we can find an invalid statement with a valid proof, which are not from the oracle $\mathtt{S}_2$. Therefore, both Equation (9) and Equation (10) will occur with only negligible probability due to the simulation soundness of ZK. □

Lemma B.4. *For $z \in \{1,2,3\}$, $|\Pr[\mathcal{E}_3 = z] - \Pr[\mathcal{E}_4 = z]| \leq negl(\lambda)$.*

Proof. We define Game 3.i for $i \in [0, Q]$, which answers the first i decryption oracle queries as in Game 4 and answers the remaining decryption oracle queries as in Game 3. It is easy to see Game 3 is identical to Game 3.0 and Game 4 is identical to Game 3.Q. Let $\mathcal{E}_{3,i}$ be the output of Game 3.i for $i \in [0, Q]$, it is sufficient to prove the following Claim:

Claim B.1. *For $i \in [1, Q]$, $z \in \{1,2,3\}$, $|\Pr[\mathcal{E}_{3,i-1} = z] - \Pr[\mathcal{E}_{3,i} = z]| \leq negl(\lambda)$.*

We give proof of Claim B.1 in Appendix B.1. □

Lemma B.5. *For $z \in \{1,2,3\}$, $\Pr[\mathcal{E}_4 = z] = \Pr[\mathcal{E}_5 = z]$.*

Proof. In Game 4 and Game 5, for every decryption query, we always have $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0) = \mathsf{HE}.\mathtt{Dec}(sk_1, ct_1)$ if all the checks pass. Therefore, the changes made in Game 5 will not change the decryption oracle responses and the two games are identical. □

Lemma B.6. *For $z \in \{1,2,3\}$, $|\Pr[\mathcal{E}_5 = z] - \Pr[\mathcal{E}_6 = z]| \leq negl(\lambda)$.*

Proof. In Game 6, we remove the aborting conditions introduced in Game 3 and Game 4. Note that once the aborting conditions are triggered, the games will output 2 or 3. Therefore, it is sufficient to show that $\Pr[\mathcal{E}_5 \in \{2,3\}] \leq negl(\lambda)$. This follows from Lemma B.3 to Lemma B.5, which shows that $|\Pr[\mathcal{E}_2 \in \{2,3\}] - \Pr[\mathcal{E}_5 \in \{2,3\}]| \leq negl(\lambda)$, and the fact that $\Pr[\mathcal{E}_2 \in \{2,3\}] = 0$. □

Lemma B.7. *$|\Pr[\mathcal{E}_6 = 1] - \Pr[\mathcal{E}_7 = 1]| \leq negl(\lambda)$.*

Proof. This comes from the IND-CPA security of HE directly. □

Lemma B.8. *$\Pr[\mathcal{E}_7 = 1] = 1/2$.*

Proof. In Game 7, b never appears in the adversary's view, and therefore, the probability that $\mathcal{A}$ can output the correct value of b is exactly $1/2$. □

Combining Lemma B.1 to Lemma B.8, we have $\Pr[\mathcal{E}_0 = 1] \leq 1/2 + negl(\lambda)$, and that completes the proof.

B.1 Proof of Claim B.1

For any $I^* \in [1, Q]$, we prove that the difference between Game 3.I^* and Game 3.(I^*-1) is negligible. The two games are identical unless the additional aborting conditions introduced in Game 4 are triggered for the I^*-th decryption query. Let $\mathtt{Exp}$ be an experiment that is identical to Game 3.I^* except that if the additional aborting condition is triggered for the I^*-th decryption oracle query, then it also aborts but outputs 4. Let $\mathcal{F}$ be the output of $\mathtt{Exp}$, then it is sufficient to prove that

$$\Pr[\mathcal{F} = 4] \leq negl(\lambda) \tag{11}$$

We define the following auxiliary experiments to prove Equation (11). Recall that we use $\mathcal{M} = \mathcal{M}(\lambda)$ to denote the upper bound of the polynomial N contained in the evaluated ciphertexts submitted to the decryption oracle.

- $\mathtt{Exp}'_{\mathcal{M}}$. This is identical to $\mathtt{Exp}$ except that in the beginning, the challenger generates $crs^{(1)}_{\mathsf{PC}}, td^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, td^{(2)}_{\mathsf{PC}}$ as follows:

$$(crs^{(1)}_{\mathsf{PC}}, td^{(1)}_{\mathsf{PC}}) \leftarrow \mathsf{PC}.\mathtt{Setup}(1^\lambda, \mathtt{C}_c[sk_0, sk_1, \mathcal{M}])$$

$$(crs^{(2)}_{\mathsf{PC}}, td^{(2)}_{\mathsf{PC}}) \leftarrow \mathsf{PC}.\mathtt{Setup}(1^\lambda, \mathtt{C}_c[sk_0, sk_1, \mathcal{M}])$$

- $\mathtt{Exp}_{\mathcal{M}}$. This is identical to $\mathtt{Exp}'_{\mathcal{M}}$ except that if the I^*-th decryption query, which is either CT or $((CT_1, \ldots, CT_L), CT)$, is for an evaluated ciphertext $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \mathit{\Pi}, l, N)$, then the challenger extracts $(a^{(1)}, j^{(1)})$ from $com^{(1)}$ and additionally checks if $a^{(1)} = 1$ when deciding whether to abort and output 4.
  In more detail, if the query is at Phase 2, it proceeds as follows:
  1. If N is not polynomial in λ: Return $\bot$.
  2. If $l > N \vee d > N$: Return $\bot$.
  3. $\mathtt{P} = \mathtt{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$
  4. If $\mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathtt{P}, \mathit{\Pi}) = 0$: Return $\bot$.
  5. If $\mathsf{PC}.\mathtt{Verify}(crs^{(1)}_{\mathsf{PC}}, N, com^{(1)}, N, \mathcal{C}, \rho^{(1)}) = 0$: Return $\bot$.
  6. If $\mathsf{PC}.\mathtt{Verify}(crs^{(2)}_{\mathsf{PC}}, N, com^{(2)}, N, \mathcal{C}, \rho^{(2)}) = 0$: Return $\bot$.
  7. $(a^{(1)}, j^{(1)}) = \mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)})$.
  8. If $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_1) \wedge a^{(1)} = 1$:
     (a) Abort the game and output 4.
  9. Return $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0)$.

  On the other hand, if the query is at Phase 4, it proceeds as follows:
  1. If N is not polynomial in λ: Return $\bot$.
  2. If $l > N \vee d > N$: Return $\bot$.
  3. $\mathtt{P} = \mathtt{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$
  4. If $\mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathtt{P}, \mathit{\Pi}) = 0$: Return $\bot$.
  5. If $\mathsf{PC}.\mathtt{Verify}(crs^{(1)}_{\mathsf{PC}}, N, com^{(1)}, N, \mathcal{C}, \rho^{(1)}) = 0$: Return $\bot$.
  6. If $\mathsf{PC}.\mathtt{Verify}(crs^{(2)}_{\mathsf{PC}}, N, com^{(2)}, N, \mathcal{C}, \rho^{(2)}) = 0$: Return $\bot$.
  7. For $i \in [1, L]$:
     (a) If $CT^* = CT_i$: Return $\top$.
     (b) Parse $CT_i = (0, ct_{i,0}, ct_{i,1}, \sigma_i)$.
     (c) If $\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_{i,0}, ct_{i,1}), \sigma_i) = 0$: Return $\top$.
  8. $(com, st) = \mathsf{PC}.\mathtt{Commit}(crs^{(3)}_{\mathsf{PC}}, L, (ct_{i,0}, ct_{i,1}, 0)_{i \in [1,L]})$.
  9. If $L \neq l \vee com \neq com^{(3)}$: Return $\top$.

10. $(a^{(1)}, j^{(1)}) = \mathsf{PC}.\mathtt{Extract}(td_{\mathsf{PC}}^{(1)}, N, com^{(1)})$.
11. If $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_1) \wedge a^{(1)} = 1$:
    (a) Abort the game and output 4.
12. Return $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0)$.

- $\mathtt{Exp}_h$. for $h \in [0, \mathcal{M}-1]$. This is identical to $\mathtt{Exp}_{h+1}$ except that in the beginning, the challenger generates $crs_{\mathsf{PC}}^{(1)}, td_{\mathsf{PC}}^{(1)}, crs_{\mathsf{PC}}^{(2)}, td_{\mathsf{PC}}^{(2)}$ as follows:

$$(crs_{\mathsf{PC}}^{(1)}, td_{\mathsf{PC}}^{(1)}) \leftarrow \mathsf{PC}.\mathtt{Setup}(1^\lambda, \mathtt{C}_c[sk_0, sk_1, h])$$

$$(crs_{\mathsf{PC}}^{(2)}, td_{\mathsf{PC}}^{(2)}) \leftarrow \mathsf{PC}.\mathtt{Setup}(1^\lambda, \mathtt{C}_c[sk_0, sk_1, h])$$

Let $\mathcal{F}'_{\mathcal{M}}$ be the output of $\mathtt{Exp}'_{\mathcal{M}}$ and let $\mathcal{F}_h$ be the output of $\mathtt{Exp}_h$ for $h \in [0, \mathcal{M}]$. We next show that $\Pr[\mathcal{F} = 4] \leq negl(\lambda)$ via proving the following lemmas.

Lemma B.9. $|\Pr[\mathcal{F} = 4] - \Pr[\mathcal{F}'_{\mathcal{M}} = 4]| \leq negl(\lambda)$.

Proof. In both games, the trapdoors $td_{\mathsf{PC}}^{(1)}, td_{\mathsf{PC}}^{(2)}$ are never used. Thus, indistinguishability between $\mathtt{Exp}$ and $\mathtt{Exp}'_{\mathcal{M}}$ comes from the predicate hiding property of PC directly. □

Lemma B.10. $|\Pr[\mathcal{F}'_{\mathcal{M}} = 4] - \Pr[\mathcal{F}_{\mathcal{M}} = 4]| \leq negl(\lambda)$.

Proof. The experiments $\mathtt{Exp}'_{\mathcal{M}}$ and $\mathtt{Exp}_{\mathcal{M}}$ behave identically unless the I^*-th decryption query, which is either CT or $((CT_1, \ldots, CT_L), CT)$, is for an evaluated ciphertext $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$ and satisfies

$$\begin{aligned} &N = poly(\lambda) \wedge d \leq N \wedge \\ &\mathsf{PC}.\mathtt{Verify}(crs_{\mathsf{PC}}^{(1)}, N, com^{(1)}, N, \mathcal{C}, \rho^{(1)}) = 1 \wedge \\ &\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_1) \wedge \\ &\mathsf{PC}.\mathtt{Extract}(td_{\mathsf{PC}}^{(1)}, N, com^{(1)}) = (0, *) \end{aligned} \tag{12}$$

If $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_0) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_1)$ and $d \leq N \leq \mathcal{M}$, we have

$$\mathtt{C}_c[sk_0, sk_1, \mathcal{M}](\mathcal{C}) = 1$$

Then by the Extractability of PC, the probability that both $\mathsf{PC}.\mathtt{Verify}(crs_{\mathsf{PC}}^{(1)}, N, com^{(1)}, N, \mathcal{C}, \rho^{(1)}) = 1$ and $\mathsf{PC}.\mathtt{Extract}(td_{\mathsf{PC}}^{(1)}, N, com^{(1)}) = (0, *)$ will be negligible. Therefore, Equation (12) can only occur with negligible probability, and the difference between the two games will also be negligible. □

Lemma B.11. *For* $h \in [1, \mathcal{M}]$, $\Pr[\mathcal{F}_h = 4] \leq \Pr[\mathcal{F}_{h-1} = 4] + negl(\lambda)$.

Proof. To prove Lemma B.11, we define the following auxiliary experiments:

- $\mathtt{Exp}_{h,1}$. This is identical to $\mathtt{Exp}_h$ except that in the beginning, the challenger generates

$$(crs_{\mathsf{PC}}^{(2)}, td_{\mathsf{PC}}^{(2)}) \leftarrow \mathsf{PC}.\mathtt{Setup}(1^\lambda, \mathtt{C}_c[sk_0, sk_1, h-1])$$

- $\mathtt{Exp}_{h,2}$. This is identical to $\mathtt{Exp}_{h,1}$ except that if the I^*-th decryption query, which is either CT or $((CT_1, \ldots, CT_L), CT)$, is for an evaluated ciphertext $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$, then the challenger additionally extracts $(a^{(2)}, j^{(2)})$ from $com^{(2)}$ and additionally checks if $a^{(2)} = 1$ when deciding whether to abort and output 4.
- $\mathtt{Exp}_{h,3}$. This is identical to $\mathtt{Exp}_{h,2}$ except that if the I^*-th decryption query, which is either CT or $((CT_1, \ldots, CT_L), CT)$, is for an evaluated ciphertext $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$, then the challenger does not extract $(a^{(1)}, j^{(1)})$ from $com^{(1)}$ and does not check if $a^{(1)} = 1$ when deciding whether to abort and output 4.
- $\mathtt{Exp}_{h,4}$. This is identical to $\mathtt{Exp}_{h,3}$ except that in the beginning, the challenger generates

$$(crs^{(1)}_{\mathsf{PC}}, td^{(1)}_{\mathsf{PC}}) \leftarrow \mathsf{PC}.\mathtt{Setup}(1^\lambda, \mathtt{C}_c[sk_0, sk_1, h-1])$$

- $\mathtt{Exp}_{h,5}$. This is identical to $\mathtt{Exp}_{h,4}$ except that if the I^*-th decryption query, which is either CT or $((CT_1, \ldots, CT_L), CT)$, is for an evaluated ciphertext $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$, then the challenger extracts $(a^{(1)}, j^{(1)})$ from $com^{(1)}$ again and additionally checks if $a^{(1)} = 1$ again when deciding whether to abort and output 4.
- $\mathtt{Exp}_{h,6}$. This is identical to $\mathtt{Exp}_{h,5}$ except that if the I^*-th decryption query, which is either CT or $((CT_1, \ldots, CT_L), CT)$, is for an evaluated ciphertext $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$, then the challenger does not extract $(a^{(2)}, j^{(2)})$ from $com^{(2)}$ and does not check if $a^{(2)} = 1$ when deciding whether to abort and output 4.

Let $\mathcal{F}_{h,i}$ be the output of $\mathtt{Exp}_{h,i}$ for $i \in [1, 6]$. Note that $\mathtt{Exp}_{h,6}$ is identical to $\mathtt{Exp}_{h-1}$, therefore, we have $\Pr[\mathcal{F}_{h,6} = 4] = \Pr[\mathcal{F}_{h-1} = 4]$. We next show that $\Pr[\mathcal{F}_h = 4] \leq \Pr[\mathcal{F}_{h-1} = 4] + negl(\lambda)$ via proving the following claims.

Claim B.2. $|\Pr[\mathcal{F}_h = 4] - \Pr[\mathcal{F}_{h,1} = 4]| \leq negl(\lambda)$.

Proof. As in $\mathtt{Exp}_h$ and $\mathtt{Exp}_{h,1}$, $td^{(2)}_{\mathsf{PC}}$ is never used, indistinguishability between the two experiments comes from the predicate hiding property of PC directly. □

Claim B.3. $|\Pr[\mathcal{F}_{h,1} = 4] - \Pr[\mathcal{F}_{h,2} = 4]| \leq negl(\lambda)$.

Proof. We prove Claim B.3 in Appendix B.1.1. □

Claim B.4. $\Pr[\mathcal{F}_{h,2} = 4] \leq \Pr[\mathcal{F}_{h,3} = 4]$.

Proof. The challenger will check fewer conditions in $\mathtt{Exp}_{h,3}$, therefore, the probability that it can output 4 will not decrease. □

Claim B.5. $|\Pr[\mathcal{F}_{h,3} = 4] - \Pr[\mathcal{F}_{h,4} = 4]| \leq negl(\lambda)$.

Proof. As in $\mathtt{Exp}_{h,3}$ and $\mathtt{Exp}_{h,4}$, $td^{(1)}_{\mathsf{PC}}$ is never used, indistinguishability between the two experiments comes from the predicate hiding property of PC directly. □

Claim B.6. $|\Pr[\mathcal{F}_{h,4} = 4] - \Pr[\mathcal{F}_{h,5} = 4]| \leq negl(\lambda)$.

Proof. We prove Claim B.6 in Appendix B.1.2. □

Claim B.7. $\Pr[\mathcal{F}_{h,5} = 4] \leq \Pr[\mathcal{F}_{h,6} = 4]$.

Proof. The challenger will check fewer conditions in $\texttt{Exp}_{h,6}$, therefore, the probability that it can output 4 will not decrease. □

Combining Claim B.2 to Claim B.7, we have $\Pr[\mathcal{F}_h = 4] \leq \Pr[\mathcal{F}_{h-1} = 4] + negl(\lambda)$, and that completes the proof of Lemma B.11.

□

Lemma B.12. $\Pr[\mathcal{F}_0 = 4] \leq negl(\lambda)$.

Proof. We prove Lemma B.12 in Appendix B.1.3. □

Combining Lemma B.9 to Lemma B.12, we have $\Pr[\mathcal{F} = 4] \leq negl(\lambda)$, i.e., the new aborting conditions introduced in Game 4 will be triggered in the I^*-query in Game 3.I^* with only negligible probability and that completes the proof.

B.1.1 Proof of Claim B.3

We prove the indistinguishability between $\texttt{Exp}_{h,1}$ and $\texttt{Exp}_{h,2}$ in this section. The two experiments behave identically unless the I^*-th decryption query, which is either CT or $((CT_1, \ldots, CT_L), CT)$, is for an evaluated ciphertext $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$ and satisfies

$$\begin{aligned} &N = poly(\lambda) \wedge \mathsf{BI}.\texttt{Verify}(crs_{\mathsf{BI}}, N, \mathsf{P}, \Pi) = 1 \wedge \\ &\mathsf{PC}.\texttt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)}) = (1, *) \wedge \\ &\mathsf{PC}.\texttt{Extract}(td^{(2)}_{\mathsf{PC}}, N, com^{(2)}) = (0, *) \end{aligned} \tag{13}$$

where $\mathsf{P} = \mathsf{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$.

We define the above event as $\mathcal{BAD}$, then it is sufficient to show that $\mathcal{BAD}$ occurs in $\texttt{Exp}_{h,1}$ with only negligible probability. We prove this by defining the following hybrids. In the discussion below, we only consider the case that the I^*-th decryption query is an evaluated ciphertext (as otherwise, $\mathcal{BAD}$ will not occur) and we will use $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$ to denote CT in the I^*-th decryption query and sets $\mathsf{P} = \mathsf{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$.

- $\mathtt{H}_0$. The hybrid $\mathtt{H}_0$ behaves identically to $\texttt{Exp}_{h,1}$, except that after receiving the I^*-th decryption query, it aborts and outputs 5 once the event $\mathcal{BAD}$ occurs; otherwise, it also aborts but outputs 6.

- $\mathtt{H}_1$. This is identical to $\mathtt{H}_0$ except that the challenger samples $\jmath^* \xleftarrow{\$} [1, \mathcal{M}]$ in the beginning. Then after receiving the $\imath^*$-th decryption query, let

$$(a, j) = \mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)})$$

  the hybrid outputs 5 if $\mathcal{BAD}$ occurs and $\jmath^* = j$; otherwise, it also aborts but outputs 6. We define $\mathcal{BAD}' = \mathcal{BAD} \wedge (\jmath^* = j)$.
- $\mathtt{H}_2$. This is identical to $\mathtt{H}_1$ except that the challenger generates

$$(crs_{\mathsf{BI}}, td_{\mathsf{BI}}) \leftarrow \mathsf{BI}.\mathtt{Setup}(1^\lambda, 1^C, \jmath^*)$$

- $\mathtt{H}_3$. This is identical to $\mathtt{H}_2$ except that after receiving the $\imath^*$-th decryption query, the challenger further extracts

$$\mathcal{W}_{\jmath^*} \leftarrow \mathsf{BI}.\mathtt{Extract}(td_{\mathsf{BI}}, N, \mathtt{P}, \Pi)$$

  Then the hybrid outputs 5 if $\mathcal{BAD}'$ occurs and $\mathtt{P}(\jmath^*, \mathcal{W}_{\jmath^*}) = 1$; otherwise, it also aborts but outputs 6. We define $\mathcal{BAD}^\dagger = \mathcal{BAD}' \wedge (\mathtt{P}(\jmath^*, \mathcal{W}_{\jmath^*}) = 1)$.
- $\mathtt{H}_4$. This is identical to $\mathtt{H}_3$ except that after receiving the $\imath^*$-th decryption query, and let

$$\begin{aligned}\mathcal{W}_{\jmath^*} = (\mathcal{C}_{\jmath^*} = (ct_{\jmath^*,0}, ct_{\jmath^*,1}, d_{\jmath^*}), \sigma_{\jmath^*}, \rho^{(1)}_{\jmath^*}, \rho^{(2)}_{\jmath^*}, \rho^{(3)}_{\jmath^*}, \\ L, \mathcal{C}_L = (ct_{L,0}, ct_{L,1}, d_L), \rho^{(1)}_L, \rho^{(2)}_L, \\ R, \mathcal{C}_R = (ct_{R,0}, ct_{R,1}, d_R), \rho^{(1)}_R, \rho^{(2)}_R)\end{aligned}$$

  be the witness extracted from Π, the hybrid outputs 5 if $\mathcal{BAD}^\dagger$ occurs and $\mathtt{C}_c[sk_0, sk_1, h](\mathcal{C}_{\jmath^*}) = 1$; otherwise, it also aborts but outputs 6. We define $\mathcal{BAD}^* = \mathcal{BAD}^\dagger \wedge (\mathtt{C}_c[sk_0, sk_1, h](\mathcal{C}_{\jmath^*}) = 1)$.

Let $\mathcal{H}_i$ be the output of $\mathtt{H}_i$ for $i \in [0, 4]$. We next show that $\Pr[\mathcal{H}_0 = 5] \leq negl(\lambda)$ via proving the following claims.

Claim B.8. $\Pr[\mathcal{H}_1 = 5] = \frac{\Pr[\mathcal{H}_0 = 5]}{\mathcal{M}}$.

Proof. Assume that the event $\mathcal{BAD}$ occurs and let $(a, j) = \mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)})$. Then we have $a = 1$. Note that the extraction algorithm of PC will output either $(0, 0)$ or $(1, j')$ for $j' \in [1, N]$ on input a vector length N, thus, we have $1 \leq j \leq N \leq \mathcal{M}$.

Now, as $\jmath^* \xleftarrow{\$} [1, \mathcal{M}]$ and it is independent of the experiment before performing the final check, the probability that $\jmath^* = j$ will be exactly $1/\mathcal{M}$ if the event $\mathcal{BAD}$ occurs, and therefore, we have

$$\Pr[\mathcal{H}_1 = 5] = \frac{\Pr[\mathcal{H}_0 = 5]}{\mathcal{M}}$$

□

Claim B.9. $|\Pr[\mathcal{H}_1 = 5] - \Pr[\mathcal{H}_2 = 5]| \leq negl(\lambda)$.

Proof. In both $\mathtt{H}_1$ and $\mathtt{H}_2$, the trapdoor td_{BI} is never used, thus indistinguishability between them comes from the index hiding property of BI directly. □

Claim B.10. $|\Pr[\mathcal{H}_2 = 5] - \Pr[\mathcal{H}_3 = 5]| \leq negl(\lambda)$.

Proof. If $\mathcal{BAD}'$ occurs, we have

$$\begin{aligned}
&N = poly(\lambda) \wedge \mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathsf{P}, \Pi) = 1 \wedge \\
&\mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)}) = (1, \jmath^*) \wedge \\
&\mathsf{PC}.\mathtt{Extract}(td^{(2)}_{\mathsf{PC}}, N, com^{(2)}) = (0, *)
\end{aligned}$$

Note that as $(1, \jmath^*)$ is the output of $\mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)})$, we have $\jmath^* \in [1, N]$. Then by the somewhere argument of knowledge property of BI, we have

$$\Pr[\mathtt{P}(\jmath^*, \mathcal{W}_{\jmath^*}) = 0] \leq negl(\lambda)$$

That is, if $\mathcal{BAD}'$ occurs, then the new check introduced in Hybrid $\mathtt{H}_3$ can be passed with all but negligible probability and that completes the proof. □

Claim B.11. $|\Pr[\mathcal{H}_3 = 5] - \Pr[\mathcal{H}_4 = 5]| \leq negl(\lambda)$.

Proof. If $\mathcal{BAD}^\dagger$ occurs, we have

$$\begin{aligned}
&N = poly(\lambda) \wedge \mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathsf{P}, \Pi) = 1 \wedge \\
&\mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)}) = (1, \jmath^*) \wedge \\
&\mathsf{PC}.\mathtt{Extract}(td^{(2)}_{\mathsf{PC}}, N, com^{(2)}) = (0, *) \wedge \\
&\mathtt{P}(\jmath^*, \mathcal{W}_{\jmath^*}) = 1
\end{aligned}$$

where $\mathcal{W}_{\jmath^*} \leftarrow \mathsf{BI}.\mathtt{Extract}(td_{\mathsf{BI}}, N, \mathsf{P}, \Pi)$. Let

$$\begin{aligned}
\mathcal{W}_{\jmath^*} = (&\mathcal{C}_{\jmath^*} = (ct_{\jmath^*,0}, ct_{\jmath^*,1}, d_{\jmath^*}), \sigma_{\jmath^*}, \rho^{(1)}_{\jmath^*}, \rho^{(2)}_{\jmath^*}, \rho^{(3)}_{\jmath^*}, \\
&L, \mathcal{C}_L = (ct_{L,0}, ct_{L,1}, d_L), \rho^{(1)}_L, \rho^{(2)}_L, \\
&R, \mathcal{C}_R = (ct_{R,0}, ct_{R,1}, d_R), \rho^{(1)}_R, \rho^{(2)}_R)
\end{aligned}$$

Note that the circuit $\mathtt{P}$ will always verify if $\mathcal{C}_{\jmath^*}$ is committed in $com^{(1)}$ in both cases (i.e., the case that $\jmath^* \leq l$ and the case that $\jmath^* > l$), thus we have

$$\mathsf{PC}.\mathtt{Verify}(crs^{(1)}_{\mathsf{PC}}, N, com^{(1)}, \jmath^*, \mathcal{C}_{\jmath^*}, \rho^{(1)}_{\jmath^*}) = 1$$

Then by the extractability of PC, we have

$$\Pr[\mathtt{C}_c[sk_0, sk_1, h](\mathcal{C}_{\jmath^*}) = 0] \leq negl(\lambda)$$

That is, if $\mathcal{BAD}^\dagger$ occurs, then the new check introduced in Hybrid $\mathtt{H}_4$ can be passed with all but negligible probability and that completes the proof. □

Claim B.12. $\Pr[\mathcal{H}_4 = 5] \leq negl(\lambda)$.

Proof. Let $\mathcal{W}_{\jmath^*} \leftarrow \mathsf{BI}.\mathtt{Extract}(td_{\mathsf{BI}}, N, \mathtt{P}, \mathit{\Pi})$ and

$$\begin{aligned}\mathcal{W}_{\jmath^*} = (\mathcal{C}_{\jmath^*} = (ct_{\jmath^*,0}, ct_{\jmath^*,1}, d_{\jmath^*}), \sigma_{\jmath^*}, \rho^{(1)}_{\jmath^*}, \rho^{(2)}_{\jmath^*}, \rho^{(3)}_{\jmath^*},\\ L, \mathcal{C}_L = (ct_{L,0}, ct_{L,1}, d_L), \rho^{(1)}_L, \rho^{(2)}_L,\\ R, \mathcal{C}_R = (ct_{R,0}, ct_{R,1}, d_R), \rho^{(1)}_R, \rho^{(2)}_R)\end{aligned}$$

Assume that $\mathtt{P}(\jmath^*, \mathcal{W}_{\jmath^*}) = 1$ and $\mathtt{C}_c[sk_0, sk_1, h](\mathcal{C}_{\jmath^*}) = 1$ (i.e. $d_{\jmath^*} \leq h$, and $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_{\jmath^*,0}) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_{\jmath^*,1})$), we consider the following two cases:

- **Case I:** $\jmath^* \leq l$**.** In this case, we have $d_{\jmath^*} = 0 \leq h - 1$ and $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_{\jmath^*,0}) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_{\jmath^*,1})$, i.e.,

  $$\mathtt{C}_c[sk_0, sk_1, h-1](\mathcal{C}_{\jmath^*}) = 1$$

  In addition, we have

  $$\mathsf{PC}.\mathtt{Verify}(crs^{(2)}_{\mathsf{PC}}, N, com^{(2)}, \jmath^*, \mathcal{C}_{\jmath^*}, \rho^{(2)}_{\jmath^*}) = 1$$

- **Case II:** $\jmath^* > l$**.** In this case, we have

  $$ct_{\jmath^*,0} = \mathsf{HE}.\mathtt{Eval}(pk_0, ct_{L,0}, ct_{R,0}), \quad ct_{\jmath^*,1} = \mathsf{HE}.\mathtt{Eval}(pk_1, ct_{L,1}, ct_{R,1})$$

  If both $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_{L,0}) = \mathsf{HE}.\mathtt{Dec}(sk_1, ct_{L,1})$ and $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_{R,0}) = \mathsf{HE}.\mathtt{Dec}(sk_1, ct_{R,1})$, then by the *strong evaluation correctness* of HE, we must have $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_{\jmath^*,0}) = \mathsf{HE}.\mathtt{Dec}(sk_1, ct_{\jmath^*,1})$, this contradicts the condition that $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_{\jmath^*,0}) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_{\jmath^*,1})$. Therefore, we must have either

  $$\mathsf{HE}.\mathtt{Dec}(sk_0, ct_{L,0}) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_{L,1})$$

  or

  $$\mathsf{HE}.\mathtt{Dec}(sk_0, ct_{R,0}) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_{R,1})$$

  Moreover, we have $d_L \leq d_{\jmath^*} - 1 \leq h - 1$ and $d_R \leq d_{\jmath^*} - 1 \leq h - 1$. Therefore, we have

  $$\mathtt{C}_c[sk_0, sk_1, h-1](\mathcal{C}_L) = 1 \quad \vee \quad \mathtt{C}_c[sk_0, sk_1, h-1](\mathcal{C}_R) = 1$$

  Besides, we have

  $$\mathsf{PC}.\mathtt{Verify}(crs^{(2)}_{\mathsf{PC}}, N, com^{(2)}, L, \mathcal{C}_L, \rho^{(2)}_L) = 1$$

  $$\mathsf{PC}.\mathtt{Verify}(crs^{(2)}_{\mathsf{PC}}, N, com^{(2)}, R, \mathcal{C}_R, \rho^{(2)}_R) = 1$$

In any case, we can always find at least one $(i, \mathcal{C}_i)$ s.t.

$$\mathtt{C}_c[sk_0, sk_1, h-1](\mathcal{C}_i) = 1 \wedge \mathsf{PC}.\mathtt{Verify}(crs^{(2)}_{\mathsf{PC}}, N, com^{(2)}, i, \mathcal{C}_i, \rho^{(2)}_i) = 1$$

where $i \in \{\mathcal{I}^*, L, R\}$. Then by the extractability of PC, we have

$$\Pr[\mathsf{PC}.\mathtt{Extract}(td^{(2)}_{\mathsf{PC}}, N, com^{(2)}) = (0, *)] \leq negl(\lambda)$$

To summarize, we proved that

$$\Pr[\mathsf{PC}.\mathtt{Extract}(td^{(2)}_{\mathsf{PC}}, N, com^{(2)}) = (0, *) \wedge \\ \mathtt{P}(\mathcal{I}^*, \mathcal{W}_{\mathcal{I}^*}) = 1 \wedge \mathtt{C}_c[sk_0, sk_1, h](\mathcal{C}_{\mathcal{I}^*}) = 1] \leq negl(\lambda)$$

However, if $\mathcal{BAD}^*$ occurs, we must have

$$\mathsf{PC}.\mathtt{Extract}(td^{(2)}_{\mathsf{PC}}, N, com^{(2)}) = (0, *) \wedge \\ \mathtt{P}(\mathcal{I}^*, \mathcal{W}_{\mathcal{I}^*}) = 1 \wedge \mathtt{C}_c[sk_0, sk_1, h](\mathcal{C}_{\mathcal{I}^*}) = 1$$

Thus, $\mathcal{BAD}^*$ occurs with only negligible probability and $\Pr[\mathcal{H}_4 = 5] \leq negl(\lambda)$. □

Combining Claim B.8 to Claim B.12, we have $\Pr[\mathcal{H}_0 = 5] \leq negl(\lambda)$, i.e., the event $\mathcal{BAD}$ will occur at $\mathtt{Exp}_{h,1}$ with only negligible probability. Therefore, $\mathtt{Exp}_{h,1}$ and $\mathtt{Exp}_{h,2}$ will behave differently with only negligible probability and that completes the proof of Claim B.3.

B.1.2 Proof of Claim B.6

The proof is similar to the proof of Claim B.3 and we include the full proof for completeness. The experiments $\mathtt{Exp}_{h,4}$ and $\mathtt{Exp}_{h,5}$ behave identically unless the $\mathcal{I}^*$-th decryption query, which is either CT or $((CT_1, \ldots, CT_L), CT)$, is for an evaluated ciphertext $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$ and satisfies

$$N = poly(\lambda) \wedge \mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathtt{P}, \Pi) = 1 \wedge \\ \mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)}) = (0, *) \wedge \\ \mathsf{PC}.\mathtt{Extract}(td^{(2)}_{\mathsf{PC}}, N, com^{(2)}) = (1, *) \tag{14}$$

where $\mathtt{P} = \mathtt{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$.

We define the above event as $\mathcal{BAD}$, then it is sufficient to show that $\mathcal{BAD}$ occurs in $\mathtt{Exp}_{h,4}$ with only negligible probability. We prove this by defining the following hybrids. In the discussion below, we only consider the case that the $\mathcal{I}^*$-th decryption query is an evaluated ciphertext (as otherwise, $\mathcal{BAD}$ will not occur) and we will use $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$ to denote CT in the $\mathcal{I}^*$-th decryption query and sets $\mathtt{P} = \mathtt{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$.

- $\mathtt{H}_0$. The hybrid $\mathtt{H}_0$ behaves identically to $\mathtt{Exp}_{h,4}$, except that after receiving the $\mathcal{I}^*$-th decryption query, it aborts and outputs 5 once the event $\mathcal{BAD}$ occurs; otherwise, it also aborts but outputs 6.

- $\mathtt{H}_1$. This is identical to $\mathtt{H}_0$ except that the challenger samples $\jmath^* \xleftarrow{\$} [1, \mathcal{M}]$ in the beginning. Then after receiving the $\imath^*$-th decryption query, let

$$(a, j) = \mathsf{PC}.\mathtt{Extract}(td^{(2)}_{\mathsf{PC}}, N, com^{(2)})$$

  the hybrid outputs 5 if $\mathcal{BAD}$ occurs and $\jmath^* = j$; otherwise, it also aborts but outputs 6. We define $\mathcal{BAD}' = \mathcal{BAD} \wedge (\jmath^* = j)$.
- $\mathtt{H}_2$. This is identical to $\mathtt{H}_1$ except that the challenger generates

$$(crs_{\mathsf{BI}}, td_{\mathsf{BI}}) \leftarrow \mathsf{BI}.\mathtt{Setup}(1^\lambda, 1^C, \jmath^*)$$

- $\mathtt{H}_3$. This is identical to $\mathtt{H}_2$ except that after receiving the $\imath^*$-th decryption query, the challenger further extracts

$$\mathcal{W}_{\jmath^*} \leftarrow \mathsf{BI}.\mathtt{Extract}(td_{\mathsf{BI}}, N, \mathtt{P}, \Pi)$$

  Then the hybrid outputs 5 if $\mathcal{BAD}'$ occurs and $\mathtt{P}(\jmath^*, \mathcal{W}_{\jmath^*}) = 1$; otherwise, it also aborts but outputs 6. We define $\mathcal{BAD}^\dagger = \mathcal{BAD}' \wedge (\mathtt{P}(\jmath^*, \mathcal{W}_{\jmath^*}) = 1)$.
- $\mathtt{H}_4$. This is identical to $\mathtt{H}_3$ except that after receiving the $\imath^*$-th decryption query, and let

$$\begin{aligned}\mathcal{W}_{\jmath^*} = (\mathcal{C}_{\jmath^*} = (ct_{\jmath^*,0}, ct_{\jmath^*,1}, d_{\jmath^*}), \sigma_{\jmath^*}, \rho^{(1)}_{\jmath^*}, \rho^{(2)}_{\jmath^*}, \rho^{(3)}_{\jmath^*},\\ L, \mathcal{C}_L = (ct_{L,0}, ct_{L,1}, d_L), \rho^{(1)}_L, \rho^{(2)}_L,\\ R, \mathcal{C}_R = (ct_{R,0}, ct_{R,1}, d_R), \rho^{(1)}_R, \rho^{(2)}_R)\end{aligned}$$

  be the witness extracted from Π, the hybrid outputs 5 if $\mathcal{BAD}^\dagger$ occurs and $\mathtt{C}_c[sk_0, sk_1, h-1](\mathcal{C}_{\jmath^*}) = 1$; otherwise, it also aborts but outputs 6. We define $\mathcal{BAD}^* = \mathcal{BAD}^\dagger \wedge (\mathtt{C}_c[sk_0, sk_1, h-1](\mathcal{C}_{\jmath^*}) = 1)$.

Let $\mathcal{H}_i$ be the output of $\mathtt{H}_i$ for $i \in [0, 4]$. We next show that $\Pr[\mathcal{H}_0 = 5] \leq negl(\lambda)$ via proving the following claims.

Claim B.13. $\Pr[\mathcal{H}_1 = 5] = \frac{\Pr[\mathcal{H}_0 = 5]}{\mathcal{M}}$.

Proof. Assume that the event $\mathcal{BAD}$ occurs and let $(a, j) = \mathsf{PC}.\mathtt{Extract}(td^{(2)}_{\mathsf{PC}}, N, com^{(2)})$. Then we have $a = 1$. Note that the extraction algorithm of PC will output either $(0, 0)$ or $(1, j')$ for $j' \in [1, N]$ on input a vector length N, thus, we have $1 \leq j \leq N \leq \mathcal{M}$.

Now, as $\jmath^* \xleftarrow{\$} [1, \mathcal{M}]$ and it is independent of the experiment before performing the final check, the probability that $\jmath^* = j$ will be exactly $1/\mathcal{M}$ if the event $\mathcal{BAD}$ occurs, and therefore, we have

$$\Pr[\mathcal{H}_1 = 5] = \frac{\Pr[\mathcal{H}_0 = 5]}{\mathcal{M}}$$

□

Claim B.14. $|\Pr[\mathcal{H}_1 = 5] - \Pr[\mathcal{H}_2 = 5]| \leq negl(\lambda)$.

Proof. In both $\mathtt{H}_1$ and $\mathtt{H}_2$, the trapdoor td_{BI} is never used, thus indistinguishability between them comes from the index hiding property of BI directly. □

Claim B.15. $|\Pr[\mathcal{H}_2 = 5] - \Pr[\mathcal{H}_3 = 5]| \leq negl(\lambda)$.

Proof. If $\mathcal{BAD}'$ occurs, we have

$$\begin{aligned}
&N = poly(\lambda) \wedge \mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathtt{P}, \Pi) = 1 \wedge \\
&\mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)}) = (0, *) \wedge \\
&\mathsf{PC}.\mathtt{Extract}(td^{(2)}_{\mathsf{PC}}, N, com^{(2)}) = (1, \jmath^*)
\end{aligned}$$

Note that as $(1, \jmath^*)$ is the output of $\mathsf{PC}.\mathtt{Extract}(td^{(2)}_{\mathsf{PC}}, N, com^{(2)})$, we have $\jmath^* \in [1, N]$. Then by the somewhere argument of knowledge property of BI, we have

$$\Pr[\mathtt{P}(\jmath^*, \mathcal{W}_{\jmath^*}) = 0] \leq negl(\lambda)$$

That is, if $\mathcal{BAD}'$ occurs, then the new check introduced in Hybrid $\mathtt{H}_3$ can be passed with all but negligible probability and that completes the proof. □

Claim B.16. $|\Pr[\mathcal{H}_3 = 5] - \Pr[\mathcal{H}_4 = 5]| \leq negl(\lambda)$.

Proof. If $\mathcal{BAD}^\dagger$ occurs, we have

$$\begin{aligned}
&N = poly(\lambda) \wedge \mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathtt{P}, \Pi) = 1 \wedge \\
&\mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)}) = (0, *) \wedge \\
&\mathsf{PC}.\mathtt{Extract}(td^{(2)}_{\mathsf{PC}}, N, com^{(2)}) = (1, \jmath^*) \wedge \\
&\mathtt{P}(\jmath^*, \mathcal{W}_{\jmath^*}) = 1
\end{aligned}$$

where $\mathcal{W}_{\jmath^*} \leftarrow \mathsf{BI}.\mathtt{Extract}(td_{\mathsf{BI}}, N, \mathtt{P}, \Pi)$. Let

$$\begin{aligned}
\mathcal{W}_{\jmath^*} = (&\mathcal{C}_{\jmath^*} = (ct_{\jmath^*,0}, ct_{\jmath^*,1}, d_{\jmath^*}), \sigma_{\jmath^*}, \rho^{(1)}_{\jmath^*}, \rho^{(2)}_{\jmath^*}, \rho^{(3)}_{\jmath^*}, \\
&L, \mathcal{C}_L = (ct_{L,0}, ct_{L,1}, d_L), \rho^{(1)}_L, \rho^{(2)}_L, \\
&R, \mathcal{C}_R = (ct_{R,0}, ct_{R,1}, d_R), \rho^{(1)}_R, \rho^{(2)}_R)
\end{aligned}$$

Note that the circuit $\mathtt{P}$ will always verify if $\mathcal{C}_{\jmath^*}$ is committed in $com^{(2)}$ in both cases (i.e., the case that $\jmath^* \leq l$ and the case that $\jmath^* > l$), thus we have

$$\mathsf{PC}.\mathtt{Verify}(crs^{(2)}_{\mathsf{PC}}, N, com^{(2)}, \jmath^*, \mathcal{C}_{\jmath^*}, \rho^{(2)}_{\jmath^*}) = 1$$

Then by the extractability of PC, we have

$$\Pr[\mathtt{C}_c[sk_0, sk_1, h-1](\mathcal{C}_{\jmath^*}) = 0] \leq negl(\lambda)$$

That is, if $\mathcal{BAD}^\dagger$ occurs, then the new check introduced in Hybrid $\mathtt{H}_4$ can be passed with all but negligible probability and that completes the proof. □

Claim B.17. $\Pr[\mathcal{H}_4 = 5] \leq negl(\lambda)$.

Proof. Let $\mathcal{W}_{\jmath^*} \leftarrow \mathsf{BI}.\mathtt{Extract}(td_{\mathsf{BI}}, N, \mathtt{P}, \Pi)$ and

$$\begin{aligned}\mathcal{W}_{\jmath^*} = (&\mathcal{C}_{\jmath^*} = (ct_{\jmath^*,0}, ct_{\jmath^*,1}, d_{\jmath^*}), \sigma_{\jmath^*}, \rho^{(1)}_{\jmath^*}, \rho^{(2)}_{\jmath^*}, \rho^{(3)}_{\jmath^*},\\ &L, \mathcal{C}_L = (ct_{L,0}, ct_{L,1}, d_L), \rho^{(1)}_L, \rho^{(2)}_L,\\ &R, \mathcal{C}_R = (ct_{R,0}, ct_{R,1}, d_R), \rho^{(1)}_R, \rho^{(2)}_R)\end{aligned}$$

Assume that $\mathtt{P}(\jmath^*, \mathcal{W}_{\jmath^*}) = 1$. Since the circuit $\mathtt{P}$ will always verify if $\mathcal{C}_{\jmath^*}$ is committed in $com^{(1)}$ in both cases (i.e., the case that $\jmath^* \leq l$ and the case that $\jmath^* > l$), we have

$$\mathsf{PC}.\mathtt{Verify}(crs^{(1)}_{\mathsf{PC}}, N, com^{(1)}, \jmath^*, \mathcal{C}_{\jmath^*}, \rho^{(1)}_{\jmath^*}) = 1$$

In addition, assume that $\mathtt{C}_c[sk_0, sk_1, h-1](\mathcal{C}_{\jmath^*}) = 1$. Then by the extractability of PC, we have

$$\Pr[\mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)}) = (0, *)] \leq negl(\lambda)$$

To summarize, we proved that

$$\begin{aligned}\Pr[\mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)}) = (0, *) \wedge&\\ \mathtt{P}(\jmath^*, \mathcal{W}_{\jmath^*}) = 1 \wedge \mathtt{C}_c[sk_0, sk_1, h-1](\mathcal{C}_{\jmath^*}) = 1] &\leq negl(\lambda)\end{aligned}$$

However, if $\mathcal{BAD}^*$ occurs, we must have

$$\begin{aligned}&\mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)}) = (0, *) \wedge\\ &\mathtt{P}(\jmath^*, \mathcal{W}_{\jmath^*}) = 1 \wedge \mathtt{C}_c[sk_0, sk_1, h-1](\mathcal{C}_{\jmath^*}) = 1\end{aligned}$$

Thus, $\mathcal{BAD}^*$ occurs with only negligible probability and $\Pr[\mathcal{H}_4 = 5] \leq negl(\lambda)$. □

Combining Claim B.13 to Claim B.17, we have $\Pr[\mathcal{H}_0 = 5] \leq negl(\lambda)$, i.e., the event $\mathcal{BAD}$ will occur at $\mathtt{Exp}_{h,4}$ with only negligible probability. Therefore, $\mathtt{Exp}_{h,4}$ and $\mathtt{Exp}_{h,5}$ will behave differently with only negligible probability and that completes the proof of Claim B.6.

B.1.3 Proof of Lemma B.12

We prove that the experiment $\mathtt{Exp}_0$ will output 4 with only negligible probability in this section. The main idea of the proof is similar to the main idea of the proof of Claim B.3.

We define $\mathcal{BAD}_2$ be the event that the I^*-th decryption query is an evaluated ciphertext at Phase II, where the queried ciphertext $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$, and satisfies

$$\begin{aligned}&N = poly(\lambda) \wedge \mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathtt{P}, \Pi) = 1 \wedge\\ &\mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)}) = (1, *)\end{aligned}$$

where $\mathtt{P} = \mathtt{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$.

We define $\mathcal{BAD}_4$ be the event that the I^*-th decryption query is an evaluated ciphertext at Phase IV, where the queried ciphertext includes the input ciphertexts $(CT_i = (0, ict_{i,0}, ict_{i,1}, i\sigma_i))_{i\in[1,L]}$ and the target ciphertext $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$, and satisfies

$$
\begin{aligned}
&N = poly(\lambda) \wedge \mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathtt{P}, \Pi) = 1 \wedge \\
&\mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)}) = (1, *) \wedge \\
&L = l \wedge \mathsf{PC}.\mathtt{Commit}(crs^{(3)}_{\mathsf{PC}}, L, (ict_{i,0}, ict_{i,1}, 0)_{i\in[1,L]}) = (com^{(3)}, *) \wedge \\
&\forall i \in [1, L], CT^* \neq CT_i \wedge \\
&\qquad \mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ict_{i,0}, ict_{i,1}), i\sigma_i) = 1
\end{aligned}
$$

where $\mathtt{P} = \mathtt{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$.

It is easy to see $\mathtt{Exp}_0$ outputs 4 only if either $\mathcal{BAD}_2$ or $\mathcal{BAD}_4$ occurs. Thus, it is sufficient to show that $\mathcal{BAD}_2 \vee \mathcal{BAD}_4$ occurs in $\mathtt{Exp}_0$ with only negligible probability. We prove this by defining the following hybrids. In the discussion below, we only consider the case that the I^*-th decryption query is an evaluated ciphertext (as otherwise, the bad events will not occur); we will use $CT = (1, \mathcal{C} = (ct_0, ct_1, d), com^{(1)}, com^{(2)}, com^{(3)}, \rho^{(1)}, \rho^{(2)}, \Pi, l, N)$ to denote the target ciphertext CT in the I^*-th decryption query, we will use $(CT_i = (0, ict_{i,0}, ict_{i,1}, i\sigma_i))_{i\in[1,L]}$ to denote the input ciphertexts in the I^*-th decryption query (if they exist), and we will set $\mathtt{P} = \mathtt{C}_e[pk_0, pk_1, crs_{\mathsf{ZK}}, crs^{(1)}_{\mathsf{PC}}, crs^{(2)}_{\mathsf{PC}}, crs^{(3)}_{\mathsf{PC}}, com^{(1)}, com^{(2)}, com^{(3)}, l, N]$.

- $\mathtt{H}_0$. The hybrid $\mathtt{H}_0$ behaves identically to $\mathtt{Exp}_0$, except that after receiving the I^*-th decryption query, it aborts and outputs 5 once the event $\mathcal{BAD}_2$ or $\mathcal{BAD}_4$ occurs; otherwise, it also aborts but outputs 6.
- $\mathtt{H}_1$. This is identical to $\mathtt{H}_0$ except that the challenger samples $\mathcal{J}^* \xleftarrow{\$} [1, \mathcal{M}]$ in the beginning. Then after receiving the I^*-th decryption query, let

  $$(a, j) = \mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)})$$

  the hybrid outputs 5 if $\mathcal{BAD}'_2$ or $\mathcal{BAD}'_4$ occurs, where we define

  $$\mathcal{BAD}'_2 = \mathcal{BAD}_2 \wedge (\mathcal{J}^* = j), \quad \mathcal{BAD}'_4 = \mathcal{BAD}_4 \wedge (\mathcal{J}^* = j)$$

  otherwise, it also aborts but outputs 6.
- $\mathtt{H}_2$. This is identical to $\mathtt{H}_1$ except that the challenger generates

  $$(crs_{\mathsf{BI}}, td_{\mathsf{BI}}) \leftarrow \mathsf{BI}.\mathtt{Setup}(1^\lambda, 1^C, \mathcal{J}^*)$$

- $\mathtt{H}_3$. This is identical to $\mathtt{H}_2$ except that after receiving the I^*-th decryption query, the challenger further extracts

  $$\mathcal{W}_{\mathcal{J}^*} \leftarrow \mathsf{BI}.\mathtt{Extract}(td_{\mathsf{BI}}, N, \mathtt{P}, \Pi)$$

Then the hybrid outputs 5 if $\mathcal{BAD}_2^\dagger$ or $\mathcal{BAD}_4^\dagger$ occurs, where we define

$$\mathcal{BAD}_2^\dagger = \mathcal{BAD}_2' \wedge (\mathtt{P}(\mathcal{J}^*, \mathcal{W}_{\mathcal{J}^*}) = 1)$$

$$\mathcal{BAD}_4^\dagger = \mathcal{BAD}_4' \wedge (\mathtt{P}(\mathcal{J}^*, \mathcal{W}_{\mathcal{J}^*}) = 1)$$

otherwise, it also aborts but outputs 6.

- $\mathtt{H}_4$. This is identical to $\mathtt{H}_3$ except that after receiving the I^*-th decryption query, and let

$$\begin{aligned}\mathcal{W}_{\mathcal{J}^*} = (&\mathcal{C}_{\mathcal{J}^*} = (ct_{\mathcal{J}^*,0}, ct_{\mathcal{J}^*,1}, d_{\mathcal{J}^*}), \sigma_{\mathcal{J}^*}, \rho_{\mathcal{J}^*}^{(1)}, \rho_{\mathcal{J}^*}^{(2)}, \rho_{\mathcal{J}^*}^{(3)},\\ &L, \mathcal{C}_L = (ct_{L,0}, ct_{L,1}, d_L), \rho_L^{(1)}, \rho_L^{(2)},\\ &R, \mathcal{C}_R = (ct_{R,0}, ct_{R,1}, d_R), \rho_R^{(1)}, \rho_R^{(2)})\end{aligned}$$

the hybrid outputs 5 if $\mathcal{BAD}_2^*$ or $\mathcal{BAD}_4^*$ occurs, where we define

$$\mathcal{BAD}_2^* = \mathcal{BAD}_2^\dagger \wedge (\mathtt{C}_c[sk_0, sk_1, 0](\mathcal{C}_{\mathcal{J}^*}) = 1)$$

$$\mathcal{BAD}_4^* = \mathcal{BAD}_4^\dagger \wedge (\mathtt{C}_c[sk_0, sk_1, 0](\mathcal{C}_{\mathcal{J}^*}) = 1)$$

otherwise, it also aborts but outputs 6.

- $\mathtt{H}_5$. This is identical to $\mathtt{H}_4$ except that after receiving the I^*-th decryption query, and let

$$\begin{aligned}\mathcal{W}_{\mathcal{J}^*} = (&\mathcal{C}_{\mathcal{J}^*} = (ct_{\mathcal{J}^*,0}, ct_{\mathcal{J}^*,1}, d_{\mathcal{J}^*}), \sigma_{\mathcal{J}^*}, \rho_{\mathcal{J}^*}^{(1)}, \rho_{\mathcal{J}^*}^{(2)}, \rho_{\mathcal{J}^*}^{(3)},\\ &L, \mathcal{C}_L = (ct_{L,0}, ct_{L,1}, d_L), \rho_L^{(1)}, \rho_L^{(2)},\\ &R, \mathcal{C}_R = (ct_{R,0}, ct_{R,1}, d_R), \rho_R^{(1)}, \rho_R^{(2)})\end{aligned}$$

the hybrid outputs 5 if $\mathcal{BAD}_2^\diamond$ or $\mathcal{BAD}_4^\diamond$ occurs, where we define

$$\mathcal{BAD}_2^\diamond = \mathcal{BAD}_2^* \wedge (\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_{\mathcal{J}^*,0}, ct_{\mathcal{J}^*,1}), \sigma_{\mathcal{J}^*}) = 1)$$

and

$$\begin{aligned}\mathcal{BAD}_4^\diamond = \mathcal{BAD}_4^* &\wedge (\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_{\mathcal{J}^*,0}, ct_{\mathcal{J}^*,1}), \sigma_{\mathcal{J}^*}) = 1)\\ &\wedge (\mathsf{PC}.\mathtt{Verify}(crs_{\mathsf{PC}}^{(3)}, l, com^{(3)}, \mathcal{J}^*, \mathcal{C}_{\mathcal{J}^*}, \rho_{\mathcal{J}^*}^{(3)}) = 1) \wedge (\mathcal{J}^* \in [1, l])\end{aligned}$$

otherwise, it also aborts but outputs 6.

Let $\mathcal{H}_i$ be the output of $\mathtt{H}_i$ for $i \in [0, 5]$. We next show that $\Pr[\mathcal{H}_0 = 5] \leq negl(\lambda)$ via proving the following claims.

Claim B.18. $\Pr[\mathcal{H}_1 = 5] = \frac{\Pr[\mathcal{H}_0 = 5]}{\mathcal{M}}$.

Proof. Assume that the event $\mathcal{BAD}_2$ or $\mathcal{BAD}_4$ occurs and let

$$(a, j) = \mathsf{PC}.\mathtt{Extract}(td_{\mathsf{PC}}^{(1)}, N, com^{(1)})$$

Then we have $a = 1$. Note that the extraction algorithm of PC will output either $(0,0)$ or $(1,j')$ for $j' \in [1,N]$ on input a vector length N, thus, we have $1 \leq j \leq N \leq \mathcal{M}$.

Now, as $\jmath^* \xleftarrow{\$} [1,\mathcal{M}]$ and it is independent of the experiment before performing the final check, the probability that $\jmath^* = j$ will be exactly $1/\mathcal{M}$ if the event $\mathcal{BAD}_2$ or $\mathcal{BAD}_4$ occurs, and therefore, we have

$$\Pr[\mathcal{H}_1 = 5] = \frac{\Pr[\mathcal{H}_0 = 5]}{\mathcal{M}}$$

□

Claim B.19. $|\Pr[\mathcal{H}_1 = 5] - \Pr[\mathcal{H}_2 = 5]| \leq negl(\lambda)$.

Proof. In both $\mathtt{H}_1$ and $\mathtt{H}_2$, the trapdoor td_{BI} is never used, thus indistinguishability between them comes from the index hiding property of BI directly. □

Claim B.20. $|\Pr[\mathcal{H}_2 = 5] - \Pr[\mathcal{H}_3 = 5]| \leq negl(\lambda)$.

Proof. If $\mathcal{BAD}'_2$ or $\mathcal{BAD}'_4$ occurs, we have

$$N = poly(\lambda) \wedge \mathsf{BI}.\mathtt{Verify}(crs_{\mathsf{BI}}, N, \mathtt{P}, \Pi) = 1 \wedge$$
$$\mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)}) = (1, \jmath^*)$$

Note that as $(1,\jmath^*)$ is the output of $\mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)})$, we have $\jmath^* \in [1,N]$. Then by the somewhere argument of knowledge property of BI, we have

$$\Pr[\mathtt{P}(\jmath^*, \mathcal{W}_{\jmath^*}) = 0] \leq negl(\lambda)$$

That is, if $\mathcal{BAD}'_2$ or $\mathcal{BAD}'_4$ occurs, then the new check introduced in Hybrid $\mathtt{H}_3$ can be passed with all but negligible probability and that completes the proof. □

Claim B.21. $|\Pr[\mathcal{H}_3 = 5] - \Pr[\mathcal{H}_4 = 5]| \leq negl(\lambda)$.

Proof. If $\mathcal{BAD}^\dagger_2$ or $\mathcal{BAD}^\dagger_4$ occurs, we have

$$\mathsf{PC}.\mathtt{Extract}(td^{(1)}_{\mathsf{PC}}, N, com^{(1)}) = (1, \jmath^*) \wedge$$
$$\mathtt{P}(\jmath^*, \mathcal{W}_{\jmath^*}) = 1$$

where $\mathcal{W}_{\jmath^*} \leftarrow \mathsf{BI}.\mathtt{Extract}(td_{\mathsf{BI}}, N, \mathtt{P}, \Pi)$. Let

$$\begin{aligned}\mathcal{W}_{\jmath^*} = (&\mathcal{C}_{\jmath^*} = (ct_{\jmath^*,0}, ct_{\jmath^*,1}, d_{\jmath^*}), \sigma_{\jmath^*}, \rho^{(1)}_{\jmath^*}, \rho^{(2)}_{\jmath^*}, \rho^{(3)}_{\jmath^*},\\ &L, \mathcal{C}_L = (ct_{L,0}, ct_{L,1}, d_L), \rho^{(1)}_L, \rho^{(2)}_L,\\ &R, \mathcal{C}_R = (ct_{R,0}, ct_{R,1}, d_R), \rho^{(1)}_R, \rho^{(2)}_R)\end{aligned}$$

Note that the circuit $\mathtt{P}$ will always verify if $\mathcal{C}_{\jmath^*}$ is committed in $com^{(1)}$ in both cases (i.e., the case that $\jmath^* \leq l$ and the case that $\jmath^* > l$), thus we have

$$\mathsf{PC}.\mathtt{Verify}(crs^{(1)}_{\mathsf{PC}}, N, com^{(1)}, \jmath^*, \mathcal{C}_{\jmath^*}, \rho^{(1)}_{\jmath^*}) = 1$$

Then by the extractability of PC, we have

$$\Pr[\mathtt{C}_c[sk_0, sk_1, 0](\mathcal{C}_{\mathcal{J}^*}) = 0] \leq negl(\lambda)$$

That is, if $\mathcal{BAD}_2^\dagger$ or $\mathcal{BAD}_4^\dagger$ occurs, then the new check introduced in Hybrid $\mathtt{H}_4$ can be passed with all but negligible probability and that completes the proof. □

Claim B.22. $\Pr[\mathcal{H}_4 = 5] = \Pr[\mathcal{H}_5 = 5]$.

Proof. If $\mathcal{BAD}_2^*$ or $\mathcal{BAD}_4^*$ occurs, we have

$$\mathtt{P}(\mathcal{J}^*, \mathcal{W}_{\mathcal{J}^*}) = 1 \wedge \mathtt{C}_c[sk_0, sk_1, 0](\mathcal{C}_{\mathcal{J}^*}) = 1$$

where $\mathcal{W}_{\mathcal{J}^*} \leftarrow \mathsf{BI}.\mathtt{Extract}(td_{\mathsf{BI}}, N, \mathtt{P}, \Pi)$. Let

$$\begin{aligned}\mathcal{W}_{\mathcal{J}^*} = (&\mathcal{C}_{\mathcal{J}^*} = (ct_{\mathcal{J}^*,0}, ct_{\mathcal{J}^*,1}, d_{\mathcal{J}^*}), \sigma_{\mathcal{J}^*}, \rho^{(1)}_{\mathcal{J}^*}, \rho^{(2)}_{\mathcal{J}^*}, \rho^{(3)}_{\mathcal{J}^*},\\ &L, \mathcal{C}_L = (ct_{L,0}, ct_{L,1}, d_L), \rho^{(1)}_L, \rho^{(2)}_L,\\ &R, \mathcal{C}_R = (ct_{R,0}, ct_{R,1}, d_R), \rho^{(1)}_R, \rho^{(2)}_R)\end{aligned}$$

Since $\mathtt{C}_c[sk_0, sk_1, 0](\mathcal{C}_{\mathcal{J}^*}) = 1$, we have $d_{\mathcal{J}^*} \leq 0$. Then we cannot have $\mathcal{J}^* > l$ as the circuit $\mathtt{P}$ requires that $d_{\mathcal{J}^*} \geq 1$ when $\mathcal{J}^* > l$. Now, since $\mathcal{J}^* \leq l$ and $\mathtt{P}(\mathcal{J}^*, \mathcal{W}_{\mathcal{J}^*}) = 1$, we always have

$$\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_{\mathcal{J}^*,0}, ct_{\mathcal{J}^*,1}), \sigma_{\mathcal{J}^*}) = 1$$

and

$$\mathsf{PC}.\mathtt{Verify}(crs^{(3)}_{\mathsf{PC}}, l, com^{(3)}, \mathcal{J}^*, \mathcal{C}_{\mathcal{J}^*}, \rho^{(3)}_{\mathcal{J}^*}) = 1$$

That is, if $\mathcal{BAD}_2^*$ or $\mathcal{BAD}_4^*$ occurs, then the additional checks introduced in Hybrid $\mathtt{H}_5$ will always be satisfied and therefore $\Pr[\mathcal{H}_5 = 5] = \Pr[\mathcal{H}_4 = 5]$. □

Claim B.23. $\Pr[\mathcal{H}_5 = 5] \leq negl(\lambda)$.

Proof. Let $\mathcal{W}_{\mathcal{J}^*} \leftarrow \mathsf{BI}.\mathtt{Extract}(td_{\mathsf{BI}}, N, \mathtt{P}, \Pi)$ and

$$\begin{aligned}\mathcal{W}_{\mathcal{J}^*} = (&\mathcal{C}_{\mathcal{J}^*} = (ct_{\mathcal{J}^*,0}, ct_{\mathcal{J}^*,1}, d_{\mathcal{J}^*}), \sigma_{\mathcal{J}^*}, \rho^{(1)}_{\mathcal{J}^*}, \rho^{(2)}_{\mathcal{J}^*}, \rho^{(3)}_{\mathcal{J}^*},\\ &L, \mathcal{C}_L = (ct_{L,0}, ct_{L,1}, d_L), \rho^{(1)}_L, \rho^{(2)}_L,\\ &R, \mathcal{C}_R = (ct_{R,0}, ct_{R,1}, d_R), \rho^{(1)}_R, \rho^{(2)}_R)\end{aligned}$$

If $\mathcal{BAD}_2^\diamond$ occurs, then we have

$$\mathtt{C}_c[sk_0, sk_1, 0](\mathcal{C}_{\mathcal{J}^*}) = 1 \wedge \mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_{\mathcal{J}^*,0}, ct_{\mathcal{J}^*,1}), \sigma_{\mathcal{J}^*}) = 1$$

Since $\mathtt{C}_c[sk_0, sk_1, 0](\mathcal{C}_{\mathcal{J}^*}) = 1$, we have

$$\mathsf{HE}.\mathtt{Dec}(sk_0, ct_{\mathcal{J}^*,0}) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_{\mathcal{J}^*,1})$$

Note that if $(pk_0, pk_1, ct_{\mathcal{J}^*,0}, ct_{\mathcal{J}^*,1}) \in \mathcal{L}$, then we will always have $\mathsf{HE}.\mathtt{Dec}(sk_0, ct_{\mathcal{J}^*,0}) = \mathsf{HE}.\mathtt{Dec}(sk_1, ct_{\mathcal{J}^*,1})$ due to the standard correctness of HE. Therefore,

if $\mathcal{BAD}_2^\diamond$ occurs, we find an instance $(pk_0, pk_1, ct_{\jmath^*,0}, ct_{\jmath^*,1})$ that is not in $\mathcal{L}$ and a valid proof $\sigma_{\jmath^*}$ for it, before seeing any simulated proofs. Then by the simulation-soundness of ZK, $\mathcal{BAD}_2^\diamond$ can occur with only negligible probability.

Next, we show that the probability that $\mathcal{BAD}_4^\diamond$ occurs is also negligible. Again, assume that $\mathcal{BAD}_4^\diamond$ occurs, then we have

$$\begin{aligned}
&\mathtt{C}_c[sk_0, sk_1, 0](\mathcal{C}_{\jmath^*}) = 1 \wedge \mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ct_{\jmath^*,0}, ct_{\jmath^*,1}), \sigma_{\jmath^*}) = 1 \wedge \\
&\mathsf{PC}.\mathtt{Verify}(crs^{(3)}_{\mathsf{PC}}, l, com^{(3)}, \jmath^*, \mathcal{C}_{\jmath^*}, \rho^{(3)}_{\jmath^*}) = 1 \wedge \jmath^* \in [1, l] \wedge \\
&L = l \wedge \mathsf{PC}.\mathtt{Commit}(crs^{(3)}_{\mathsf{PC}}, L, (ict_{i,0}, ict_{i,1}, 0)_{i\in[1,L]}) = (com^{(3)}, *) \wedge \\
&CT^* \neq CT_{\jmath^*} \wedge \mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ict_{\jmath^*,0}, ict_{\jmath^*,1}), i\sigma_{\jmath^*}) = 1
\end{aligned}$$

We first show the following facts:

Claim B.23.1. $\Pr[\mathcal{BAD}_4^\diamond \wedge (ict_{\jmath^*,0}, ict_{\jmath^*,1}) \neq (ct_{\jmath^*,0}, ct_{\jmath^*,1})] \leq negl(\lambda)$

Proof. If $\mathcal{BAD}_4^\diamond$ occurs, we have $\jmath^* \in [1, l]$ and $L = l$. Let

$$(com, st) \leftarrow \mathsf{PC}.\mathtt{Commit}(crs^{(3)}_{\mathsf{PC}}, L, (ict_{i,0}, ict_{i,1}, 0)_{i\in[1,L]})$$

$$\rho \leftarrow \mathsf{PC}.\mathtt{Open}(crs^{(3)}_{\mathsf{PC}}, L, st, \jmath^*)$$

then we have

$$\mathsf{PC}.\mathtt{Verify}(crs^{(3)}_{\mathsf{PC}}, L, com, \jmath^*, (ict_{\jmath^*,0}, ict_{\jmath^*,1}, 0), \rho) = 1$$

due to the correctness of PC.

Next, since $L = l$, $com = com^{(3)}$, and

$$\mathsf{PC}.\mathtt{Verify}(crs^{(3)}_{\mathsf{PC}}, l, com^{(3)}, \jmath^*, \mathcal{C}_{\jmath^*}, \rho^{(3)}_{\jmath^*}) = 1$$

(assuming $\mathcal{BAD}_4^\diamond$ occurs), we have $\mathcal{C}_{\jmath^*} = (ict_{\jmath^*,0}, ict_{\jmath^*,1}, 0)$, i.e.,

$$(ict_{\jmath^*,0}, ict_{\jmath^*,1}) = (ct_{\jmath^*,0}, ct_{\jmath^*,1})$$

with all but negligible probability due to the binding property of PC. □

Claim B.23.2. $\Pr[\mathcal{BAD}_4^\diamond \wedge (ict_{\jmath^*,0}, ict_{\jmath^*,1}) = (ct_0^*, ct_1^*)] \leq negl(\lambda)$ *(Recall that* $CT^* = (0, ct_0^*, ct_1^*, \sigma^*)$*)*

Proof. If $\mathcal{BAD}_4^\diamond$ occurs and $(ict_{\jmath^*,0}, ict_{\jmath^*,1}) = (ct_0^*, ct_1^*)$, we have

$$\mathsf{ZK}.\mathtt{Verify}(crs_{\mathsf{ZK}}, (pk_0, pk_1, ict_{\jmath^*,0}, ict_{\jmath^*,1}), i\sigma_{\jmath^*}) = 1$$

$$(ct_0^*, ct_1^*, \sigma^*) \neq (ict_{\jmath^*,0}, ict_{\jmath^*,1}, i\sigma^*_{\jmath^*})$$

and

$$(pk_0, pk_1, ict_{\jmath^*,0}, ict_{\jmath^*,1}) \notin \mathcal{L}$$

where the last equation comes from the fact that $(ict_{\jmath^*,0}, ict_{\jmath^*,1}) = (ct_0^*, ct_1^*)$ and $(pk_0, pk_1, ct_0^*, ct_1^*) \notin \mathcal{L}$. This will occur with only negligible probability due to the simulation soundness of ZK. □

Combining Claim B.23.1 and Claim B.23.2, we have

$$\begin{aligned}
&\Pr[\mathcal{BAD}_4^{\diamond} \wedge (ct_{\jmath^*,0}, ct_{\jmath^*,1}) = (ct_0^*, ct_1^*)] \\
=&\Pr[\mathcal{BAD}_4^{\diamond} \wedge (ct_{\jmath^*,0}, ct_{\jmath^*,1}) = (ct_0^*, ct_1^*) \wedge (ict_{\jmath^*,0}, ict_{\jmath^*,1}) \neq (ct_{\jmath^*,0}, ct_{\jmath^*,1})] \\
&+ \Pr[\mathcal{BAD}_4^{\diamond} \wedge (ct_{\jmath^*,0}, ct_{\jmath^*,1}) = (ct_0^*, ct_1^*) \wedge (ict_{\jmath^*,0}, ict_{\jmath^*,1}) = (ct_{\jmath^*,0}, ct_{\jmath^*,1})] \\
\leq& negl(\lambda)
\end{aligned} \tag{15}$$

Next, we assume that $\mathcal{BAD}_4^{\diamond}$ occurs and $(ct_{\jmath^*,0}, ct_{\jmath^*,1}) \neq (ct_0^*, ct_1^*)$. Since $\mathtt{C}_c[sk_0, sk_1, 0](\mathcal{C}_{\jmath^*}) = 1$, we have

$$\mathsf{HE}.\mathtt{Dec}(sk_0, ct_{\jmath^*,0}) \neq \mathsf{HE}.\mathtt{Dec}(sk_1, ct_{\jmath^*,1})$$

and therefore $(pk_0, pk_1, ct_{\jmath^*,0}, ct_{\jmath^*,1}) \notin \mathcal{L}$. Moreover, $\sigma_{\jmath^*}$ is a valid proof for $(pk_0, pk_1, ct_{\jmath^*,0}, ct_{\jmath^*,1}) \in \mathcal{L}$. In addition, although a simulated proof σ^* for an invalid instance $(pk_0, pk_1, ct_0^*, ct_1^*)$ is given, we have

$$((pk_0, pk_1, ct_{\jmath^*,0}, ct_{\jmath^*,1}), \sigma_{\jmath^*}) \neq ((pk_0, pk_1, ct_0^*, ct_1^*), \sigma^*)$$

That means, assuming $\mathcal{BAD}_4^{\diamond}$ occurs and $(ct_{\jmath^*,0}, ct_{\jmath^*,1}) \neq (ct_0^*, ct_1^*)$, we find (again) an instance $(pk_0, pk_1, ct_{\jmath^*,0}, ct_{\jmath^*,1})$ that is not in $\mathcal{L}$ and a valid proof $\sigma_{\jmath^*}$ for it, and the instance and the proof is not the one simulated by the simulator. Then by the simulation-soundness of ZK, this will happen with only negligible probability, i.e.,

$$\Pr[\mathcal{BAD}_4^{\diamond} \wedge (ct_{\jmath^*,0}, ct_{\jmath^*,1}) \neq (ct_0^*, ct_1^*)] \leq negl(\lambda) \tag{16}$$

From Equation (15) and Equation (16), we have $\Pr[\mathcal{BAD}_4^{\diamond}] \leq negl(\lambda)$.

Since each of $\mathcal{BAD}_2^{\diamond}$ and $\mathcal{BAD}_4^{\diamond}$ will occur with only negligible probability, $\mathtt{H}_5$ will output 5 with only negligible probability and that completes the proof. □

Combining Claim B.18 to Claim B.23, we have $\Pr[\mathcal{H}_0 = 5] \leq negl(\lambda)$, i.e., the bad events will occur in $\mathtt{Exp}_0$ with only negligible probability. Therefore, $\mathtt{Exp}_0$ will output 4 with only negligible probability and that completes the proof of Lemma B.12.